

STRESZCZENIE

W dzisiejszych czasach bardzo często stosowane są rozwiązania, które wykorzystują wykrywanie mowy. Istnieje wiele asystentów głosowych, aplikacji przetwarzających sygnały dźwiękowe na tekst, itp. W ostatnich czasach popularna stała się wszelako rozumiana komunikacja pomiędzy człowiekiem a maszyną. Z tego względu niniejsza praca opisuje możliwe solucje dla problemu wykrywania i śledzenia emocji, które mogłyby zostać wykorzystane do efektywniejszej interakcji oraz zrozumienia. Opisane zostały najpopularniejsze metody ekstrakcji cech z sygnałów dźwiękowych, używane do detekcji algorytmy; przybliżono informacje na temat znanych podziałów emocji, a także przedstawiono, w jaki sposób stworzona została aplikacja webowa. W pracy szczegółowo przeanalizowano wytrenowane dwa modele używające sieci CNN, gdzie jeden z nich używa do ekstrakcji cech wyłącznie MFCC, jeden oparty o LSTM, oraz własną propozycję, model Transformer. Otrzymane rozwiązania wykazały wysoką dokładność dla zbiorów testowych oraz spisały się pozytywnie podczas testu na specjalnie przygotowanym pliku audio. Wymieniono także odkryte w nich wady i ograniczenia. Na zakończenie podsumowano i zaproponowano możliwe dalsze prace związane z rozwojem utworzonego modułu.

Słowa kluczowe: detekcja emocji z głosu, przetwarzanie sygnałów dźwiękowych, ekstrakcja cech, śledzenie emocji, uczenie maszynowe, CNN, LSTM, MFCC, model Transformer

Dziedzina nauki i techniki zgodna z OECD Nauki inżynieryjne i techniczne, Robotyka i Automatyka

ABSTRACT

Nowadays, solutions using speech detection are very common. There are many voice assistants, applications that convert audio signals into text, etc. Recently human-machine communication is becoming more and more popular. For these reasons the paper describes possible solutions for the problem of detecting and tracking emotions, which could be used for more effective interaction and understanding. It described most popular methods of feature extraction from audio signals, used for detection algorithms, divisions of emotions and how the application was created. The paper analyzed in detail trained two models using CNNs, where one of them uses only MFCC for feature extraction, one based on LSTM, and its own proposal, the Transformer model. Results showed high accuracy for test sets and performed positively when tested on a specially prepared audio file. The flaws and limitations discovered in any model are also described. Finally, there are informations about possible further development of the created module.

Keywords: speech emotion detection, sound record processing, feature extraction, emotion tracking, machine learning, CNN, LSTM, MFCC, Transformer model

OECD consistent field of science and technology classification: Nauki inżynieryjne i techniczne, Robotyka i Automatyka

SPIS TREŚCI

1 WSTĘP I CEL PRACY (JAKUB MAKOWSKI)	9
1.1 Cel i zakres pracy	10
1.2 Układ pracy	10
2 TEORIA EMOCJI (JAKUB MAKOWSKI)	11
3 TEORIA PRZETWARZANIA DŹWIĘKU (JAKUB BIEŃKOWSKI)	14
3.1 Elementarne cechy dźwięku	14
3.2 Podstawowe cechy głosu	15
3.3 Metody ekstrakcji cech z próbki głosowej	16
3.3.1 Dyskretna transformata falkowa	17
3.3.2 LPC	18
3.3.3 PLP	20
3.3.4 MFCC	22
3.4 Porównanie przedstawionych metod	23
4 ASPEKTY UCZENIA MASZYNOWEGO (JAKUB MAKOWSKI)	25
4.1 Wstęp do sieci neuronowych	26
4.2 Konwolucyjne sieci neuronowe	26
4.3 Rekurencyjne sieci neuronowe	27
4.4 Long Short-Term Memory	28
4.5 Model oparty o Transformer (Jakub Bieńkowski)	30
5 PRZEGLĄD ROZWIĄZAŃ (JAKUB BIEŃKOWSKI, JAKUB MAKOWSKI)	34
5.1 Podział emocji (Jakub Makowski)	34
5.2 Przegląd datasetów (Jakub Makowski)	34
5.3 Preprocessing danych (Jakub Bieńkowski)	35
5.4 Modele wykrywania emocji	37
5.5 Podsumowanie	39
6 OPIS ROZWIĄZAŃ (JAKUB MAKOWSKI)	40
6.1 Użyte zbiory danych	40
6.2 Model oparty o CNN	42
6.2.1 Dane uczące	42
6.2.2 Opis działania	43
6.3 Model oparty o LSTM	44
6.3.1 Dane uczące	44
6.3.2 Opis działania	45

6.4	Model oparty o MFCC	46
6.4.1	Dane uczące	46
6.4.2	Opis działania	47
6.5	Model oparty o transformer (Jakub Bieńkowski)	48
6.5.1	Dane uczące	49
6.5.2	Opis działania	50
6.6	Aplikacja webowa	51
7	WYNIKI (JAKUB MAKOWSKI)	53
7.1	Trenowanie modeli	53
7.2	Test dokładności	56
7.3	Podsumowanie	57
8	ZAKOŃCZENIE (JAKUB BIEŃKOWSKI, JAKUB MAKOWSKI)	58
8.1	Ograniczenia i problemy	58
8.2	Zastosowania	58
8.3	Możliwy dalszy rozwój	58
	SPIS RYSUNKÓW	61
	SPIS TABEL	62
	BIBLIOGRAFIA	63
	DODATEK A. OPIS DYPLOMU (JAKUB MAKOWSKI)	68
	DODATEK B. INSTRUKCJA DLA UŻYTKOWNIKA (JAKUB BIEŃKOWSKI)	73
	DODATEK C. INSTRUKCJA DLA PROGRAMISTY (JAKUB MAKOWSKI)	74

LISTA SYMBOLI

x_i	- i -ta wartość próbki
N	- liczba próbek w sygnale, długość ramki
H	- przesunięcie ramki
M	- nakładanie ramki
T	- całkowita liczba próbek dla danej ramki
s_t	- wartość próbki sygnału w chwili t
s	- współczynnik skali
$\psi()$	- funkcja falkowa
n	- próbka sygnału
$x(n)$	- sygnał dyskretny
l	- przesunięcie
$W(l, s)$	- wynik dyskretnej transformaty falkowej
$R(k)$	- Wartość autokorelacji
k	- opóźnienie, kolejne iteracje
α	- wektor współczynników LPC
R	- macierz otrzymana podczas obliczania estymatorów funkcji autokorelacji
r	- wektor autokorelacji
E	- błąd predykcji
Ω	- częstotliwość
$P_w(\Omega)$	- widmo mocy
$ReS_w(\Omega)$	- część rzeczywista widma
$ImS_w(\Omega)$	- część urojona widma
w	- pulsacja
$\psi(\Omega)$	- funkcja preemfazy
$\theta(\Omega_i)$	- konwolucja z krzywą maskującą krytycznego pasma
$E_w(\Omega)$	- energia skompresowanego sygnału
k_n	- wartości stałe
x_n	- sygnał przed preemfazą
x'_n	- sygnał po preemfazie
α	- stały współczynnik
$HAM(N)$	- funkcja Hamminga
S_m	- energia pasma
m	- numer filtra
X_r	- widmo ramki
$H_m(k)$	- zestaw filtrów
c_i	- współczynniki MFCC

M	- liczba użytych filtrów
i	- numer współczynnika
$(f * g)(t)$	- splot funkcji
t	- zmienna niezależna
τ	- przesunięcie
$d\tau$	- element całkowania
σ	- funkcja sigmoidalna
x	- sygnały wejściowe
h	- sygnał wyjściowy
i	- bramka wejściowa
f	- bramka "zapominania"
o	- bramka wyjściowa
C	- komórka pamięci
W, b	- macierze wag
t	- indeks aktualnej wartości

LISTA SKRÓTÓW

AI	- Sztuczna inteligencja (ang. <i>Artificial Intelligence</i>)
HCI	- Interakcja człowiek-komputer (ang. <i>Human-Computer Interaction</i>)
SER	- Rozpoznawanie emocji o sygnale dźwiękowym (ang. <i>Speech Emotion Recognition</i>)
CNN	- Konwolucyjna sieć neuronowa (ang. <i>Convolutional Neural Network</i>)
LSTM	- Long Short-Term Memory
MFCC	- Parametry mel-cepstralne (ang. <i>Mel Frequency Cepstral Coefficients</i>)
ML	- Uczenie maszynowe (ang. <i>Machine Learning</i>)
SVM	- Metoda wektorów nośnych (ang. <i>Support Vector Machines</i>)
RBF	- Radialne funkcje bazowe (ang. <i>Radial Basis Function</i>)
DNN	- Głęboka sieć neuronowa (ang. <i>Deep Neural Network</i>)
RNN	- Rekurencyjna sieć neuronowa (ang. <i>Recurrent Neural Network</i>)
DA	- Augumentacja danych (ang. <i>Data Augumentation</i>)
ReLU	- Rectified Linear Unit
NLP	- Przetwarzanie języka naturalnego (ang. <i>Natural Language Processing</i>)
CEC	- Stała karuzela błędów (ang. <i>Constant Error Carousel</i>)
ZCR	- wskaźnik przejść przez zero (ang. <i>Zero Crossing Rate</i>)
RMSE	- Energia średniokwadratowa (ang. <i>Root Mean Square Energy</i>)
GMM	- Model mieszanin Gaussowskich (ang. <i>Gaussian Mixture Model</i>)
LPCC	- Współczynniki cepstralne liniowej predykcji (ang. <i>Linear Prediction Cepstral Coefficients</i>)
DWT	- Dyskretna transformata falkowa (ang. <i>Discrete wavelet Transform</i>)
LPC	- Predykcja liniowa (ang. <i>Linear Prediction coding</i>)
PLP	- percepcyjna liniowa predykcja (ang. <i>Perceptual Linear Prediction</i>)
HMM	- Ukryty model Markova (ang. <i>Hidden Markov Model</i>)
DFT	- Dyskretna transformata Fouriera (ang. <i>Discrete Fourier Transform</i>)
IEMOCAP	- Interactive Emotional Dyadic Motion Capture
RAVDESS	- Ryerson Audio-Visual Database of Emotional Speech and Song
TESS	- Toronto Emotional Speech Set
CREMA-D	- Crowd-sourced Emotional Multimodal Actors Dataset
SAVEE	- Surrey Audio-Visual Expressed Emotion
HDF5	- Hierarchical Data Format version 5
BLEU	- Bilingual Evaluation Understudy
FLOP	- Floating Point Operations Per-Second
GPT	- Generative Pre-trained Transformer
BERT	- Bidirectional Encoder Representations from Transformers
GELU	- Gaussian Error Linear Units

1. WSTĘP I CEL PRACY (JAKUB MAKOWSKI)

W obecnych czasach zauważalny jest dynamiczny rozwój dziedziny ICT (ang. *Information and Communication Technologies*) z naciskiem na sztuczną inteligencję (AI - ang. *Artificial Intelligence*). Jesteśmy w stanie zaobserwować jej wpływ niemal w każdym aspekcie życia codziennego; od systemów rekomendacyjnych, przez tworzenie inteligentnych botów do gier, aż po zaawansowane algorytmy analizujące dane medyczne w celu diagnozy chorób. Rozwiązania z użyciem AI są powszechne. Stosują je nawet ludzie nie mający na co dzień do czynienia z branżą ICT. Przyczyniło się to do tworzenia coraz bardziej skomplikowanych algorytmów sztucznej inteligencji i stosowania jej za każdym razem, gdy stanowi to łatwiejszą oraz efektywniejszą formę rozwiązania problemu. Nie zawsze jednak rozwiązania z zastosowaniem AI okazują się najlepsze, gdyż do ich wykonania potrzebna jest często duża moc obliczeniowa. Czasami standardowe algorytmy radzą sobie z problemem szybciej i są przy tym mniej kosztowne.

W ostatnich latach zapotrzebowanie na komunikację między człowiekiem a maszyną znacząco wzrosło. Tworzy się wiele rozwiązań wykorzystujących interakcję pomiędzy człowiekiem i komputerem (HCI - ang. *Human-Computer Interaction*). Do najpopularniejszych rozwiązań należą systemy asystentów głosowych, takich jak na przykład: Alexa (firmy Amazon), czy też Siri (firmy Apple). Według *Insider Intelligence* w Ameryce do końca 2023 roku liczba użytkowników asystentów głosowych przekroczy 145 milionów [1]. Ze względu na ogromny potencjał takich interfejsów, prowadzony jest szereg badań mających na celu ich ulepszenie. Używane są one nie tylko w rozwiązaniach związanych z aspektami tzw. **smart home** (inteligentnym domu), lecz także wspomagają i ułatwiają życie osobom niepełnosprawnym [2]. Dzięki takim rozwiązaniom osoby, które nie są samodzielne, mogą zaoszczędzić czas oraz siły, na przykład poprzez obsługę telewizora głosem, bez konieczności szukania pilota.

Jednym z wielu problemów HCI jest brak zrozumienia uczuć człowieka ze strony komputera. Urządzenie nie zawsze jest w stanie zrozumieć, co człowiek chce przekazać w swojej wypowiedzi. Zrozumienie emocji, jakie odczuwa osoba w danym momencie, mogłoby ułatwić poprawną interpretację komputerowi i zapobiec, w dużej mierze, występowaniu nieporozumień. W tym celu rozpoczęto prowadzić szereg intensywnych badań zagłębiających się w temacie detekcji emocji z sygnałów dźwiękowych (SER - ang. *Speech Emotion Recognition*).

Dźwięk jest jednym z głównych nośników informacji o aktualnie odczuwanych emocjach człowieka. Cechy głosu takie jak: ton, tempo, modulacja i intensywność zawierają najważniejsze dane potrzebne do rozpoznania aktualnego stanu emocjonalnego rozmówcy. AI pozwala przeanalizować i wyekstrahować odpowiednie cechy głosu, a to umożliwia wykrycie emocji osoby mówiącej.

Tematem niniejszej pracy inżynierskiej jest stworzenie systemu wykorzystującego sieci neuronowe, w taki sposób, aby umożliwić detekcję emocji z wypowiedzi użytkownika. Praca omawia teoretyczne zagadnienia powiązane z uczeniem maszynowym, opisuje szczegółowo używane sieci neuronowe oraz podaje wady i zalety danych rozwiązań. Opisuje także proces przygotowywania danych do trenowania sieci, a także umożliwienia ich poprawnego działania w zbudowanej aplikacji. Proponuje rozwiązania problemu związanego z komunikacją człowiek-maszyna i przedstawia ich skuteczność.

1.1. Cel i zakres pracy

Celem pracy jest stworzenie modułu, pozwalającego na poprawną identyfikację wyrażonej emocji w sygnale dźwiękowym. Moduł oparty jest o wykorzystanie sztucznych sieci neuronowych, dokładniej sieci konwolucyjnej (CNN - *Convolutional Neural Network*), sieci LSTM (*Long Short-Term Memory Networks*), sieci typu transformer oraz parametrów mel-cepstralnych (MFCC - *Mel-Frequency Cepstral Coefficients*). Moduł powinien prawidłowo wykrywać emocje z wypowiedzi użytkownika w czasie rzeczywistym z mikrofonu lub po wgraniu pliku audio. Detekcja winna działać dla każdego z czterech wprowadzonych do aplikacji modeli.

1.2. Układ pracy

Praca rozpoczyna się opisem teorii emocji, w której przedstawione zostały znane podziały stanów emocjonalnych. W kolejnym rozdziale pochyłono się nad teorią przetwarzania dźwięku, gdzie opisane dokładnie zostały powszechnie stosowane techniki mające na celu ekstrakcję emocji. Następnie ukazano aspekty uczenia maszynowego, w szczególności uczenia z nadzorem. Podane zostały tam dokładne informacje o najpopularniejszych modelach i algorytmach. Kolejno przybliżono przykładowe prace, w których użyto niektóre z opisanych wcześniej metod. Po czym zostały przedstawione cztery rozwiązania problemu, modele stworzone z użyciem kolejno sieci CNN, sieci LSTM, parametrów mel-cepstralnych oraz sieci typu transformer. Zawarto tam informacje o użytych datasetach, dokładne opisy poszczególnych rozwiązań, a także krótko przedstawiono informacje na temat samej aplikacji. Rozdział następny zawiera rezultaty oraz analizę testów przeprowadzonych na modelach. Porównane zostały wyniki każdego z rozwiązań, które testowane były na tym samym pliku audio, po czym wypisano wnioski. W zakończeniu podsumowano całą pracę, wymieniono napotkane w trakcie jej realizacji ograniczenia, jak również zaproponowano możliwe dalsze prace mające na celu rozwinięcie i ulepszenie rozwiązania.

2. TEORIA EMOCJI (JAKUB MAKOWSKI)

Z psychologicznego punktu widzenia emocje to złożone doświadczenie świadomości, odczuć cielesnych i zachowań, które odzwierciedla osobiste znaczenie rzeczy, wydarzenia lub stanu sprawy [3]. Z punktu widzenia naszej pracy możemy uznać emocje za sygnały, które mają na celu wywołać reakcję osoby odbierającej je lub wyrazić aktualny stan uczuć osoby mówiącej. Takie sygnały dostarczane również podczas mowy człowieka, są przekazywane przez tak zwane prozodia (brzmieniowe właściwości mowy). Kluczowymi atrybutami prozodii są wysokość, czas trwania i intensywność sygnału dźwiękowego mowy. Te cechy umożliwiają kodowanie informacji na temat intensywności, rytmu i intonacji, które determinują jaka emocja jest odzwierciedlana przez osobę mówiącą.

Podstawowymi emocjami możemy zdefiniować te, które są wrodzone, uniwersalne oraz odgrywają znaczącą rolę w przetrwaniu każdego człowieka [4]. Ekman w roku 1999 opracował listę pięciu podstawowych emocji, znanych również jako *emocje pierwotne* [5]. Każda z nich ma powiązane ze sobą unikalne cechy:

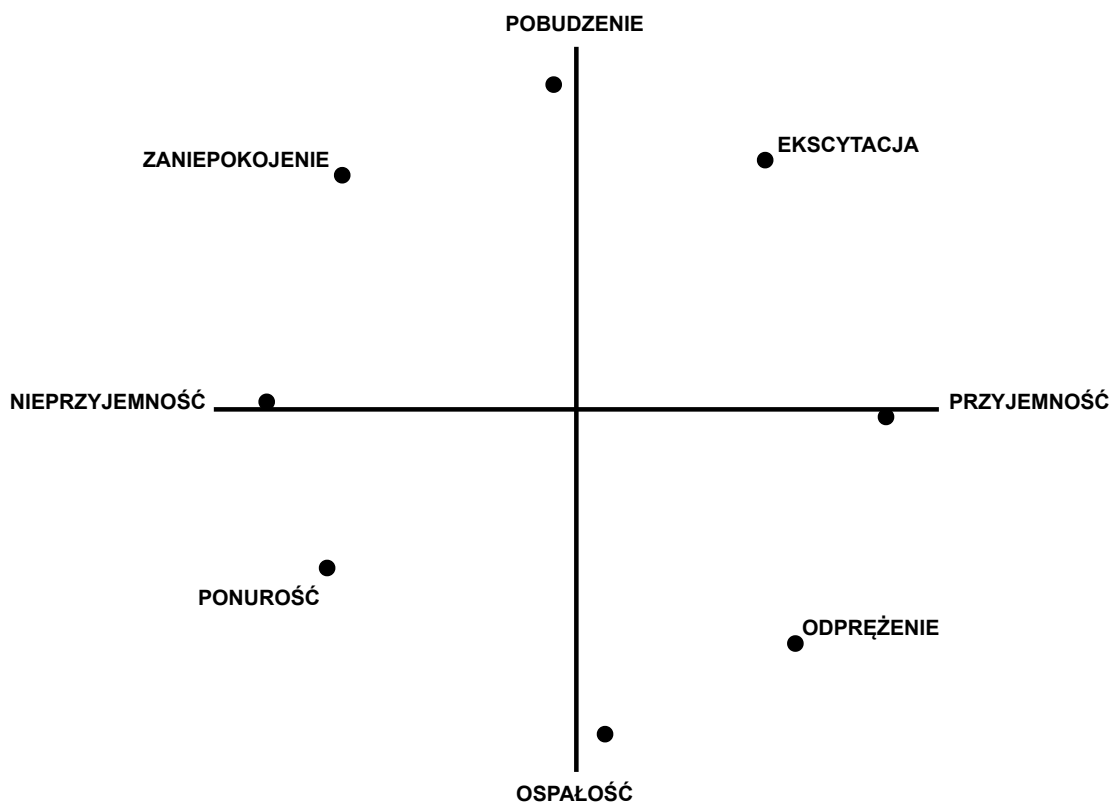
1. Złość – uznana została jako emocja negatywna łączona z odczuciem irytacji, frustracji czy wściekłości. Podczas niej osobie towarzyszą zazwyczaj energiczne gesty, silne napięcie mięśni w okolicach szczęki i czoła, szybkie tempo mówienia oraz głośne i ostre tony.
2. Obrzydzenie – sklasyfikowano jako wstręt, odrzucenie, czy dezaprobatę wobec czegoś. W trakcie zauważyć można wycofywanie się, zmarszczenie nosa, jak i unikanie słów związanych z odrzucającym czynnikiem.
3. Szczęście – zaliczono jako emocję pozytywną związaną z doświadczeniem przyjemności, odczuciem entuzjazmu. Osoba może wykazywać uśmiech, energiczne ruchy, jak i szybkie tempo mówienia połączone z radością w głosie oraz śmiechem.
4. Smutek – rozpoznano jako emocję negatywną powiązaną z poczuciem żalu, straty, czy przygnębienia. Człowieka odczuwającego smutek zazwyczaj cechuje opuszczona głowa, napięcie mięśniowe w okolicach ust i oczu, a także powolne tempo wypowiedzania słów w połączeniu z obniżeniem tonu.
5. Strach – powiązany został z reakcją na zagrożenie lub niebezpieczeństwo. Łączy się z powstrzymaniem się od ruchów, napięciem mięśni w okolicach szyi, przyspieszonymi oddechami oraz napięciem w głosie w połączeniu z wyższym tonem.

Oprócz wyżej zdefiniowanych przez Ekmana stanów emocjonalnych na potrzeby rozwiązań problemów związanych z detekcją emocji wiele prac zalicza do tego zbioru także [6]:

- Zaskoczenie – sklasyfikowane jako reakcję na nieoczekiwane, nagłe wydarzenie. Podczas niego towarzyszą osobie zazwyczaj niekontrolowane ruchy ciała, uniesione brwi wraz z szeroko otwartymi oczyma, jak i zdumienie w głosie połączone z przerywaniem mowy.

- Emocja neutralna – często nazywana również jako spokój. Osoba wykazująca ten stan emocjonalny nie odczuwa żadnej z wyżej wymienionych intensywnych emocji. Występuje często przy zrelaksowaniu oraz opanowaniu się od reakcji na bodźce, które mogłyby wywołać silne odczucia.

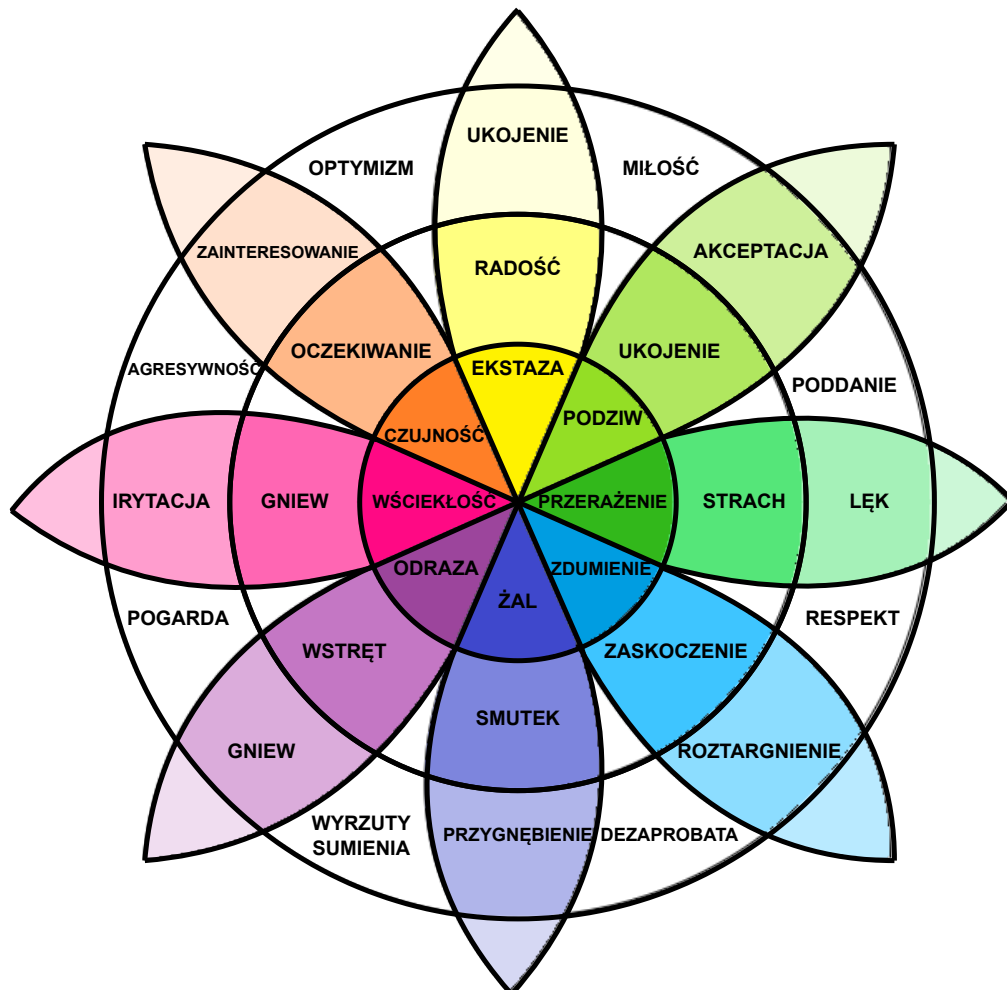
Podział emocji jako model kołowy zaproponowali w roku 1980 Russell oraz Pratt [7]. Skupili oni się na teorii, w której emocje podzielić można za pomocą wymiarów: przyjemność - nieprzyjemność (ang. *Pleasant - Unpleasant*), pobudzenie - ospałość (ang. *Arousing - Sleepy*). Za pomocą tych stanów utworzono model przypominający koło, które znane jest jako *circumplex model of affect*. Przedstawiony na rysunku 2.1 model został uporządkowany podział ośmiu deskryptorów emocjonalnych, gdzie różne emocje rozmieszczono na okręgu w zależności od ich oceny na tle wymienionych wymiarów [7].



Rysunek 2.1: Kołowy uporządkowany podział ośmiu deskryptorów afektywnych

W roku 1980 swoją teorię i propozycję podziału względem emocji przedstawił także Plutchik. Jego model opiera się na kołowym diagramie, przedstawionym na rysunku 2.2. Uznał on ośmiu podstawowych emocji: radość, zaufanie, strach, zaskoczenie, smutek, wstręt, gniew, a także oczekiwanie [8]. Każda z nich posiada swoje przeciwieństwo, usytuowane po drugiej stronie diagramu względem środka koła. Twierdził on, że emocje mogą się łączyć, a także każda z nich występować może w różnej intensywności, przez co tworzone są nowe odczucia, jak np. miłość, optymizm. Teoria ta cieszy się szerokim uznaniem w psychologii, a także znalazła zastosowanie w projektowaniu stron internetowych, czy też marketingu [9]. Niektórzy uważają jednak, że model ten może stanowić duże uproszczenie i nie uwzględnia złożoności ludzkich emocji.

W dziedzinie wykrywania emocji z sygnałów dźwiękowych oraz w naszej pracy największe zastosowanie znajduje pierwszy model, gdzie emocje podzielono na pojedyncze i unikalne. W uczeniu maszynowym propozycja ta wpasowuje się najbardziej ze względu na prostotę definicji. Nie komplikuje ona opisu jednej emocji jako połączenie kilku oraz umożliwia łatwe działanie na zbiorach danych. Dodatkowo większość datasetów aktualnie dostępnych używa właśnie ten podział, co proponuje to rozwiązanie jako najbardziej optymalne i najłatwiejsze w obsłudze.



Rysunek 2.2: Rozpięta paraboloida emocji Plutchika

W niniejszej pracy użyto model Ekmana wraz z jednym dodatkowym stanem emocjonalnym - emocja neutralna. Wybór ten nastąpił ze względu na prostą definicję emocji do użycia w uczeniu maszynowym, a także ze względu na duże zbiory danych używające taki właśnie podział emocji. Nie wzięto pod uwagę emocji zaskoczenia, gdyż często w datasetach znajduje się znacznie mniej nagrań tej emocji w porównaniu do pozostałych. Kolejne dwa modele podziału emocji okazują się być bardziej złożone i rzadko stosowane przy tworzeniu zbiorów danych. Ich użycie mogłoby skutkować trudniejszą definicją poszczególnych emocji, znalezieniem odpowiednich nagrań, a przez to mniejszą dokładnością modeli.

3. TEORIA PRZETWARZANIA DŹWIĘKU (JAKUB BIEŃKOWSKI)

Analiza sygnałów dźwiękowych jest ważnym elementem inżynierii dźwięku, która łączy w sobie elementy matematyki, fizyki i informatyki. Używa się jej w celu odkrycia informacji na temat struktury, charakterystyk i zachowań tych sygnałów. Jedną z powszechnie stosowanych metod analizy sygnałów jest transformata Fouriera, która umożliwia reprezentację sygnału w dziedzinie częstotliwości [10].

Dźwięk jest zaburzeniem mechanicznym w ośrodku sprężystym, gazowym, ciekłym lub stałym (fale sprężyste) wywołującym subiektywne wrażenie słuchowe u człowieka lub zwierząt [11]. Natomiast mowa jest przykładem sygnału niestacjonarnego, która cechuje się tym, że charakterystyki statyczne (wartość średnia, wartość średnia kwadratowa, funkcja korelacji) są zależne od wyboru chwili początkowej [12][13]. Głos jest zjawiskiem akustycznym wynikającym z drgań fałd głosowych w krtani, wzmacnianych i kształtowanych przez jamę ustną, gardło oraz zatoki nosowe. Jego charakterystyka obejmuje cechy prozodyczne i spektralne. Prozodia są brzmieniowymi właściwościami mowy, które nakładane są na głoskowy, sylabiczny i wyrazowy ciąg wypowiedzi. Cechy wpływające na prozodia to: intonacja, rytmiczność, tempo mówienia oraz pauzy w wypowiedzi. Natomiast do cech spektralnych należą charakterystyki głosu, które można zmierzyć za pomocą analizy widmowej. W ich skład wchodzi: formanty, czyli wzmocnione pasma częstotliwości w spektrum dźwięku, wysokość dźwięku, barwa głosu oraz energia. Ludzki głos natomiast jest doskonałym przykładem sygnału niestacjonarnego, ze względu na wykazywanie wielu zmiennych cech jak tonacja i intensywność, co oznacza zmianę jego częstotliwości i amplitudy w czasie. Poniżej dokładnie opisano podstawowe cechy dźwięku i głosu oraz metody najczęściej używane podczas ekstrakcji cech z sygnałów dźwiękowych.

3.1. Elementarne cechy dźwięku

Wysokość tonu to cecha pozwalająca na rozróżnienie tonu wysokiego od niskiego. Jest to liczba cykli wibracji na sekundę, mierzona w hercach. Mimo, że wysokość tonu, może być interpretowana jako częstotliwość, to jednak oba pojęcia nie są jednoznaczne. Wysokość tonu może być określana dla dźwięku, który ma częstotliwość możliwą do rozróżnienia od szumu [14].

Amplituda to siła dźwięku związana z energią fali dźwiękowej. Jest nią miara wahań ciśnienia powietrza spowodowanych dźwiękiem, a człowiek jest w stanie określić głośność tego dźwięku. Jako jednostkę amplitudy przyjmuje się decybel. Przyjmuje się tą miarę ze względu na to, że ludzkie ucho reaguje na dźwięk w sposób zbliżony bardziej do skali logarytmicznej niż liniowej [15].

Barwa dźwięku (inaczej tembr) jest to cecha pozwalająca odróżnić dwa dźwięki o tej samej wysokości i głośności, ale pochodzące z różnych źródeł. Zależy ona od harmonicznosci i częstotliwościowych składników w dźwięku. Jakość tembru jest określana na podstawie harmonicznosci, dlatego dźwiękom takim jak muzyka przypisuje się wysoką jakość tembru, a na przykład odgłosom ulicy niską [16].

Trwałość jest to ilość czasu trwania dźwięku. Trwałość tonu ma wpływ na tembr i rytm dźwięku. W fizyce czas trwania dźwięku zaczyna się w momencie zarejestrowania go i kończy się, gdy już nie jest on możliwy do wykrycia [14].

3.2. Podstawowe cechy głosu

Intonacja (inaczej wysokość dźwięku) jest jednym z najważniejszych elementów składających się na analizę dźwiękową. Informacje zawarte w intonacji, takie jak: ton podstawowy (czyli średnia częstotliwość wibracji strun głosowych), zakres tonacji (inaczej amplituda częstotliwości osoby mówiącej), czy jitter i shimmer (inaczej: nieregularności w głosie człowieka, gdzie jitter odnosi się do szybkich zmian w częstotliwości, a shimmer do zmian w amplitudzie) pozwalają na analizę głosową. Jak opisuje artykuł pt. "Acoustic profiles in vocal emotion expression" w przypadku częstotliwości głosu wyższy ton, czy średnio wyższa tonacja często łączona jest z pozytywnymi odczuciami, jak radość, czy ekscytacja. Natomiast bardziej płaski wykres częstotliwości może świadczyć o negatywnych emocjach, takich jak smutek, czy wściekłość lub oznacza, że na tej podstawie nie możemy wnioskować o emocji [17].

Kolejnym z podstawowych cech głosu jest tempo mowy, które stanowi szybkość wypowiedzania się osoby. Jest ono mierzone liczbą głosek, sylab, wyrazów na jednostkę czasu. Wyróżnia się trzy rodzaje tempa mowy: wolne (wł. *lento*), umiarkowane (wł. *moderato*) i szybkie (wł. *allegro*). Tempo mówienia osoby może determinować wiele rzeczy, np.: wiek osoby mówiącej, zazwyczaj osoba starsza mówi wolniej i robi dłuższe pauzy, przy czym osoba młodsza mówi szybciej. Wypowiedź szybka z krótkimi przerwami, może oznaczać radość, czy entuzjazm, a wypowiedź wolniejsza z dłuższymi pauzami prawdopodobnie oznacza smutek lub zmartwienie. Zmiany w tempie mowy mogą być spowodowane różnymi stanami psychofizjologicznymi mówcy.

Następnym zaliczanym do podstawowych cech jest natężenie głosu, które jest cechą odnoszącą się do głośności, z jaką osoba mówi, która może być mierzona w skali decybelowej. Każda osoba ma indywidualny profil natężenia głosu, który zmienia się wraz z wiekiem. Na jego podstawie jesteśmy w stanie określić wiek, płeć i stan zdrowia osoby mówiącej [18]. Głośność wypowiedzi można też skojarzyć ze wskaźnikiem emocji, a mianowicie większe natężenie koreluje z emocjami, typu gniew, czy ekscytacja; natomiast mniejsze kojarzyć można ze smutkiem [19].

Następny zaliczany do tego zbioru jest rytm głosu, który jest wzorcem, w jakim dźwięki i pauzy są rozmieszczone w czasie trwania wypowiedzi rozmówcy. Zmiany w natężeniu i tempie mowy powodują modyfikacje rytmu głosu. Na rytm składają się cechy, takie jak: tempo, sposób akcentowania (nacisk na określone sylaby lub słowa) i intonacja. Rytm jest elementem, który wpływa na percepcję osoby mówiącej, pokazując, czy jest ona pewna siebie, zrelaksowana, czy nerwowa. Na podstawie rytmu głosu jesteśmy w stanie również wywnioskować stan emocjonalny mówcy. Gdy osoba mówiąca ma monotony rytm, może świadczyć to o apatii, smutku lub zmęczeniu, a gdy zmienny rytm – o niepokoju, złości, czy agresji [19].

Fundamentalny element analizy dźwięku stanowi również energia. Jej podstawowym aspektem w dziedzinie sygnałów dźwiękowych jest amplituda. Im wyższe wartości amplitudy otrzymujemy tym większą energię dźwięku możemy zaobserwować. Dzięki obliczeniu energii dźwięku dla każdej pojedynczej ramki sygnału jesteśmy w stanie uzyskać pełną statystykę na temat całej próbki mowy. Wszystkie powyższe wymienione cechy wpływają na energię głosu.

3.3. Metody ekstrakcji cech z próbki głosowej

RMSE (ang. *Root Square Mean Energy*) jest powszechnie używaną miarą do kwantyfikacji średniej mocy sygnału, która obliczana jest na podstawie poniższego wzoru:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (3.1)$$

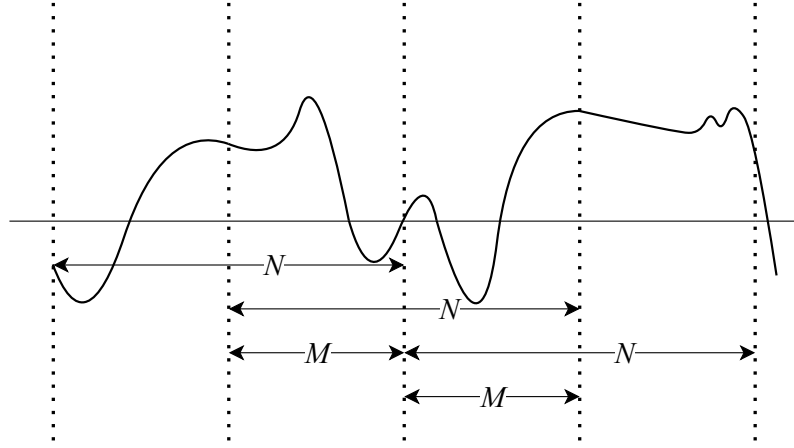
gdzie N jest liczbą próbek w sygnale, a x_i reprezentuje i -tą wartość próbki [20]. Jest ona często stosowana w produkcji muzyki, ponieważ daje lepszy obraz ogólnej głośności całego utworu, co później może być dostosowane do standardów danego gatunku muzycznego. Pozwala też na bardziej subtelne kształtowanie dynamiki sygnału, dlatego używana jest również często przy produkcjach filmowych. W detekcji emocji jest to stosowane jako wskaźnik intensywności wypowiedzi, co może świadczyć o wyższym poziomie ekscytacji, bądź gniewu. RMSE jest zazwyczaj jednym z elementów ekstrakcji cech, który dołączany jest razem z innymi technikami ekstrakcji pozwalających na kompleksową analizę stanów emocjonalnych. Poprzez wykorzystanie tego mechanizmu wraz z metodą analizującą cechy prozodyczne i spektralne, jako wejście do algorytmów uczenia maszynowego, możemy dokładniej analizować wzorce danych emocji.

Ramkowanie, które przedstawiono na rysunku 3.1, jest techniką przetwarzania sygnałów, polegającą na podziale oryginalnego sygnału na bloki, często nazywane ramkami, o długości N , z nakładaniem M i przesunięciem ramki H , gdzie

$$H = N - M. \quad (3.2)$$

Ramkowanie jest przydatne, ponieważ mowa jest niestacjonarnym sygnałem, a jego właściwości powinny być ekstrahowane z małych bloków sygnału [21]. Proces tej metody obejmuje konwertowanie wartości skoku i długości ramki z jednostek czasowych na próbki, obliczenia oczekiwanej liczby ramek, poszerzenia sygnału w razie potrzeby wypełnienia ram oraz wyliczenia ich indeksów i pobrania.

Okienkowanie (ang. *Windowing*) - w momencie wycinania z sygnału bloku próbek (ramkowania), powstaje zjawisko przecieku widma. Aby temu zapobiec stosujemy funkcje okna. Są one operatorami matematycznymi wpływającymi na kształt oraz właściwości widma analizowanej ramki sygnału. Funkcja powinna mieć tę samą długość, co ramka, aby nałożenie jej polegało na przemnożeniu konkretnych elementów obu wektorów przez siebie [22]. Do wykonania okienkowania mamy do wyboru różne funkcje takie jak: funkcja prostokątna, Hamminga, Von Hanna lub Blackmana. Nie istnieje uniwersalna funkcja i wybór jest kompromisem między tłumieniem listków bocznych a szerokością prążka [23].



Rysunek 3.1: Wykres obrazujący ramkowanie [21]

ZCR (ang. *Zero Crossing Rate*) polega na zliczaniu ile razy sygnał przechodzi przez poziom zero dla określonej ramki czasowej [24]. Obliczanie wskaźnika przejść przez zero odbywa się po jego ramkowaniu. Proces ten można opisać następująco:

$$ZCR = \frac{1}{2T} \sum_{t=1}^{T-1} |sgn(s_t) - sgn(s_{t-1})| \quad (3.3)$$

gdzie T to całkowita liczba próbek dla danej ramki, s_t to wartość próbki sygnału w chwili t , a funkcja $sgn()$ zwraca wartość 1 dla dodatniej wartości danej próbki, -1 dla ujemnej wartości i zero dla wartości równych zero. ZCR jest używany powszechnie do rozróżniania rodzajów dźwięków. Wysoka wartość tego współczynnika świadczyć może o wystąpieniu dużej ilości dźwięków nieharmonicznych, takich jak szum, a niska oznaczać może rozpoznanie mowy, bądź śpiewu. Tę metodę często wykorzystuje się podczas analizy muzyki, przykładowo do rozpoznania danego instrumentu, ponieważ posiada on charakterystyczne wzorce ZCR. Ta technika sama w sobie jest ograniczona z powodu dużej wrażliwości na szum oraz faktu zwracania tylko jednej informacji na temat sygnału. Dlatego do bardziej skomplikowanych zadań, takich jak detekcja emocji ZCR używa się w zintegrowanych systemach połączonych z metodami ekstrakcji dającymi więcej informacji. Należą do nich: LPCC (ang. *Linear Prediction Cepstral Coefficients*), MFCC (ang. *Mel-frequency cepstral coefficients*) i tym podobne.

3.3.1. Dyskretna transformata falkowa

DWT (ang. *Discrete Wavelet Transform*) to narzędzie często używane w przetwarzaniu sygnałów pozwalające na analizę dźwięku w dziedzinach czasu i skali, która jest odpowiednikiem osi częstotliwości w FFT, gdzie duża skala odpowiada małej częstotliwości [23]. Funkcja falkowa to zazwyczaj krótka fala, zbiegająca do zera dla argumentów dążących do nieskończoności. Ma ona lepsze zastosowanie w analizie sygnałów niestacjonarnych, gdyż może być dostosowywana i lokalizowana w czasie poprzez odpowiednie dostosowanie skali i przesunięcia w czasie.

Zasada działania tej transformaty polega na zmianie skali i przesunięcia falki bazowej, która pozwala na dokładniejszą analizę na większym spektrum rozdzielczości. W przypadku DWT kolejne skale są potęgami dwójki, a samo równanie można zapisać jako:

$$W(l, s) = 2^{s/2} \sum_n x(n) \psi(2^s n - l) \quad (3.4)$$

gdzie l oznacza przesunięcie, $2^{s/2}$ skale, $\psi()$ funkcję falkową oraz $x(n)$ reprezentują dyskretny sygnał [25]. W dodatku wynik analizy falkowej można przedstawić w formie skalogramu, gdzie oś pozioma to czas (przesunięcie falki), a oś pionowa to skala falki. Kolor falki oznacza wartość amplitudy składowej falki. Mimo małej czytelności, skalogram pozwala uwidocznić zmiany przebiegu składowych o różnych skalach [23].

DWT stosuje się do wielu zadań, nie tylko związanych z analizą dźwiękową; na przykład używa się tej transformaty do zadań kompresji obrazów i wideo. Pozwala ona na oddzielenie ważnych cech obrazu, co umożliwia skuteczną kompresję. Stosowana jest również do analizy sygnałów biomedycznych, takich jak badanie elektrokardiograficzne (EKG) lub badanie elektroencefalograficzne (EEG), gdyż dyskretna transformata falkowa pozwala na precyzyjne określenie zmiany części sygnału w czasie. Zadaniem DWT może być również usuwanie szumów, ponieważ podczas używania filtrów pasmowo przepustowych jesteśmy w stanie oddzielić części składowe sygnału uznawane za szum od sygnałów niosących użyteczne informacje. Zastosowanie znajduje także podczas rozpoznawania wzorców mowy, na przykład do ekstrakcji cech, które mogą być używane do wykrywania zwyródnień mowy lub klasyfikacji stanów emocjonalnych. Detekcja emocji na podstawie transformaty falkowej polega na dekompozycji sygnału na różne składowe częstotliwościowe w celu identyfikacji cech związanych z poszczególnymi emocjami [26].

3.3.2. LPC

Linear Predictive Coding jest techniką analizy sygnału dźwiękowego, często stosowaną w przetwarzaniu mowy i kodowaniu audio. Głównym założeniem LPC jest możliwość przybliżania próbki dźwięku jako kombinacji poprzednich próbek [27]. Procedura opisująca metodę LPC zobrażowana została na rysunku 3.2. Jednym z jej etapów jest ramkowanie, podczas którego sygnał jest dzielony na krótkie fragmenty, których długość zazwyczaj wynosi od 20 do 30 ms (przy częstotliwości próbkowania 44,1 kHz jest to od ok. 880 do 1325 próbek), ze względu taki czas trwania fonemu (podstawowa jednostka dźwiękowa w języku, która służy do rozróżnienia znaczenia słów) w ludzkiej mowie. Kolejnym etapem LPC jest okienkowanie, które zostaje nakładane na każdą ramkę, w celu minimalizacji dyskontynuacji sygnału, która jest nagłymi zmianami wartości sygnału spowodowanymi ramkowaniem. Przemnożenie sygnału poprzez odpowiednią funkcję okna ma na celu zmniejszenie różnicy w wartościach sygnału. Ostatni z etapów to autokorelacja, która jest miarą podobieństwa. Obliczana jest ona poprzez sumowanie iloczynów próbek sygnału z ich przesunięciami.

$$R(k) = \sum_{n=0}^{N-k-1} x(n)x(n+k) \quad (3.5)$$

gdzie $R(k)$ to wartość autokorelacji dla opóźnienia k , $x(n)$ jest sygnałem, z którego jest ona obliczana, N jest długością ramki, a n jest indeksem bieżącej próbki.

Konwersja na parametry LPC jest procesem polegającym na wykorzystaniu wyników autokorelacji ramek sygnału w celach estymacji współczynników filtru predykcyjnego, których zadaniem jest minimalizacja błędu predykcji. Obliczenie współczynników LPC wymaga rozwiązania równania:

$$\alpha = -R^{-1} \cdot r \quad (3.6)$$

gdzie R jest macierzą otrzymaną podczas obliczania estymatorów funkcji autokorelacji, α to wektor współczynników LPC, a r to wektor autokorelacji. Rozwiązania tego równania bazują głównie na metodzie Lavirsona-Durbina. Polega ona początkowym przeliczeniu wektora r z prawej strony równania na lewą:

$$R \cdot \alpha = r \quad (3.7)$$

Ustalany jest początkowy błąd predykcji E_0 jako pierwsza wartość współczynnika autokorelacji $r(0)$, który jest całkowitą mocą sygnału. Dla pierwszej iteracji współczynnik α_1 obliczany jest przy pomocy równania:

$$\alpha_1 = \frac{r(1)}{r(0)} \quad (3.8)$$

po czym aktualizowany jest błąd predykcji w następujący sposób:

$$E_1 = (1 - \alpha_1^2)E_0 \quad (3.9)$$

Kolejne współczynniki dla następnych iteracji k są obliczane w następujący sposób:

$$\alpha_k = \frac{r(k) - \sum_{j=1}^{k-1} \alpha_j \cdot r(k-j)}{E_{k-1}} \quad (3.10)$$

gdzie α_j jest współczynnikiem autoregresji, które mają na celu minimalizację błędu predykcji sygnału. Współczynniki autoregresji są aktualizowane przy użyciu wzoru:

$$\alpha_j^{(nowy)} = \alpha_j^{(stary)} - \alpha_k \cdot \alpha_{k-j}^{(stary)} \quad (3.11)$$

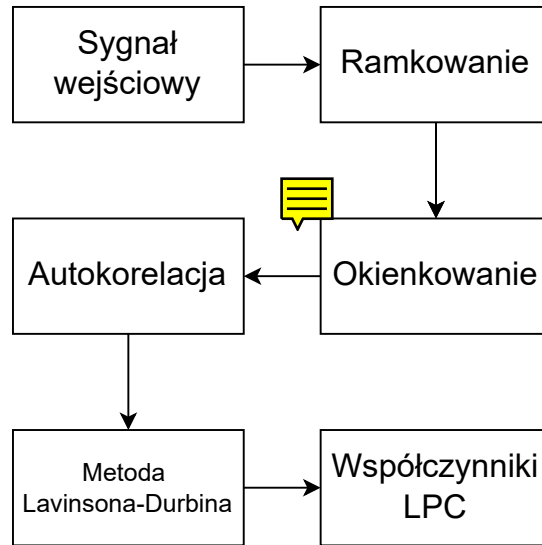
po czym aktualizujemy błąd predykcji:

$$E_k = (1 - \alpha_k^2)E_{k-1} \quad (3.12)$$

Proces jest powtarzany do momentu obliczenia wszystkich współczynników LPC[28].

Poprzez iteracje stopniowo zwiększa się skomplikowanie modelu dzięki dodawaniu kolejnych współczynników, gdzie przy każdej iteracji zmniejsza się wartość błędu predykcji. Wynikiem tej metody są współczynniki LPC, które później wykorzystywane są w celach rekonstrukcji sygnału do desygnowanych zadań analizy.

Obecnie współczynniki LPC mają zastosowanie w wielu zadaniach. Przykładem wykorzystania tej metody jest VoIP (ang. *Voice over IP*), gdzie użycie współczynników LPC pomaga redukować przepustowość potrzebną do przesyłania mowy poprzez zastosowanie modelu źródło-filtr, który interpretuje mowę jako wynik filtru kanału głosowego. Pozwalają one także na kompresję mowy, w celu zmniejszenia danych jakie telefon komórkowy potrzebuje do przesłania głosu [29]. LPC używane jest także w



Rysunek 3.2: Architektura LPC [27]

syntezatorach, gdzie pozwalają na imitację instrumentów dętych jak i głosu ludzkiego [30]. W kontekście problemu SER, LPC dostarcza informacji o formantach mowy, czyli o pasmach częstotliwości w dźwięku, w granicach których tony ulegają znacznemu wzmocnieniu. Są one unikalne dla różnych sygnałów dźwiękowych, a ich zbiór dla danego dźwięku określa jego barwę, co umożliwia skorelowanie jej ze stanem emocjonalnym mówcy [31]. Z ekstrahowanych współczynników LPC system detekcji jest w stanie klasyfikować wiele stanów emocjonalnych.

3.3.3. PLP

Perceptual Linear Prediction jest techniką analizy mowy, która ma na celu dokładne odwzorowanie sposobu percepcji dźwięku przez ludzkie ucho. PLP używa aspektów skali melowej i krytycznych pasm słuchowych w celu jak najlepszego odwzorowania rzeczywistego słuchu człowieka. Ta technika ma na celu odrzucenie nieistotnych informacji, takich jak szumy i dźwięki niepodobne do ludzkiego głosu. W założeniu jest ona podobna do LPC, ale bardziej zbliżona do ludzkiego systemu głosowego [27]. Proces PLP polega na kwantyzacji okienkowanego sygnału, aby ograniczyć dyskontynuację sygnału, po czym oblicza się widmo mocy okienkowanego sygnału za pomocą szybkiej transformaty Fouriera:

$$P_w(\Omega) = [ReS_w(\Omega)]^2 + [ImS_w(\Omega)]^2 \quad (3.13)$$

gdzie $P_w(\Omega)$ jest widmem mocy, $ReS_w(\Omega)$ to część rzeczywista widma, a $ImS_w(\Omega)$ jest częścią urojoną. W tej technice również stosuje się filtr Barka, który wykonuje trzy zadania: zniekształcenie, wygładzenie i próbkowanie częstotliwości. Aby zastosować ten filtr trzeba przekształcić częstotliwość, żeby dostosować jej skalę do percepcji ludzkiego ucha i aby ją przekształcić używamy wzoru:

$$\Omega(w) = \frac{6}{1200\pi} \ln \left(\frac{\Omega}{1200\pi} + \sqrt{\left(\frac{\Omega}{1200\pi} \right)^2 + 1} \right) \quad (3.14)$$

gdzie w jest pulsacją. Po skorzystaniu z tego filtru stosuje się preemfazę równo-głośności ważącą wyjścia z filtru Barka, która ma na celu symulować wrażliwość ludzkiego słuchu. Funkcję preemfazy nakłada się

na spektrum. W zależności od częstotliwości funkcja przyjmuje inne wartości:

$$\begin{aligned}
 \psi(\Omega) &= 0, & \text{dla } \Omega < -1.3 \\
 \psi(\Omega) &= 102.5\Omega + 0.5, & \text{dla } -1.3 \leq \Omega \leq -0.5 \\
 \psi(\Omega) &= 1, & \text{dla } -0.5 < \Omega \leq 0.5 \\
 \psi(\Omega) &= 10^{1.0\Omega - 0.5}, & \text{dla } 0.5 < \Omega \leq 2.5 \\
 \psi(\Omega) &= 0, & \text{dla } \Omega > 2.5
 \end{aligned} \tag{3.15}$$

Po tym kroku liczona jest konwolucja z krzywą maskującą krytycznego pasma przy użyciu wzoru:

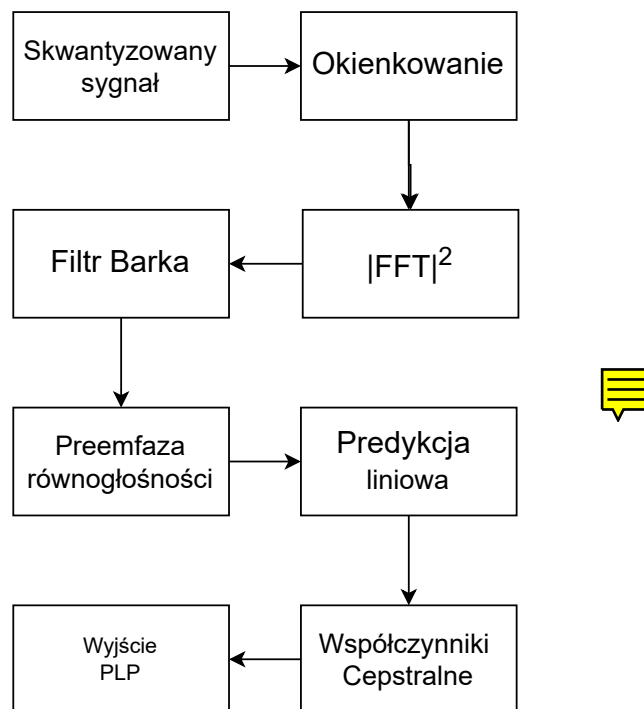
$$\theta(\Omega_i) = \sum_{i=-1.3}^{2.5} P(\Omega - \Omega_i) \cdot \psi(\Omega_i) \tag{3.16}$$

gdzie $\theta(\Omega_i)$ jest obliczoną konwolucją. To działanie ma na celu zmniejszenie rozdzielczości spektralnej, aby umożliwić próbkowanie. Kolejnym krokiem jest zastosowanie krzywej równego natężenia dźwięku do przekształconego widma, która pozwala symulować nierówną wrażliwość ludzkiego ucha na różne częstotliwości.

Następnie stosuje się kompresję amplitudy pozwalającą przedstawić nieliniowy związek między intensywnością sygnału a postrzeganą głośnością przez ludzkie ucho:

$$E_w(\Omega) = \frac{E_w(\Omega)}{w^2 + k_1 w^4} \times \frac{w^2 + k_2}{2w^2 + k_3} \tag{3.17}$$

przy czym wartości k są stałymi wartościami umożliwiającymi to przekształcenie. Na zniekształconym widmie jest wykonywana autokorelacja, po czym następuje obliczanie współczynników liniowych metodą Lavirsona-Durbina tak jak dla LPC [32].



Rysunek 3.3: Proces metody PLP [27]

PLP jest wykorzystywane w systemach rozpoznawania mowy ze względu na: bardziej zbliżoną do ludzkiej reprezentację cech, co sprawdza się korzystnie w algorytmach detekcji. Ta technika jest również używana podczas tworzenia syntezy mowy, gdyż przez lepsze odwzorowywanie ludzkich cech mowy, PLP umożliwia produkowanie bardziej zrozumiałych i bliższych naturalnemu brzmieniu głosów. Metoda ta zastosowanie znajduje również do wykrywania emocji z głosu przez to, że jest w stanie dokładnie znaleźć różnice w tonacji, tempie, głośności i jakości dźwięku. W uczeniu maszynowym używa się tej techniki zazwyczaj w połączeniu z innymi metodami ekstrakcji, na przykład z MFCC do dokładniejszej analizy cech z głosu.

3.3.4. MFCC

Mel-Frequency Cepstral Coefficients jest jedną z popularniejszych metod w przetwarzaniu i analizie sygnałów dźwiękowych - w szczególności w rozpoznawaniu mowy, bądź identyfikacji dźwięków. Jako elementarne czynniki MFCC uznawane są:

- Skala melowa, która jest zaprojektowana w sposób odpowiadający ludzkiej percepcji dźwięku. Koncentruje się na niższych częstotliwościach ze względu na to, że ludzkie ucho jest wrażliwsze dla zmian w niższych tonach niż wyższych.
- Cepstrum, które jest przekształceniem transformaty Fouriera z logarytmu widma mocy sygnału. Najpierw przekształca się sygnał za pomocą FFT (ang. *Fast Fourier Transform*). Następnie obliczany jest logarytm z widma mocy, w celu oddzielenia źródła sygnału od jego transmisji. Na koniec stosuje się odwrotną do niego transformację dającą cepstrum sygnału.

Proces ekstrakcji cech w MFCC jest skomplikowany i składa się z kilku czynników. Pierwszym z nich jest zastosowanie filtra preemfazy w celu wzmacniania wyższych częstotliwości.

$$x'_n = x_n - \alpha x_{n-1} \quad (3.18)$$

gdzie x_n i x'_n to odpowiednio sygnał przed i po preemfazie, a α to stały współczynnik zazwyczaj przyjmowany jako 0.97 [33]. Umożliwia to zrównoważenie spektrum sygnału, jak i poprawia ogólną jego analizę. Kolejnym krokiem jest podzielenie dźwięku na krótkie ramki. Następnie na każdą z nich nakładana jest funkcja okna, w celu zminimalizowania dyskretyzacji pojawiającej się na ich krawędziach. Zazwyczaj nakładana jest funkcja Hamminga przedstawiona jako:

$$\text{Ham}(N) = 0.54 - 0.46 \cos\left(\frac{2\pi n}{N-1}\right) \quad (3.19)$$

gdzie N jest całkowitą długością ramki a n jest kolejnymi próbkami $n = 1, 2, \dots, N$.

Następnie w procesie ekstrakcji stosuje się obliczanie szybkiej transformaty Fouriera, dzięki czemu uzyskujemy spektrum mocy sygnału bądź obliczane jest widmo mocy poprzez $|FFT|^2$. Na obliczone spektrum nakładany jest bank filtrów melowych, których zadaniem jest ekstrakcja informacji istotnych z punktu widzenia percepcji dźwięku przez człowieka.

$$S_m = \sum_{k=1}^N |X_r(k)|^2 H_m(k) \quad (3.20)$$

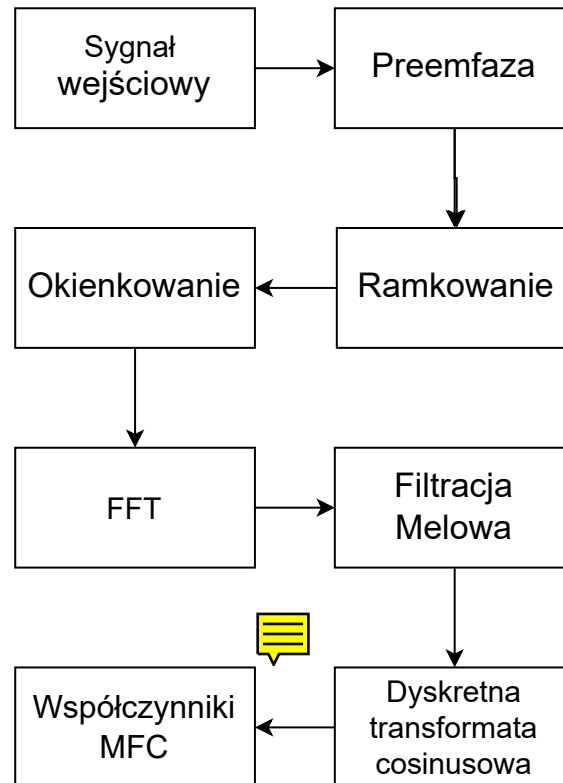
przy czym S_m jest energią pasma, m jest numerem filtra a X_r jest widmem ramki.

Ostatnim elementem ekstrakcji jest logarytmowanie amplitudy uzyskanej z filtrów melowych, po czym stosuje się dyskretną transformację cosinusową, z której uzyskiwany jest zestaw współczynników nazywanych współczynnikami MFCC opisanymi wzorem:

$$c_i = \sqrt{\frac{2}{M}} \sum_{m=1}^M \log(S_m) \cos\left(\frac{\pi i}{M}(m - 0.5)\right) \quad (3.21)$$

gdzie M jest liczbą użytych filtrów, a i to numer współczynnika.

MFCC jest szeroko stosowaną metodą ekstrakcji cech, która jest używana do wielu zadań, między innymi do rozpoznawania dźwięków. Przykładowo, używając tej metody jesteśmy w stanie rozróżnić dźwięki natury od dźwięków miasta, czy też instrumenty muzyczne. Jednakże MFCC ma największe zastosowanie w połączeniu z algorytmami uczenia maszynowego. Stosowane jest to często do rozpoznawania mowy, gdyż ze względu na wysoką dokładność jest w stanie określić w niej subtelne różnice pośród poszczególnych osób. Metoda ta potrafi również dokładnie określać cechy mowy, takie jak: tonacja, intonacja i tym podobne, dlatego też świetnie sprawdza się w detekcji emocji z głosu.



Rysunek 3.4: Proces ekstrakcji cech przy użyciu MFCC [27]

3.4. Porównanie przedstawionych metod

Wyżej wymienione metody są często wybieranymi technikami ekstrakcji cech, ale wybór odpowiedniej z nich nie jest tak prosty, gdyż każda ma swoje zalety i wady.

Metoda LPC cechuje się łatwością implementacji ze względu na prostą i bardzo precyzyjną matematykę[27]. Również charakteryzują się niską liczbą współczynników, które są w stanie efektywnie

oddać charakterystykę widmową sygnału, lecz te współczynniki są ze sobą mocno skorelowane, co oznacza, że niektóre współczynniki są częściowo redundantne (funkcjonalnie niepotrzebne), co wpływa negatywnie na analizę mowy[27]. Przez liniową skalę używaną w metodzie LPC reprezentacja mowy nie jest adekwatna[27], dlatego ta metoda może nie być efektywna w analizie mowy.

Z kolei MFCC w odróżnieniu od LPC nie opiera się na cechach liniowych i jest podobna do ludzkiego systemu percepcji słuchowej oraz posiada niską korelację między współczynnikami, co pozwala na lepszą reprezentację unikalnych cech mowy[27]. Ta metoda zapewnia również dobrą dyskryminację, co oznacza skuteczne rozpoznanie różnych sygnałów[27]. Z drugiej strony MFCC posiada ograniczoną reprezentację sygnałów mowy, ze względu na to, że ta metoda skupia się wyłącznie na mocy widma sygnału oraz przez to, że ta metoda posiada niską odporność na szum[27].

Zalety metody PLP są podobne do zalet MFCC, czyli również przetwarza sygnał na wzór ludzkiego ucha. Dodatkowo podobnie do LPC jest w stanie dokładnie reprezentować charakterystykę widmową sygnału przy użyciu niewielkiej liczby współczynników[27]. Ta metoda jest również w stanie zmniejszyć rozbieżności między sygnałami dźwięcznymi (np. samogłoski) i bezdźwięcznych (np. spółgłoski szumiące), co zwiększa dokładność rozpoznawania mowy[27]. Jednakże skuteczność PLP jest zależna od całej ogólnej charakterystyki spektralnej, co sprawia, że metoda może nie być skuteczna w przypadku zakłóceń wywołanych przez np. szum, czy używany sprzęt do nagrywania[27].

3.5. Wątki humanistyczne w interpretacji dźwięku

Ten fragment został dodany jako materiał testowy i nie dotyczy technicznej teorii przetwarzania sygnałów audio. Skupia się na humanistycznym rozumieniu głosu jako nośnika pamięci, emocji oraz relacji społecznych.

3.5.1. Głos jako element narracji kulturowej

W literaturze i teatrze głos może pełnić funkcję znaku obecności bohatera, środka budowania napięcia oraz sposobu ujawniania relacji między postaciami. Analiza ta ma charakter interpretacyjny, a nie algorytmiczny.

3.5.2. Symbolika ciszy w tekstach kultury

Cisza bywa przedstawiana jako przestrzeń kontemplacji, konfliktu lub wykluczenia. Jej znaczenie zależy od kontekstu historycznego i estetycznego, dlatego nie odnosi się bezpośrednio do ekstrakcji cech z próbek głosowych.

4. ASPEKTY UCZENIA MASZYNOWEGO (JAKUB MAKOWSKI)

Sztuczna inteligencja może zostać opisana jako dziedzina nauki i inżynierii, która zajmuje się tworzeniem inteligentnych maszyn, tj. zdolnych do samodzielnego działania i podejmowania decyzji, a w szczególności programów komputerowych [34]. Zadaniem AI jest zrozumienie ludzkiej inteligencji, tj. zdolności do wykonywania zadań wymagających ludzkiego rozumowania. Zalicza się do nich zdolność uczenia się, przetwarzanie języka naturalnego, tworzenie rozwiązań problemów oraz wiele innych ludzkich umiejętności intelektualnych. Prekursorem badań o tematyce sztucznej inteligencji był brytyjski matematyk Alan Turing. To on w roku 1950 opracował znany na całym świecie test Turinga, określający różnice między odpowiedzią maszyny a człowieka w tekście, co było spowodowane zapytaniem "Czy maszyny mogą myśleć?" [35]. Dzięki jego zaangażowaniu i pasji, coraz więcej ludzi zaczęło interesować się dziedziną AI i prowadzić badania w tym kierunku, co przyczyniło się do przyspieszenia jej rozwoju.

Mówiąc wprost, sztuczna inteligencja jest dziedziną, która łączy informatykę oraz duże zbiory danych do rozwiązywania problemów. W przypadku problemów opartych o sygnał dźwiękowy, najczęściej wykorzystywanym podzbiorem AI jest uczenie maszynowe (ML - ang. *Machine learning*). Michell, amerykański informatyk oraz profesor, definiuje ML jako "zdolność programu komputerowego do uczenia się z doświadczenia E w odniesieniu do pewnej klasy zadań T i miary wydajności P, jeśli jego wydajność wykonania zadań T, mierzona za pomocą P, poprawia się z doświadczeniem E" [36]. W uczeniu maszynowym można wyróżnić aspekty takie jak uczenie ze wzmocnieniem, uczenie bez nadzoru, a także uczenie z nadzorem, w skład którego wchodzi sieci neuronowe, sieci głębokie i inne.

Każdy z typów uczenia maszynowego różni się od siebie w znaczący sposób. Uczenie ze wzmocnieniem przedstawia podejście, w którym program uczy się podejmować decyzje poprzez integrację ze środowiskiem. Wykonuje on działania metodą prób i błędów, po których otrzymuje nagrody lub kary w zależności od ich wyniku. To rozwiązanie znajduje zastosowanie w robotyce, nawigacji czy grach takich jak szachy.

Z kolei uczenie bez nadzoru polega na uczeniu z nieoznaczonych danych. Jego zadaniem jest odnalezienie wzorców bądź struktur danych bez ich oznaczania lub jakiegokolwiek zewnętrznej interwencji, co przyczynia się do podejmowania lepszych decyzji. Ten rodzaj ML stosuje się w zadaniach segmentacji, analizy rynkowej, czy np. detekcji anomalii.

Ostatnim z głównych typów uczenia maszynowego jest uczenie z nadzorem. Polega ono na nauczaniu algorytmu za pomocą danych, które zostały oznaczone odpowiednimi etykietami. Program porównuje wyniki wyjściowe kolejnych danych wejściowych, dzięki czemu uczy się przewidywać rezultaty dla nowych wejść. Znane są z tego algorytmy takie jak drzewa decyzyjne, sieci neuronowe, metoda wektorów nośnych (SVM - ang. *Support Vector Machines*), czy regresja liniowa. Znajduje to zastosowanie w rozpoznawaniu obrazów, prognozowaniu danych oraz rozpoznawaniu mowy i nie tylko. Ze względu na analizę sygnałów dźwiękowych, to właśnie uczenie nadzorowane zostało użyte do rozwiązania problemu opisywanego przez pracę.

4.1. Wstęp do sieci neuronowych

Sieci neuronowe są systemami przetwarzania informacji, które inspirowane są działaniem ludzkiego mózgu. Ich podstawowym elementem jest sztuczny neuron, mający na celu naśladowanie funkcjonowania biologicznego neuronu. Sieci składają się z warstw, w których znajduje się określona liczba takich "sztucznych nerwów". Są to kolejno warstwa wejściowa, jedna lub więcej warstw ukrytych oraz warstwa wyjściowa. Neurony znajdujące się w warstwach pełnią rolę jednostek przetwarzających, odbierając sygnały wejściowe i generując sygnał wyjściowy. Oparty jest on o funkcję aktywacji (ang. *Activation Function*), która odpowiada za decyzję w sprawie aktywowania odpowiedniego neuronu [37]. W przypadku wielu warstw ukrytych uznaje się sieć neuronową za głęboką (DNN - ang. *Deep Neural Network*). Większość modeli, które zajmują się rozwiązywaniem złożonych problemów, aktualnie używanych można nazwać sieciami głębokimi.

Do rozwiązywania poszczególnych zadań sieci muszą być trenowane na odpowiednich dla solucji zbiorach danych. Jest kilkanaście typów sieci neuronowych w zależności od posiadanych architektur oraz występujących w nich warstw. W bardzo dobry sposób radzą one sobie ze złożonymi danymi, przez co stanowią rozwiązanie dla bardziej skomplikowanych problemów, jak np. detekcja twarzy, systemy wizyjne w pojazdach autonomicznych, przetwarzanie języka naturalnego, czy rozpoznawanie dźwięków i wykrywanie emocji [38]. Najwięcej rozwiązań problemów powiązanych z sygnałami dźwiękowymi opiera się o sieci konwolucyjne, rekurencyjne sieci neuronowe (RNN - ang. *Recurrent Neural Networks*), jej modyfikację LSTM (ang. *Long Short-Term Memory*), a także w ostatnich czasach popularność w tej dziedzinie zdobywają modele oparte o transformer. Niżej opisane zostały, najważniejsze dla naszej pracy, architektury sieci neuronowych.

4.2. Konwolucyjne sieci neuronowe

Swój początek konwolucyjne sieci neuronowe zawdzięczają eksperymentom przeprowadzanym w latach 50-tych i 60-tych XX wieku przez Hubela oraz Wiesela [39]. Polegały one na podłączeniu mikroelektrod do mózgu kota i analizie reakcji zwierzęcia na obrazki. Podczas badań zauważono, że na kreskę ukośną reagowała jedna grupa neuronów, natomiast na pionową inna. Dzięki temu odkryli oni typy komórek znajdujące się w korze wzrokowej, zwane prostymi oraz komórkami złożonymi. Następnie zaoferowali także kaskadowy model, który zastosowanie znajdował w odszukiwaniu wzorców. Przełomowym dla CNN badaczem komputerowym był LeCun, który w latach 80-tych XX wieku wykorzystując propagację wsteczną, modyfikował składniki jądra splotu [40]. CNN łączy trzy koncepcje architektoniczne, zapewniając jakiś stopień zmiany oraz odporności na zniekształcenia. Należą do nich lokalne pola recepcyjne, współdzielone wagi (lub replikacja wag) oraz czasami przestrzenny lub tymczasowy *subsampling* [41]. Jak sama nazwa mówi sieci tego typu wykorzystują splot. Obliczany jest on za pomocą:

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau \quad (4.1)$$

gdzie $(f * g)(t)$ to wynik splotu funkcji f i g w punkcie t , τ to przesunięcie względem t , a $d\tau$ jest elementem całkowania. W wyniku tych działań "łączone" są obydwie funkcje. Podstawową częścią architektury konwolucyjnych sieci neuronowych są warstwy splotowe (konwolucyjne) lub pulujące (ang. *pooling lay-*

ers). Warstwy te zawierają wyuczone filtry (jądra), które nie są opisywane w definicji CNN ze względu na ich dobieranie i optymalizację podczas trenowania sieci [42]. Sama sieć najczęściej posiada w swojej architekturze wiele warstw ukrytych, co stanowi ją w wielu przypadkach głęboką siecią neuronową, tak samo jak i RNN.

Uczenie CNN odbywa się przy użyciu algorytmu propagacji wstecznej, dzięki któremu dostosowywane są wagi filtrów splotowych, w taki sposób aby zminimalizować funkcję straty. Za poprawę efektywności trenowania sieci odpowiada wiele technik, a niektóre z nich stanowią:

- Augmentacja danych (DA - ang. *Data Augmentation*), która polega na sztucznym zwiększaniu rozmiarów zestawu danych uczących wprowadzając przebudowane wersje konkretnych obrazów. Stosuje np. obracanie, przesunięcia, skalowanie w celu redukcji przeuczenia modelu i skutecznego działania na nowych danych. Okazuje się ona kluczowa przy problemach wizji komputerowej, gdzie umożliwia modelowi radzenie sobie z różnorodnością obrazów, co często występuje w rzeczywistych przypadkach.
- Normalizacja wsadowa (ang. *Batch Normalization*) poprawia szybkość, stabilność oraz wydajność procesu trenowania sieci. Normalizuje ona na poziomie każdego wsadu wyjścia warstw wewnętrznych, co niweluje problem zanikającego gradientu (ang. *vanishing gradient*), a to umożliwia używanie szybszego uczenia oraz upraszcza proces optymalizacji.
- Funkcje aktywacji, a dokładniej ReLU (ang. *Rectified Linear Unit*) oraz jej transformacje (*Leaky ReLU*, *Parametric ReLU*), wprowadzając nieliniowość do procesu trenowania, umożliwiają modelowi uczyć się wzorców o większej złożoności danych. ReLU charakteryzuje się ogromną popularnością zdobytą dzięki prostocie obliczeń, a także skuteczności rozwiązywania częstego w sieciach problemu *vanishing gradient*.
- Wczesne zatrzymywanie (ang. *Early Stopping*) polega na obserwacji wyników wydajności w trakcie trenowania sieci i zatrzymaniu tego procesu, gdy wykryty zostanie brak poprawy rezultatów. Model unika w ten sposób problem przetrenowania, a także uskutecznia to pracę na danych, które nigdy wcześniej nie były do niego wprowadzane.

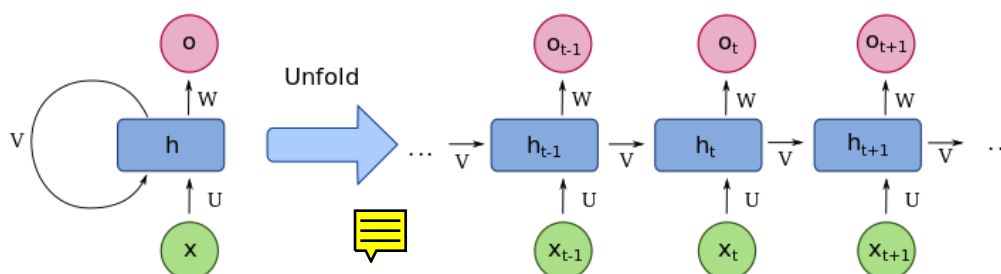
Dzięki swojej efektywności sieci CNN posiadają szerokie zastosowania w wielu dziedzinach. Bardzo dużą rolę spełniają przy automatycznej analizie obrazów i wideo, do czego dążono przy opracowywaniu ich u samego początku. Nie ograniczyło to jednak rozprzestrzenienia się ich możliwości w innych dziedzinach. Często konwolucyjne sieci neuronowe używane są przy analizach medycznych, systemach autonomicznej jazdy, czy właśnie analizie mowy i detekcji emocji.

4.3. Rekurencyjne sieci neuronowe

Sieci RNN posiadają najbardziej ogólną architekturę sieci neuronowych, a ich graf połączeń przedstawia cykle. Nazwano tak ścieżki, które przechodząc przez kolejne połączenia prowadzą do początkowego neuronu [37]. Wyjścia poprzednich kroków wprowadzane są ponownie do sieci jako wejścia przy kolejnych iteracjach, jak na rysunku 4.1. Dzieje się to dzięki przypisaniu opóźnień każdemu z połączeń

między neuronami. Tak właśnie następuje rekurencja, skąd wzięta się nazwa danej modyfikacji. Pozwala to sieci wykorzystywać informacje z przeszłości do aktualnych obliczeń, co prowadzi do poprawy działania sieci. Zdolność ta umożliwia zastosowanie ich do przetwarzania danych sekwencyjnych, do jakich również należą sygnały dźwiękowe. Standardowe rozwiązanie okazuje się nie najlepszym w przypadku uczenia się na dłuższych sekwencjach. RNN spotyka się wtedy z problemami takimi jak zanikający, czy też rzadziej eksplodujący gradient. Znalaziono jednak rozwiązania tych problemów modyfikując architekturę tych sieci, w wyniku czego powstały LSTM oraz GRU (ang. *Gated Recurrent Unit*). Dzięki nim możliwym stało się efektywniejsze uczenie na długich sekwencjach danych. Do zalet RNN zaliczana jest więc: możliwość pracy na danych wejściowych o dowolnej długości, brak zwiększania modelu w zależności od wejść, posiadanie historycznych informacji oraz rozkład wag w czasie [43]. Z drugiej strony obliczenia nie są zadowalająco szybkie, bardzo trudno dotrzeć do danych sprzed dawna oraz nie ma możliwości uwzględnienia jakichkolwiek przyszłych danych dla aktualnego stanu.

Istnieje wiele rozwiązań używających sieci RNN, np. przy przetwarzaniu języka naturalnego (NLP - ang. *Natural Language Processing*), tworzeniu modeli językowych, analizie oraz prognozie szeregów czasowych, czy rozpoznawaniu zdarzeń w ciągłych strumieniach wideo. Modyfikację RNN, tj. LSTM, w obecnych czasach stosuje się najczęściej, gdy pod uwagę brane są rekurencyjne sieci neuronowe. Modele oparte na tej architekturze znajdują zastosowanie w złożonych zadaniach, jak generowanie tekstu, tłumaczenie maszynowe oraz analiza sygnałów dźwiękowych. Z tego też względu niżej poświęcono rozdział dokładnie opisujący tę zaawansowaną formę RNN.



Rysunek 4.1: Schemat jedno-jednostkowej rekurencyjnej sieci neuronowej, gdzie x to stan wejściowy, h stan ukryty, o to stan wyjściowy, a U , V , W to wagi sieci

4.4. Long Short-Term Memory

LSTM to modyfikacja rekurencyjnych sieci neuronowych, która zaprojektowana została z myślą o zniewielowaniu problemu związanego z uczeniem się RNN na długich sekwencjach danych. Stanowi zatem rozwiązanie powszechnego zanikania gradientu. Przedstawili ją Hochreiter oraz Schmidhuber w swoim artykule z 1997 roku [44]. Podstawową jednostkę w architekturze, która na celu ma radzenie sobie z długotrwałymi zależnościami, stanowi komórka pamięci. Jej centrum to stała karuzela błędów (CEC - ang. *Constant Error Carousel*), umożliwiająca przechowywanie oraz ochronę stanu wewnętrznego, dzięki

dłuższym okresie czasu, przy czym nie prowadzi do znacznych zaników sygnału. Komórka LSTM zawiera oprócz tego trzy kluczowe elementy, do których należą bramka wejściowa (ang. **Input Gate**), bramka wyjściowa (ang. **Output Gate**) oraz bramka "zapomnij" (ang. *Forget Gate*), która wprowadzona została po zauważeniu w badaniach braku możliwości resetu stanów [45]. Każdy z tych elementów pełni znaczące funkcje względem komórki.

Pierwszy z nich (bramka wejściowa) decyduje, które z przychodzących fragmentów informacji zostaną dodane do danych przechowywanych w komórce. Wykorzystywana do tego jest sigmoidalna funkcja aktywacji, która generując każdemu z wejść wartości od 0 do 1, uznaje za ważne te, którym bliżej do jedynki, co skutkuje ich przechowaniem w pamięci.

Następna bramka (zapominania) ustanawia, które dane znajdujące się w pamięci powinny zostać usunięte lub "zapomniane". Działa w podobny sposób do wejściowej, a co za tym idzie wartości posiadające mniejszą odległość do 1 niż do 0, decyduje iż ta informacja zostanie zachowana i nie należy jej usuwać lub o niej zapominać.

Ostatnia natomiast (bramka wyjściowa) stanowi o tym, które z informacji powinny zostać użyte przy wygenerowaniu wyjścia sieci. Używa ona do tego sigmoidalnej funkcji aktywacji na danych znajdujących się w komórce pamięci, po czym za pomocą kolejnej funkcji aktywacyjnej, tym razem tanh, skaluje wyniki od -1 do 1. One stosowane są następnie do wygenerowania ostatecznego wyniku sieci.

Opisywany LSTM, który zobaczyć można na rysunku 4.2, można opisać w następujący sposób:

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{Ci}C_{t-1} + b_i) \quad (4.2)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{Co}C_t + b_o) \quad (4.3)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{Cf}C_{t-1} + b_f) \quad (4.4)$$

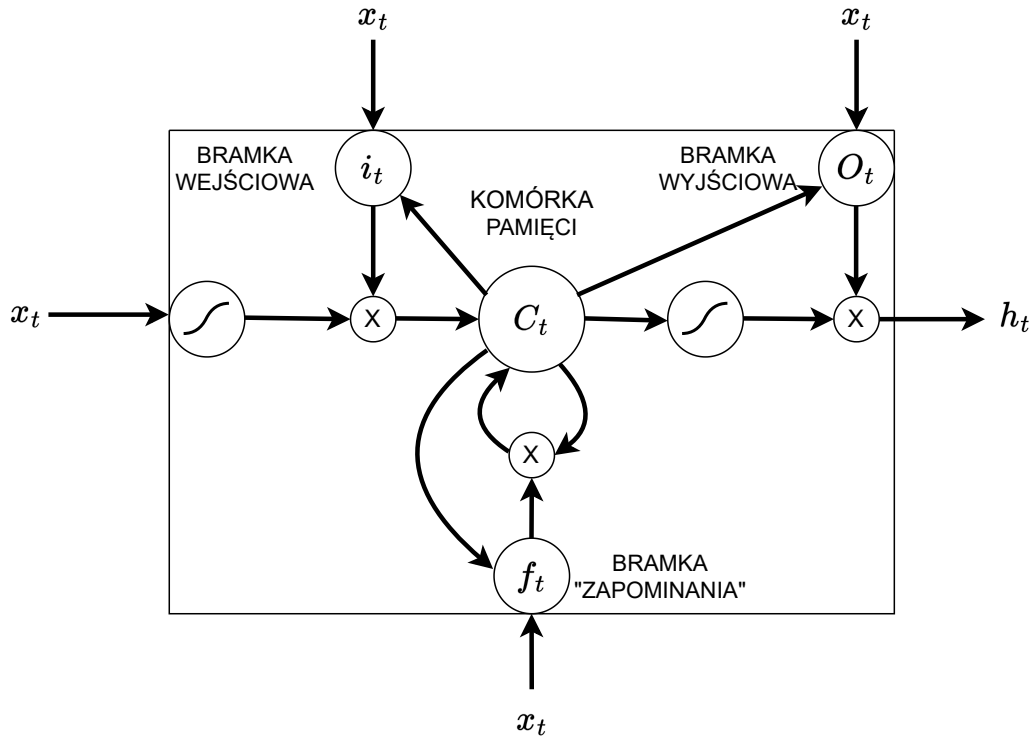
$$C_t = f_t C_{t-1} + i_t \tanh(W_{xC}x_t + W_{hC}h_{t-1} + b_C) \quad (4.5)$$

$$h_t = o_t \tanh(C_t) \quad (4.6)$$

gdzie i , o , f i C to kolejno bramka wejściowa, wyjściowa, zapominająca i komórka pamięci; x to sygnały wejściowe, σ to funkcja sigmoidalna, W oraz b to macierze wag, a indeks t jest aktualną wartością, gdzie $t - 1$ oznacza wartość poprzednią.

Właśnie dzięki kontroli przepływu informacji przez bramki, LSTM są w stanie, przez dłuższe okresy czasu, zachowywać ważne informacje. Przy czym eliminują problem zanikającego gradientu, często występującego w tradycyjnych sieciach RNN. Sprawia to, że sieci te wykazują się wysoką skutecznością przy modelowaniu sekwencji danych, gdzie zależności zachodzące pomiędzy ich elementami mogą być rozciągnięte w czasie.

Trenowanie LSTM opiera się na algorytmie propagacji wstecznej. Ze względu na swoją unikalną strukturę, do nauczania potrzebne jest dostosowanie długości sekwencji wejściowych. Celem tego działania jest efektywna nauka sieci dla zależności długoterminowych. Najlepiej, gdy używany jest bardzo duży zbiór danych reprezentujących problem, tak aby sieć "zrozumiała" złożoność zadania. Ważnym jest również staranne dobranie parametrów takich jak liczba warstw, ilość jednostek w każdej z nich oraz



Rysunek 4.2: Architektura komórki LSTM zawierająca bramkę zapominania

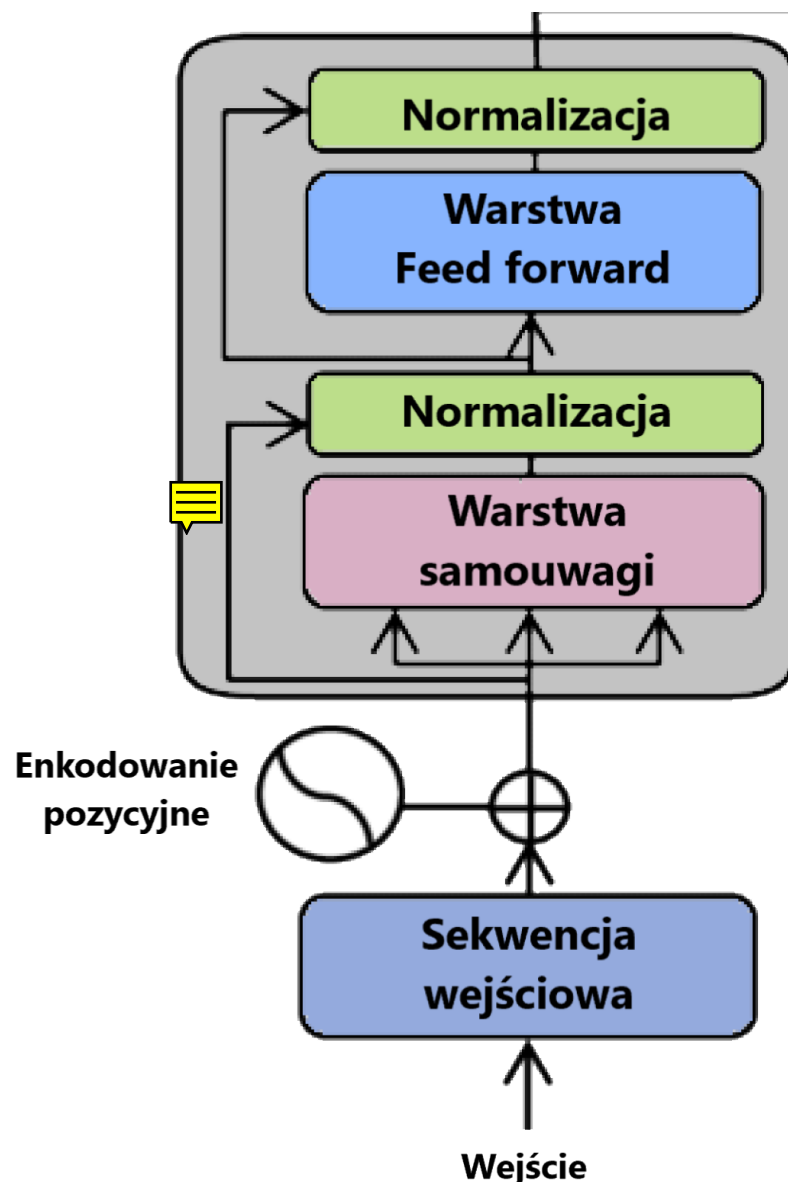
szybkość uczenia, by nie doprowadzić do niechcianych zachowań charakterystycznych dla rekurencyjnych sieci neuronowych.

Zastosowanie *Long Short-Term Memory* jest bardzo szerokie, jak w przypadku wielu spośród sieci neuronowych. Używane są często w dziedzinie NLP, przy komponowaniu muzyki, a także rozpoznawaniu mowy. Zdolność modelowania złożonych układów danych ustanawia LSTM, jako jedne z popularniejszych rozwiązań przy problemach polegających na analizie danych sekwencyjnych. Do takich zadań należy także SER. Sieci wykorzystanie znajdują przy badaniu cech akustycznych oraz prozodycznych mowy, co umożliwia im identyfikację emocji. Do tych celów często łączone są różne typy sieci oraz algorytmy, aby uzyskać jak najlepsze rozwiązania.

4.5. Model oparty o Transformer (Jakub Bieńkowski)

Rewolucyjny model wprowadzony przez zespół Google Brain w 2017, oryginalnie desygnowany do przetwarzania języka naturalnego znajduje zastosowania w przetwarzaniu obrazów i analizie dźwiękowej [46]. Rewolucyjność modelu transformer polega na odejściu od rekurencji czy konwolucji używanych kolejno w RNN i CNN, a jego architektura składa się z warstw *encodera* 4.3 i *decodera* 4.4 oraz bloku *multi-head attention*. Zbudowany jest on z N takich identycznych warstw, gdzie N jest jednym z hiperparametrów (czyli parametry, które mają istotny wpływ na działanie i wydajność modelu). Każda próbka wejściowa jest podzielona na trzy wektory: *query* (wektor zapytania), który mierzy podobieństwo między innymi elementami, *key* (wektor klucza), który przypisuje kolejno wagi do każdego elementu oraz *value* (wektor wartości), który przechowuje ważną informację dla danego problemu.

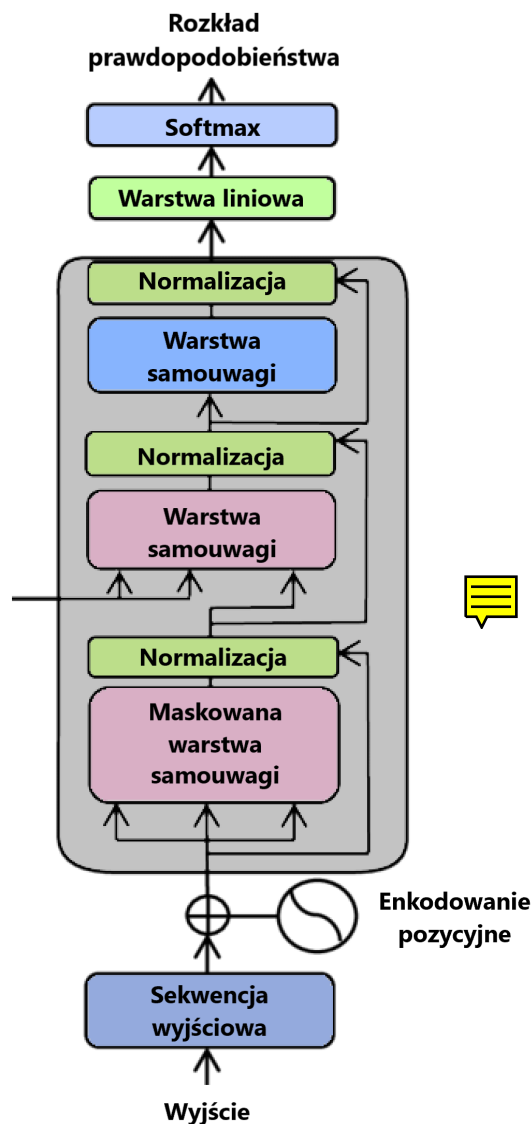
Warstwy *encoder* i *decoder* są opatrzone w blok samo uwagi, który pozwala na lepsze zrozumienie kontekstu dla każdego elementu poprzez wektoryzowanie sekwencji wejściowej. Wszystkie warstwy dla tego modelu są opatrzone w normalizację danych, w której występuje połączenie rezydualne, które jest techniką mającą na celu rozwiązanie problemu zanikającego gradientu, poprzez zapewnienie ścieżki, którą gradient może być bezpośrednio propagowany wstecz do wcześniejszych warstw[47]. Po tym połączeniu wykonywane jest zadanie stabilizacji uczenia. Przez to, że transformer używa *multi-head-attention* to proces uwagi jest równolegle wykonywany przez wiele elementów, gdzie dla każdego elementu jest przypisywany inny zestaw wag dla wektorów, co pozwala na równoczesne skupienie się na wielu cechach w jednej sekwencji.



Rysunek 4.3: Architektura enkodera modelu transformer [48]

W warstwie dekodera występuje maskowana warstwa samo uwagi, której zadaniem jest zapobiegnięcie przewidywania następnej danej poprzez przeglądanie przyszłych, co zapewnia predykcje tylko i wyłącznie na podstawie wcześniejszych próbek 4.4. Po wykonaniu tego procesu wyniki wektorowe są

łączone w jeden wektor, który przekazuje się do warstwy feed-forward 4.4. Stanowi ona zazwyczaj sieć neuronową, przekształcającą nieliniowo dane. Po niej następuje połączenie rezydualne, których wyjście z jednej warstwy nie jest tylko przekazywane do następnej warstwy, ale jest również dodawane do wyjścia z jednej z kolejnych warstw 4.4. Celem tego połączenia jest przekazanie informacji z poprzednich warstw do późniejszych etapów aby utrzymać kontekst w całym procesie co powoduje szybsze i efektywniejsze uczenie, jak i do rozwiązuje problem zanikających gradientów. Warstwa liniowa w dekodерze ma na celu przekształcenie jego wyjścia na wektor logitów (czyli funkcji przekształcających prawdopodobieństwa na logarytm szans), który następnie przekazywany jest do funkcji softmax, przetwarzającej go na rozkład prawdopodobieństwa.



Rysunek 4.4: Architektura dekodera modelu transformer [48]

W artykule [46], który jako pierwszy przedstawił model typu transformer, ukazano różnice między modelami bazującymi na konwolucyjnych, czy rekurencyjnych sieciach w zadaniu tłumaczenia maszynowego z języka angielskiego na język niemiecki, jak i z tłumaczenia języka angielskiego na język

francuski w ramach warsztatów WMT14 (ang. *Workshop on Statistical Machine Translation 2014*). Model Transformer w zadaniu tłumaczenia z języka angielskiego na niemiecki pobił rekord z wynikiem 28,4 w skali BLEU (ang. *Bilingual Evaluation Understudy*) o ponad 2 punkty nad drugim najlepszym modelem convS2s Ensemble, który jest siecią bazującą na konwolucyjnych warstwach encoder i decoder. Rekord dla modelu transformer został pobity w przypadku drugiego zadania tłumaczenia na język francuski, gdzie wynik wynosił 41,8 BLEU. Warto zaznaczyć że koszty treningu modelu w jednostce FLOP (ang. *Floating Point Operations Per Second*), czyli liczby operacji zmiennoprzecinkowych jakie komputer wykonuje w ciągu sekundy, jest zdecydowanie mniejsza niż w przypadku modelu ConS2S Ensemble, co pokazuje jak w modelu Transformer odejście od podejścia sekwencyjnego czy rekurencyjnego wpływa na wydajność, przy czym nie oddziałuje to na wyniki modelu, a nawet je poprawia [46].

Modele transformer pozwoliły na tworzenie skomplikowanych rozwiązań NLP (ang. *Natural-language programming*), takich jak GPT (ang. *Generative Pre-trained Transformer*) czy BERT (ang. *Bi-directional Encoder Representations from Transformers*), a także stosowane są jako solucja innych problemów. W kontekście wykrywania emocji z próbki dźwiękowej same działanie modelu się nie zmienia, a najważniejszym zadaniem jest odpowiednia augmentacja danych i ekstrakcja cech.

5. PRZEGLĄD ROZWIĄZAŃ (JAKUB BIEŃKOWSKI, JAKUB MAKOWSKI)

Wykrywanie emocji z sygnałów dźwiękowych to złożony problem, który posiada wiele propozycji rozwiązań w zależności od przyjętych założeń. Używane jest wiele różnych narzędzi do analizy sygnałów, przetwarzania danych, a także wykrywania emocji. Każda z nich posiada wady i zalety z perspektywy różnych potrzeb. Do uzyskania większej ilości danych treningowych bardzo często stosuje się augmentację danych, przy czym wykorzystywane są różnego rodzaju funkcje przekształcające sygnały dźwiękowe. Do problemu SER (ang. *Speech Emotion Recognition*) utworzono wiele propozycji rozwiązań, gdzie często używane są tam sieci CNN, LSTM w różnych modyfikacjach, czy też model transformer. W tym rozdziale przedstawiono rozwiązania wykorzystujące takie metody.

5.1. Podział emocji (Jakub Makowski)

W rozwiązaniach problemu SER emocje dzielone są zazwyczaj na: złość, wstręt, strach, szczęście, smutek oraz emocję neutralną, ze względu na duże ilości tak właśnie podzielonych datasetów. Dla przykładu w pracy Aouani i Ayed użyte zostały wszystkie z nich oprócz emocji neutralnej [49]. Podział ten wydaje się być najpopularniejszy, ze względu na prostą grę aktorską. W podobny sposób rozdzielenie zostało wykonane w pracy [50]. Emocje podzielono ze względu na używane zbiory danych, dla pierwszego datasetu były to: złość, znudzenie, wstręt, strach, szczęście, emocja neutralna oraz smutek; natomiast dla drugiego: złość, ekscytacja, frustracja, szczęście, neutralność i smutek [50].

5.2. Przegląd datasetów (Jakub Makowski)

W pracy Zhao, Mao i Chena zostały użyte dwa publiczne zbiory danych *Berlin EmoDB* i *Interactive Emotional Dyadic Motion Capture* (IEMOCAP) [50]. Pierwszy z nich zawiera siedem emocji, przy czym każda emocja zawiera prawie tyle samo oznaczonych nagrań, aby dokładność klasyfikacji mogła zostać prawidłowo oceniona. Wypowiadany przez profesjonalnych aktorów emocjami były gniew, znudzenie, obrzydzenie, strach, szczęście, smutek oraz emocja neutralna. Łącznie użytych zostało 535 zdań, pochodzących z codziennej komunikacji [50]. Natomiast IEMOCAP zawiera 1150 wypowiedzi pięciu par aktorów, gdzie odgrywane były scenariusze afektywne: złość, podekscytowanie, frustracja, szczęście, neutralność, smutek i zaskoczenie. Często wykorzystywany jest także zbiór danych *Ryerson Audio-Visual Database of Emotional Speech and Song* (RAVDESS), gdzie znajduje się 1440 plików audio z nagraniami aktorów udających 7 emocji, 5 podstawowych z definicji Ekmana oraz zaskoczenie i emocję neutralną [51].

Surrey Audio-Visual Express Emotion (SAVEE) wykorzystany został w pracy opisującej model oparty o CNN, przy czym składa się on ze 120 wypowiedzi czterech mężczyzn, którzy odgrywali siedem emocji, jak w przypadku RAVDESS [52]. Artykuł [53] porównuje model Transformer przy detekcji emocji wydobytej z głosu do bardziej standardowych rozwiązań innych modeli. W pracy przedstawione zostały używane zbiory danych oraz etapy augmentacji danych, ekstrakcji cech z próbek głosowych, metody ewaluacji metryk oraz używany model. Wykorzystane zostały datasety takie jak: BAVED (ang. *Basic*

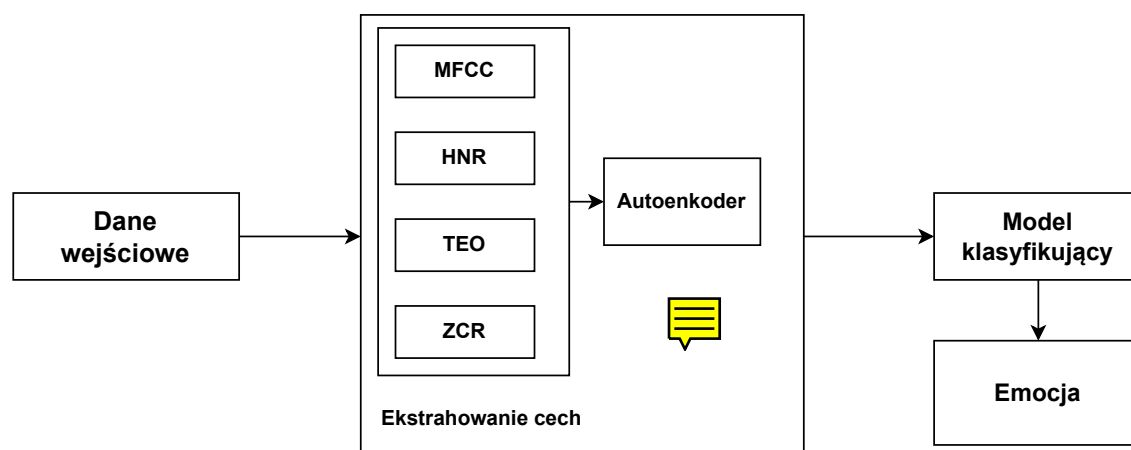
Arabic Vocal-Emotions Dataset, SAVEE (ang. *Surrey Audio-Visual Expressed Emotion*), EMOVO oraz EMO-DB. BRAVED jest kolekcją plików audio zawierających 7 słów w języku arabskim w celu określenia emocji. Dodatkowo każde słowo zostało podzielone na trzy klasy 0, 1 i 2, które korespondują z ludzkimi emocjami. Klasa 0 określa niski poziom emocjonalny, podobny do zmęczenia, bądź przygnębienia[53]. Przypadku 1 opisywane są emocje neutralne, a klasa 2 określa próbki głosowe z wyższym nakładem emocjonalnym takim jak radość, smutek i złość. Dataset zawiera 1935 próbek nagranych przez 61 osób. SAVEE jest zbiorem danych zawierających 480 nagrań audio w języku angielskim określających 7 typów emocji. Klasyfikowane emocje to: radość, smutek, wstręt, złość i stan neutralny.

EMOVO jest zbiorem danych złożonym 588 wypowiedzi w języku włoskim. Nagrania zostały stworzone przez wypowiedzi 6 aktorów. Dodatkowo dataset został podzielony na dwie grupy z 24 anota-torami w każdej grupie. Wykrywa 6 emocji takie jak wstręt, strach, gniew, radość, smutek i stan neutralny.

Kolejnym używanym przez nich zbiorem danych jest EMO-DB. Jest to niemiecki zbiór danych, zawierających 535 plików audio o różnych długościach nagranych przez 5 mężczyzn i 5 kobiet [53]. Pliki również są podzielone w 7 klas emocji: Wstrętu, wściekłości, znużenia, radości, strachu, stanu neutralnego oraz smutku.

5.3. Preprocessing danych (Jakub Bieńkowski)

Artykuł [49] porusza problem wrywania emocji z głosu przy użyciu złożonego systemu preprocesowania danych, który jest złączeniem 39 współczynników MFCC, ZCR, HNR (ang. *Harmonic to Noise Rate*) oraz TEO (ang. *Teager Energy Operator*). Dane te później są przekazywane najpierw do SVM (ang. *Support Vector Machine*) w celu wykrycia emocji. Następnie współczynniki wysyłane są do autoenkodera w celu ekstrakcji znaczących parametrów z wydobytych współczynników, które później są klasyfikowane jako emocje przy pomocy SVM. Schemat działania przedstawiony został na rysunku 5.1. Model wyuczony został przy pomocy datasetu *RML emotion database* zawierającym 720 audiowizualnych próbek przedstawiających emocje takie jak: gniew, wstręt, strach, smutek, radość i zaskoczenie. Model osiągnął dokładność wykrywania emocji równą 70,37% dla metody bez autoenkodera i 72,83% z autoenkoderem [49].



Rysunek 5.1: Schemat rozwiązania [49]

Współczynniki MFCC obliczane są poprzez połączenie 12 współczynników MFCC + energia, 12 współczynników Δ MFCC i energia oraz $\Delta\Delta$ MFCC + energia [49]. W celu obliczenia Δ MFCC i $\Delta\Delta$ MFCC wykorzystywane są wzory:

$$\Delta Cep(i) = \alpha \sum_{j=1}^2 (Cep(i+j) - Cep(i-j)) \quad (5.1)$$

gdzie α oznacza pewną stałą $\approx 0,2$, a funkcja $Cep(i)$ oznacza dany współczynnik MFCC. $\Delta\Delta Cep(i)$ jest obliczany w ten sposób:

$$\Delta\Delta Cep(i) = \Delta Cep(i+1) - \Delta Cep(i-1) \quad (5.2)$$

Kolejnym współczynnikiem jest ekstrahowany przy pomocy funkcji TEO. Przeznaczona jest ona do rozróżniania złożonych cech mowy takich jak intonacja, akcent itp.

HNR mierzy proporcje dźwięku harmonicznego w głosie mierzonej w decybelach [49]. Opisuje ona rozkład energii akustycznej pomiędzy harmonijną a nieharmonijną częścią emitowanego spektrum głosowego [49].

ZCR oblicza ilość przejść przez punkt zero w określonej ramce sygnału, dzielonej na ilość próbek w tej ramce [49].

Celem złączenia wszystkich współczynników z powyższych metod w jeden wektor wejściowy jest poprawa identyfikacji emocji.

Augmentacja wyżej wymienionych datasetów dla artykułu [53] odbywa się poprzez użycie trzech technik. Pierwszą z nich jest nałożenie szumu Gaussa (ang. *Gaussian noise*) na dane treningowe. Jest on charakteryzowany przez wartość amplitudy σ . Dobór odpowiedniej wartości σ jest kluczowy, ponieważ zbyt mała wartość może zmniejszać wydajność modelu, a zbyt duża może utrudniać optymalizację modelu [53]. Celem techniki jest zwiększenie różnorodności danych treningowych, co zapobiega występowaniu przeuczenia modelu. Następnie używana jest technika *Pitch shifting* polegająca na zmianie wysokości tonu sygnału mowy. Zmiana następuje poprzez przesunięcie wysokości tonu o określoną liczbę kroków [53]. Ostatnia użyta technika augmentacji danych w tym rozwiązaniu to *Time Stretching*. Polega ona na zmianie tempa sygnałów mowy bez zmiany tonu [53]. Ta technika pozwala uzyskać dłuższych i krótszych nagrań, co zwiększa różnorodność danych treningowych. Zastosowanie wyżej wymienionych technik ma na celu zwiększenie ilości i różnorodności danych treningowych, co pomaga w przypadku przeuczenia modelu, jak i zwiększa zdolność generalizacji nowych danych.

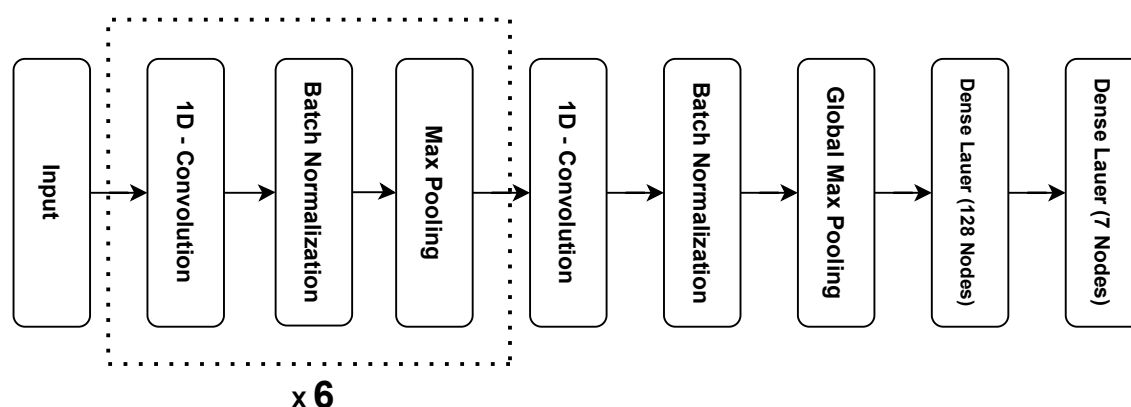
Do zadania preprocessingu danych użyto cech ekstrahowanych z wyżej wymienionych datasetów przy pomocy: mel-spektrogramu, chromatogramu, MFCC, Δ MFCC, $\Delta\Delta$ MFCC, reprezentacji *Tonnetz* oraz kontrastu spektralnego. Do wydobycia cech przy pomocy mel-spektrogramu, sygnały są dzielone w ramki, dla których stosuje się szybką transformatę Fouriera [53]. Spektrum częstotliwości jest podzielone przez częstotliwości o równych odstępach, w celu uzyskania skali Mel dla ramki sygnału. W kontekście cech chromatogramu używa się 12 różnych klas wysokości dźwięku, korzystając z transformaty STFT (ang. *Short-term Fourier transform*) oraz grupowania [53]. Cechy te mają na celu różnicować harmonię i klasy wysokości dźwięku. Cechy MFCC reprezentują krótkoterminowe spektrum mocy, podczas gdy cepstrum w skali melowej ma równo rozmieszczone pasma częstotliwości i umożliwia lepszą klasyfikację

poprzez pomiar zdolności ludzkiego ucha do odbioru sygnału audio [53]. MFCC, Δ MFCC i $\Delta\Delta$ MFCC tworzą trzy zestawy po 40 ekstrahowanych cech. *Tonnetz* jest relacją między wzrostem i spadkiem sygnału mowy w sieci harmonicznej i jest opisana jako sześciowymiarowa przestrzeń wysokości dźwięku [53]. Cechy wyciągane przy pomocy tej metody mają charakterystykę tonalną i mają ważną rolę w rozróżnianiu dźwięków środowiskowych. Ostatnią metodą ekstrakcji cech jest kontekst spektralny. Polega ona na różnicy średniej kwadratowej między szczytami spektralnymi a dolinami w spektrum ramek sygnału. Całkowita liczba ekstrahowanych cech wynosi 273, które następnie są używane w treningu proponowanego modelu.

5.4. Modele wykrywania emocji

Do tej pory utworzono już wiele modeli opartych o CNN, które są w stanie wykrywać aktualny stan emocjonalny mówcy na wysokim poziomie skuteczności. Jako przykład można zaobserwować wyniki pracy opublikowanej w roku 2019, gdzie opisany został model stworzony przy pomocy sieci konwolucyjnych i użyciu datasetu SAVEE [52]. Model ten składa się z 7 jednowymiarowych warstw splotowych, gdzie po każdej z nich występowała warstwa normalizacji wsadowej i warstwa *Max Pooling* o rozmiarze 2, oprócz po ostatniej, tak jak widać na rysunku 5.2. W każdej z warstw konwolucyjnych znajdowały różne ilości filtrów, kolejno: 32, 64, 128, 256, 512, 1024 i 1024, gdzie rozmiary jąder wynosiły kolejno: 21, 19, 17, 15, 13, 11 oraz 9. Po ostatniej z warstw splotowych zaimplementowano *Global Max Pooling*, a następnie warstwę głęboką o ilości 128 węzłów i ostatnią również głęboką o ilości węzłów 7, ze względu na taką ilość etykiet emocji [52].

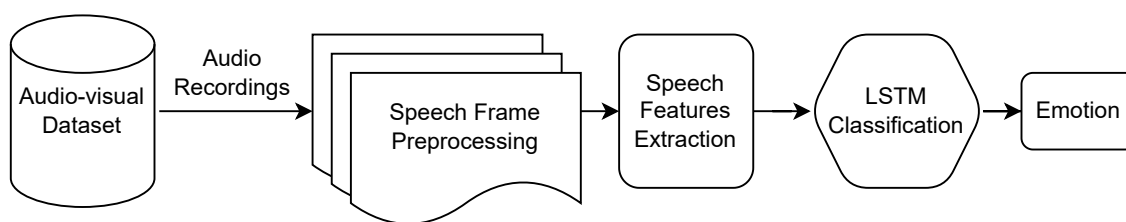
Model ten trenowany był na zbiorze danych SAVEE. Do uczenia użyto także optymalizatora *Adam*, a także jako funkcję straty wykorzystano *caterogical crossentropy*. Proces ten ustawiony został na 400 epok i osiągnął średnią dokładność, na zbiorze testowym złożonym z 80 plików, równą 83,61%.



Rysunek 5.2: Schemat modelu CNN [52]

Przykład użycia LSTM do propozycji rozwiązania problemu SER stanowi praca [54]. Zaproponowano w niej system przedstawiony na rysunku 5.3, a do jego trenowania użyto zbiór danych *Ryerson Multimedia Lab* (RML), który posiada audio-wizualne nagrania sześciu emocji nagranych za pomocą kamery cyfrowej. Każdy kolejny element systemu miał odpowiednie zadanie. Pierwszy blok odpowiadał za ekstrakcję plików audio z RML w częstotliwości próbkowania równej 16000 Hz. Kolejno usuwane

są cisza i szum z plików dźwiękowych. Następnie ekstrahowane są cechy dźwięku za pomocą MFCC i tak powstałe wektory przekazywane są jako wejście do LSTM. Model trenowano przez 4000 epok i dla *timestamp* równego 10 otrzymano *accuracy* na zbiorze testowym równe 97% [54].



Rysunek 5.3: Schemat blokowy propozycji rozwiązania problemu SER z użyciem LSTM [54]

Do oceny uczenia proponowanego przez autorów artykułu [53] modelu transformer zostały użyte cztery metryki oceny takie jak dokładność, precyzja, czułość i miara F1. Dla każdej prognozy modelu, użyto macierzy pomyłek, w celu wykrycia prawdziwych pozytywnych, fałszywie pozytywnych, prawdziwie negatywnych i fałszywie negatywnych wyników [53]. Metryka dokładności jest używana do obliczania częstotliwości, z jaką klasy dźwięku mogą być dokładnie określone z całego sygnału mowy. Czułość za to sprawdza liczbę pozytywnych przypadków, które są dokładnie wykrywane przez proponowany model, a precyzja sprawdza jak dokładnie wykryte są rzeczywiste wypowiedzi za pomocą precyzji. Natomiast miara F1 sprawdza równowagę między precyzją a czułością danego modelu. Metryki oceny używane są do oceny proponowanego modelu Transformer i są powszechnie stosowane do mierzenia wydajności modeli używanych do wykrywania, klasyfikacji i systemów prognozowania [53]. Proponowany model Transformer bazuje na standardowej architekturze opisanej w rozdziale 4.5. Dane wejściowe modelu przyjmowane są jako sekwencję tokenów, gdzie każdy token reprezentowany jest przez 273-wymiarowy wektor, który oznacza ilość wyekstrahowanych cech. Warstwa enkoder posiada 2656905 trenowanych parametrów, a warstwa wyjściowa posiada 7 klas wynikowych przypisujących odpowiednią emocję dla wejściowej próbki głosowej. Sam model był trenowany dla każdego datasetu oddzielnie. Każdy zbiór danych został podzielony na zbiór treningowy posiadający 80% wszystkich nagrań i zbiór testowy złożony z reszty nagrań.

Dla datasetu BAVED model ma inną warstwę wyjściową złożoną z 3 klas, w których wykrywane są takie emocje jak zmęczony/wykończony, stan neutralny oraz radość/smutek/złość [53]. Wynik średni predykcji emocji dla zbioru testowego wynosił 95%. W przypadku EMO-DB średni wynik spośród 7 emocji dla zbioru testowego wynosił 93%, dla zbioru SAVEE wynosił 84%, a dla EMOVO 91%. Wyniki zostały również porównane z bazowymi metodami SER (ang. *Speech Emotion Recognition* opartymi o modele CNN (*Convolutional neural network*) trenowanym na datasecie EMO-DB, DCNN (*Deep Convolutional neural network*) trenowanym na zbiorach SAVEE i EMO-DB, rozmytym drzewie decyzyjnym trenowanym na EMO-DB oraz SVM (*Support Vector Machine*) trenowanym przy pomocy EMOVO. Wyniki dla rozwiązania ukazanego w pracy były wyższe niż wyniki z bazowych metod w porównaniu do odpowiadających zbiorów danych.

5.5. Podsumowanie

W propozycjach rozwiązań problemu SER używane są różnorodne podziały emocji, zbiory danych, metody ich przetwarzania, czy modele. Z ich przeglądu wywnioskowano, że podział emocji w tej pracy powinien opierać się o dobrane datasety, zbierając dane tak, aby były w podobnej ilości dla każdej z emocji. Ze względu na wiele propozycji modeli, w naszej pracy porównane zostaną rozwiązania wykorzystujące: CNN wraz z preprocessingiem używającym ZCR, RMSE i MFCC; LSTM z preprocessingiem używającym tych samych metod, CNN tylko z MFCC i sieć typu transformer.

6. OPIS ROZWIĄZAŃ (JAKUB MAKOWSKI)

W niniejszym rozdziale dokładnie opisane zostały cztery modele służące do wykrywania emocji z głosu. Trzy pierwsze to gotowe modele, odpowiednio opisane w literaturze naukowej z przetrenowanymi wagami, które zostały w kontekście niniejszej pracy zaadoptowane do problemu. Ostatni zaprezentowany model został oparty o transformer i stanowi autorską propozycję rozwiązania problemu detekcji emocji. Do uczenia użyte zostały ogólnodostępne zbiory danych.

6.1. Użyte zbiory danych

Pierwszym używanym w naszej pracy datasetem jest *Ryerson Audio-Visual Database of Emotional Speech and Song* (RAVDESS), który zawiera łącznie 1440 plików audio (16-bit, 48kHz, w formacie Waveform Audio File – WAV) z nagraniami 24 profesjonalnych aktorów (12 kobiet, 12 mężczyzn), którzy mówią w neutralnym akcencie północnoamerykańskim. Na nagrania składają się dwa stwierdzenia zagrane (nie są to nagrania z codziennych sytuacji) z różnymi emocjami: szczęście, smutek, złość, strach, zaskoczenie, wstręt oraz emocja neutralna (spokój), przy czym każde z nich wypowiedziane jest w dwóch poziomach intensywności (normalnej i silnej). Wszystkie z plików posiadają unikalne nazwy, gdzie kolejne numery oddzielone myślnikami oznaczają: modalność, kanał wokalny, emocję, intensywność, stwierdzenie, powtórzenie i numer aktora [55].

Drugi, *Toronto Emotional Speech Set* (TESS), jest to zbiór nagrań w języku angielskim, które wypowiedzane zostały w zdaniu nośnym "Say the word ..." o odpowiednim (granym) zabarwieniu emocjonalnym. W nagraniach przedstawione jest siedem emocji: złość, wstręt, strach, szczęście, przyjemne zaskoczenie, smutek oraz stan neutralny. Łącznie w datasetcie znajduje się 2800 plików audio [56].

Kolejny użyty zbiór danych, *Crowd-sourced Emotional Multimodal Actors Dataset* (CREMA-D), składa się z 7442 plików audio nagranych przez 91 aktorów (48 mężczyzn, 43 kobiety) w wieku od 20 do 74 lat, którzy reprezentują różne rasy i pochodzenie etniczne (Azjaci, Afroamerykanie, Kaukascy, Hiszpanie i nieokreśleni). Aktorzy mieli za zadanie wypowiedzieć jedno z dwunastu zdań na czterech różnych poziomach intensywności (niski, średni, wysoki i nieokreślony), z okazaniem jednej z sześciu emocji: złość, wstręt, strach, szczęście, smutek oraz emocja neutralna [57].

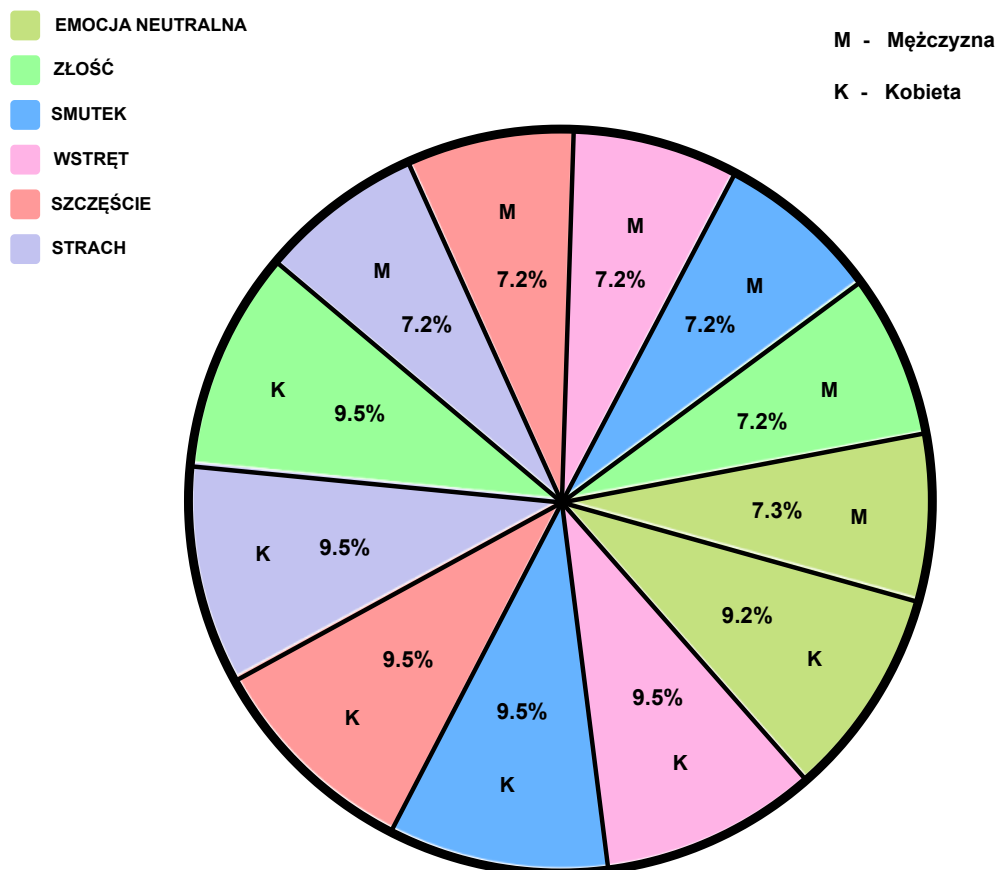
Ostatnim wykorzystanym datasetem został *Surrey Audio-Visual Express Emotion* (SAVEE). Jest to baza danych nagrana przez czterech mężczyzn, dla których językiem ojczystym jest angielski (oznaczeni jako DC, JE, JK, KL), w wieku od 27 do 31 lat. Znajdujące się w niej emocje również posiadają siedem kategorii, tak jak wyżej opisane bazy. Łącznie znajduje się w niej 120 wypowiedzi każdego z mężczyzn, co daje 480 nagrań audio [58].

Każdy z datasetów pełni szczególną rolę w uczeniu opisanych niżej modeli. Dzięki nim otrzymujemy znacznie lepsze wyniki, zapobiegamy przetrenowaniu, a także umożliwiamy dokładniejsze detekcje dla np. różnych płci. Z powodu małych ilości plików z emocją - zaskoczenie, została ona pominięta w trenowanych przez nas modelach. Elementem działania, który wykonywany był w naszej pracy na zbiorach danych, jest ekstrakcja cech (ang. *feature extraction*). Ma ona za zadanie przekształcić surowe dane au-

dio w zestaw numerycznych reprezentacji, aby umożliwić modelowi uczenie na nich. W tym etapie użyto trzy funkcje ekstrakcji:

- **zcr()** - mierzy ilość przejść sygnału przez zero, co łatwo i skutecznie umożliwia charakteryzację dźwięku
- **rmse()** - oblicza średnią kwadratową wartości sygnału, co stanowi miarę jego energii
- **mfcc()** - służy do opisu ogólnego kształtu dźwięku.

Jak można zauważyć na rysunku 6.1, ilość zebranych plików w wyniku łączenia dla każdej emocji okazuje się być podobna. Wszystkich plików audio użytych do trenowania modeli było 11510. Stosunek nagrań mężczyzn do nagrań kobiet wyniósł 4974:6536, gdzie emocje: strach, złość, szczęście, wstręt, smutek, dla kobiet posiadają po 1096 nagrań, dla mężczyzn po 827; emocję neutralną dla mężczyzn wykazuje 839 nagrań, a dla kobiet 1056 nagrań.



Rysunek 6.1: Procentowy podział zebranych plików audio w zależności od emocji i płci

6.2. Model oparty o CNN

Jedno ze znalezionych rozwiązań problemu SER, stanowi "model oparty o CNN" zaprezentowany w [59]. Model ten, wykonany przez Hamida, dawał wyniki w okolicach 97% dla zbioru testowego [59]. Przy uczeniu sieci użyto cztery wyżej wymienione datasety. Na całość składa się kilka istotnych etapów, które zostały poniżej opisane.

6.2.1. Dane uczące

Do trenowania modelu użyta została baza danych, która stanowi połączenie datasetów: RAVDESS, TESS, CREMA-D oraz SAVEE. Po załadowaniu pliki audio z każdego z zestawów przeszły preprocessing. Dla RAVDESS utworzono iterację po katalogach zawierających nagrania aktorów, gdzie następnie rozdzielana jest nazwa pojedynczego pliku, wyodrębniając w ten sposób poszczególne informacje. Stworzone zostały dwie ramki danych (ang. *DataFrame*), jedna dla emocji, druga dla ścieżek plików, a następnie połączono je w jedną tabelę ułatwiającą kolejne działania. Dla datasetów TESS, CREMA-D i SAVEE wykonano takie same operacje, tzn. wyekstrahowano emocje z nazw plików, po czym utworzono ścieżki i *DataFrame*'y, tak jak dla pierwszego ze zbiorów. Dla każdego z nich pominięta została emocja zaskoczenia, aby ilość danych z każdej emocji była przybliżona do siebie.

Następnie wszystkie zbiory połączono w jeden wielki *DataFrame*. W nim znajdowały się kolejno etykiety emocji i ścieżki do plików audio, w których one występują. Łącznie znalazło się w nim 11510 pozycji, podzielone jak na rysunku 6.1. Prawie wszystkie z emocji posiadały taką samą liczbę plików, tzn. tylko emocja neutralna miała ich mniej. Przeprowadzona została także augmentacja danych, w celu zapewnienia jeszcze większej ilości zróżnicowanych danych i zapobieganiu przetrenowania (ang. *overfitting*). W tym właśnie celu wykorzystano cztery funkcje:

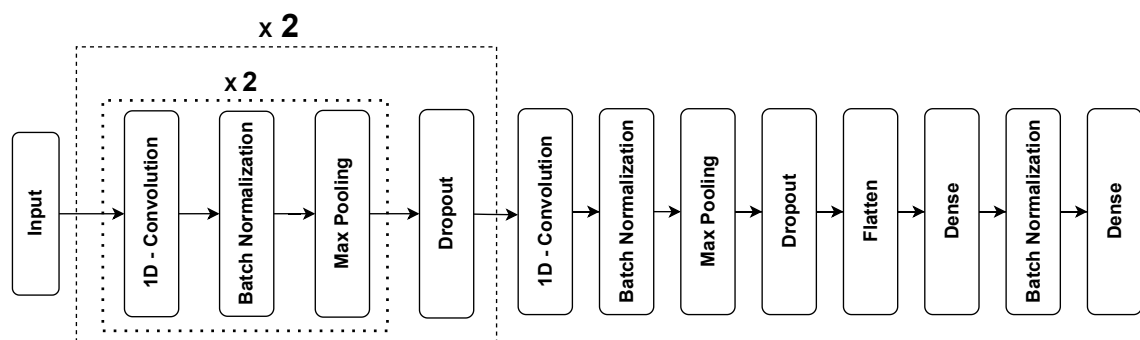
- **noise()** - dodawała szum do dźwięków w sposób losowy. Jej intensywność była proporcjonalna do maksymalnej amplitudy danych dźwięków oraz pomnożona przez losowy współczynnik. Dzięki temu model mógł nauczyć się ignorować nieistotne zakłócenia w danych.
- **stretch()** - zmieniała tempo nie zmieniając przy tym tonacji sygnałów dźwiękowych. Używała ona funkcji *time_stretch* z biblioteki *librosa*, aby rozciągać lub ściskać dane w czasie [60]. Umożliwia to symulację różnego tempa mowy, czy też rytmu.
- **shift()** - przesuwała dane dźwiękowe w czasie. Samo przesunięcie określane było jako losowy zakres. Pozwala to na symulację opóźnień w nagraniach, co ułatwia modelowi radzenie sobie z różnymi warunkami czasowymi.
- **pitch()** - odpowiadała za zmianę tonacji dźwięku, przy zachowaniu tempa. Użyto w niej funkcji *pitch_shift*, która w prosty sposób zmieniała tonację. Pochodzi ona z tej samej biblioteki pythonowej (*librosa*), która znajduje szerokie zastosowanie w zadaniach związanych z przetwarzaniem audio. Model dzięki funkcji *pitch* może później lepiej radzić sobie z różnicami w tonie osób mówiących.

Następnie dokonano ekstrakcji cech z użyciem funkcji opisanych w 6.1 dla każdego z plików audio, przez co tworzone jeden wektor cech nagrania, łączący wszystkie z fragmentów powstałych podczas

ekstrakcji. Przy działaniu tym wykorzystano pierwsze 2,5 sekundy każdego z nagrań, gdyż zauważono, że emocja wyrażona była zawsze w tym czasie. To skutkowało zmniejszeniem ilości niepotrzebnych danych i pozwoliło na przyspieszenie procesu ekstrakcji. Dla okienkowania ustawiono długość ramki równą 2048 próbek, gdyż wartość ta pozwala rozróżniać dźwięki mowy. Natomiast długość przesunięcia przyjęta została jako 512 próbek, aby strata danych nie była zbyt duża. Dla plików surowych oraz augmentowanych wyliczone zostały wartości liczbowe, które zapisane zostały w jeden wektor. Użyto po tym OneHotEncoder(), który służył do przekształcenia etykiet w wektor binarny. W ten sposób utworzono wielki zbiór danych. Podzielono go na dane treningowe zawierające 80% (36832 wektory) oraz testowe odpowiadające reszcie, czyli 20% (9208 wektorów). Tak przygotowany zbiór uczący wykorzystano do trenowania modelu.

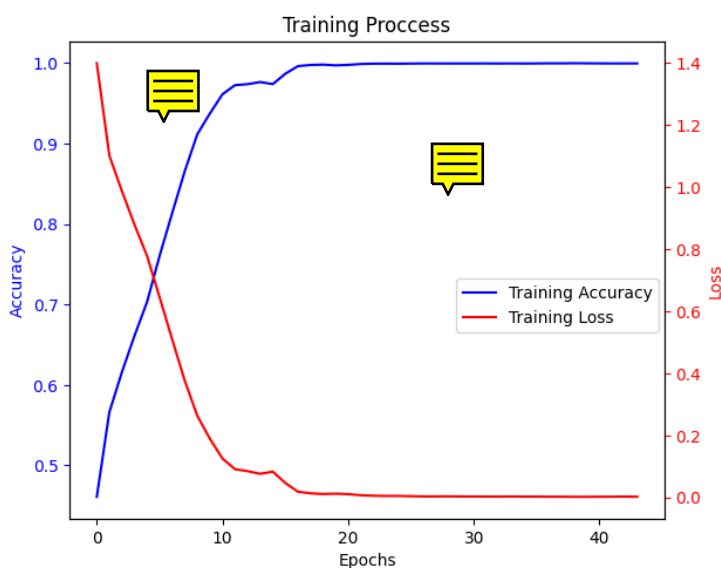
6.2.2. Opis działania

Jako dane wejściowe do trenowania modelu wchodzi wektor danych treningowych o rozmiarze (36832, 2376), gdzie pierwsza liczba oznacza ilość pojedynczych nagrań, a druga jest wielkością wektora danych liczbowych wyekstrahowanych cech. Architektura modelu CNN, widoczna na rysunku 6.2, składa się z pięciu warstw konwolucyjnych. Posiadają one kolejno: 512, 512, 256, 256, 128 filtrów i wielkość okien splotu: 5, 5, 5, 3, 3. Ustawiono je w taki sposób, aby wyjście miało taki sam rozmiar jak dane wejściowe. Po każdej z nich wstawiono warstwę *Batch Normalization*, której zadaniem jest normalizacja danych podczas trenowania za pomocą średniej i odchylenia standardowego, przez co następuje przyspieszenie procesu trenowania. Po każdej z nich użyto warstwy *Max Pooling*, która odpowiadała za redukcję wymiarów danych wyjściowych, aby zabezpieczyć model przed przeuczeniem, gdzie dla każdej ustawiono wartości: maksymalny rozmiar okna pulingowego na 2 i jego przesunięcie przy każdym kolejnym kroku również jako 2. Dodano także warstwy *Dropout* o częstotliwości równej 0,2, które zamieniały losowo część neuronów podczas treningu, by w ten sposób ograniczyć overfitting. Następnie warstwa *Flatten* spłaszczała otrzymane dane wejściowe. Po czym wykorzystano *Dense* z ustawionymi parametrami: 512 filtrów oraz funkcją aktywacji ReLU (pojawiała się również w każdej warstwie konwolucyjnej), co dodawało nieliniowości. Na koniec ustawiona została druga warstwa *Dense* z 6 jednostkami, ze względu na taką ilość etykiet emocji, a także z funkcją aktywacji *softmax*, która odpowiada za obliczenie prawdopodobieństwa przynależności do każdej z emocji.



Rysunek 6.2: Schemat modelu "opartego o CNN"

Kompilacja modelu wykonana została przy użyciu standardowych wyborów dla zadań klasyfikacji emocji, tj. optymalizatora *adam* i funkcji straty *categorical_crossentropy*. Po skompilowaniu nastąpiło rozpoczęcie procesu trenowania. Uznano, że liczba epok równa 50 oraz *batch_size* równy 64 (liczba próbek przetwarzanych naraz) będą najbardziej optymalne. Podczas treningu model był testowany na zbiorze walidacyjnym (9208 wektorów), co pozwalało na monitorowanie wydajności i zapobieganie przetrenowaniu. Do redukcji szybkości uczenia się, gdy nie zauważano poprawy, użyto funkcji *lr_reduce*. Zmieniała ona częstotliwość uczenia się, aby pomóc w dalszej optymalizacji i uniknięciu utknięcia w lokalnych minimach. Dzięki zastosowaniu wszystkich tych elementów proces uczenia, którego wyniki widzimy na rysunku 6.3, stał się bardziej wydajny i mniej podatny na typowe problemy występujące podczas trenowania sieci.



Rysunek 6.3: Proces uczenia modelu "opartego o CNN"

Działanie modelu polega na wczytaniu pierwszych 2,5 sekundy pliku audio, do którego dostęp otrzymuje przez podanie jego ścieżki. Następnie wyekstrahowane cechy przekształcane są na wymagany przez model kształt (1, 2376), skalowane i podawane jako dane wejściowe. Tam następuje stworzenie tablicy numerycznej prawdopodobieństwa przynależności do danej z klas emocji. Wynik przekształcany jest na etykietę i zwracany jako wyjście.

6.3. Model oparty o LSTM

Kolejnym użytym w naszej pracy został model "oparty o LSTM", wykonany przez użytkownika o pseudonimie Gem [61]. Otrzymane przez niego wyniki na danych testowych oscylowały w granicach 90%. Niżej opisane dokładnie zostało przygotowanie danych treningowych, a także sposób i proces uczenia.

6.3.1. Dane uczące

Zbiór danych w tym modelu zbudowany jest również z czterech wyżej wymienionych datasetów i tak jak ustalono dla naszej pracy - zaskoczenie nie zostaje uwzględnione. W odróżnieniu od poprzedniego

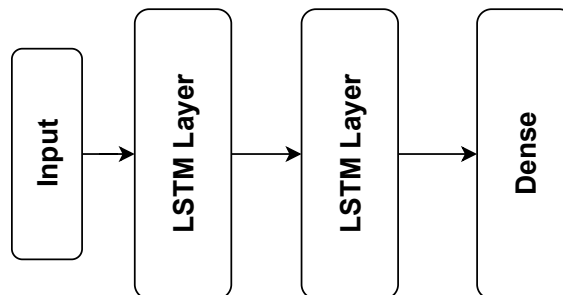
rozwiązania Gem rozdziela pliki audio względem płci, a także wybiera tylko ich część z wybranych datasetów. Po czym w jego rozwiązaniu wybrane zostają nagrania żeńskie ze względu na ich większość, jak widać na rysunku 6.1. W przypadku danych treningowych modelu uczonego przez nas zostawione zostały wszystkie nagrania, a łącznie w naszym zbiorze znalazło się 11510 pozycji.

Dane po połączeniu w jeden zbiór, tak jak w przypadku opisanym w 6.2.1, przeszły preprocessing. Przygotowanie danych dla modelu LSTM różniło się od opisanego wyżej. Po wczytaniu pliku audio zwracana była częstotliwość próbkowania. Następnie próbki zostały przekonwertowane do typu *float32* i usunięto szumy oraz ciszę poniżej 25dB. Następnie wykonano dopełnienie (ang. *padding*), tak aby każdy z sygnałów podawanych na wejście modelu miał taką samą długość, tj. 180000 próbek.

Kolejny krok w preparacji danych stanowiła ekstrakcja cech z plików audio, która została przeprowadzona za pomocą tych samych funkcji, jak w modelu "opartym o CNN". Również dla okienkowania ustawiono długość ramki równą 2048 próbek oraz długość przesunięcia jako 512 próbek. Później użyto *zcr()*, aby zmierzyć częstotliwość zmian sygnału i ułatwić charakteryzację, *rms()*, żeby zmierzyć energię sygnału i ocenić jego głośność oraz *mfcc()* do określenia ogólnego kształtu dźwięku, przy czym liczbę zwracanych wartości MFCC ustawiono na 13. Następnie otrzymane wyniki wprowadzone zostały do macierzy (ang. *matrix*) wraz z etykietami emocji im odpowiadającymi. Dane zostały podzielone na zbiór uczący (10128 nagrań) oraz walidacyjny i testowy w stosunku 88:12, przy czym z tych 12% danych, 30% przypisane zostało jako zestaw testujący (415 plików), a reszta jako zestaw walidacyjny (967 plików). Tak przygotowane dane wykorzystano w procesie trenowania opisanym niżej.

6.3.2. Opis działania

Dane wejściowe wprowadzane do modelu miały rozmiar (10128, 15), gdzie 15 to ilość wartości powstałych podczas ekstrakcji cech: *zcr*, *rms* oraz *mfcc*, a 10128 oznacza liczbę wszystkich wprowadzonych plików. Budowa modelu opiera się o sieć LSTM, a dokładnie o jej dwie warstwy i warstwę gęstą, co widać na rysunku 6.4. Obydwie warstwy LSTM zawierają 64 jednostki. Pierwsza z nich zwraca sekwencję wyjść przy każdym przykładzie wejściowym, co przetwarzane następnie jest przez warstwę drugą. Warstwa druga nie posiada parametru *return_sequences*, przez co zwraca tylko ostatnią sekwencję wyjść, gdyż znajduje się przed warstwą gęstą. Ona zaś składa się z 6 jednostek, ze względu na taką liczbę emocji, a także z funkcji aktywacji *softmax*, by ta przypisywała prawdopodobieństwo należenia do każdej z nich.



Rysunek 6.4: Schemat modelu "opartego o LSTM"

Do kompilacji modelu zastosowano, typową dla zadań klasyfikacji wielokryterialnej, entropię

krzyżową (jak w modelu poprzednim) jako funkcję straty, a także optymalizator *RMSProp*. Trenowanie sieci odbyło się na 300 epokach oraz *batch_size* równym 6, a jego proces można zauważyć na rysunku 6.5. Wykorzystano także funkcję zwrotną, która za zadanie miała zmniejszyć o 0,01 współczynnik uczenia się, w przypadku braku poprawy walidacyjnej dokładności przez 100 epok. Miało na celu to wesprzeć optymalizację oraz zapobiec zatrzymaniu się procesu trenowania na lokalnych minimach.



Rysunek 6.5: Proces uczenia modelu "opartego o LSTM"

Aby zastosować ten model, na początku wprowadzamy plik audio, po czym z jego pierwszych 2,5 sekundy ekstrahowane są jego cechy i zapisywane do wektora. On przechowuje numeryczne dane o cechach sygnału dźwiękowego, które przetwarzane są następnie przez model i następuje predykcja emocji. Zwracane są prawdopodobieństwa należności do każdej z emocji, przy czym wybierana zostaje ta z maksymalną wartością, wtedy wynik przekształcony na odpowiednią etykietę zostaje wysłany na wyjście.

6.4. Model oparty o MFCC

Ostatnim z zapożyczonych rozwiązań użytych w naszej pracy jest model "oparty o MFCC". Nazwano go w ten sposób, ze względu na zastosowanie przy ekstrakcji cech tylko techniki MFCC. Używana w nim do trenowania jest sieć konwolucyjna. Został on wykonany przez Hussiena, a jego dokładność na danych testowych osiągnęła 93% [62]. Poniżej opisane zostało przygotowanie zbioru uczącego oraz dokładny opis rozwiązania.

6.4.1. Dane uczące

Zbiór trenowania w danym rozwiązaniu stworzony został z wykorzystaniem datasetów RAVDESS, TESS, CREMA-D, a także SAVEE. Na samym początku po wczytaniu plików zostały one podzielone ze względu na płeć aktorów, a ścieżki oraz etykiety emocji zapisane do jednego dużego DataFrame'u jak w 6.2.1. Połączono zatem 11510 wszystkich plików, gdzie na rysunku 6.1 możemy zauważyć w jakim stosunku

względem płci wykorzystano nagrania. Dodatkowo wykonana została augmentacja danych, aby zwiększyć ich ilość danych i zapobiec w ten sposób przetrenowaniu sieci. Użyto w tym celu takie funkcje jak w przypadku modelu "opartego o CNN", przy czym oprócz *noise()*, *stretch()*, *shift()* i *pitch()* zastosowano także:

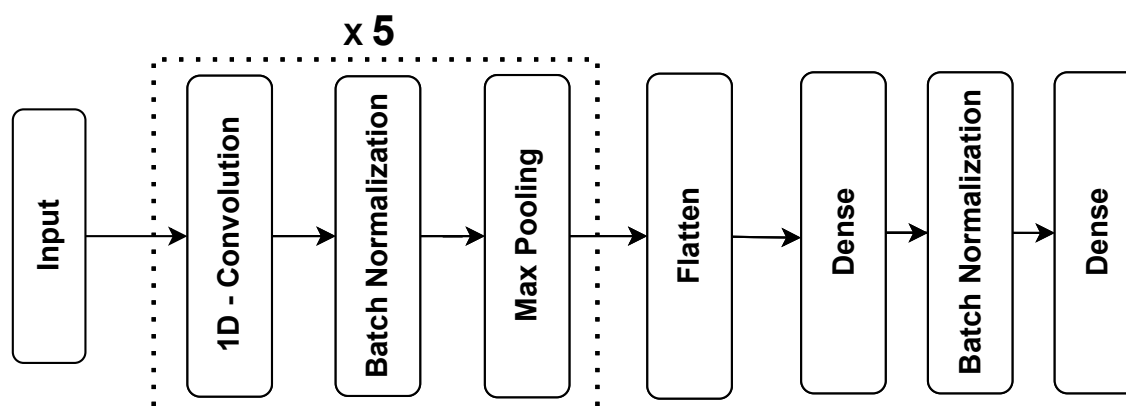
- **higher_speed()** ma za zadanie zwiększyć tempo nagrania dźwiękowego. Wykorzystuje ona do tego funkcjonalność biblioteki librosa *time_stretch()*, co sprawia że działa w podobny sposób do *stretch()*. Ustawiono w niej parametr odpowiadający za prędkość na 1,25, aby otrzymać przyspieszenie nagrania o 25%.
- **lower_speed()** jest przeciwieństwem wyżej opisanej funkcji. Ma ona na celu zmniejszenie tempa nagrania dźwiękowego, używając też *time_stretch()*, a jej parametr ustawiony został na 0,75. W ten sposób otrzymujemy nagranie spowolnione o 25%.

W modelu tym, tak jak i w poprzednich dane przed rozpoczęciem trenowania muszą zostać specjalnie przygotowane. Proces ten składa się z kilku części. Na samym początku każdy plik dźwiękowy jest wczytywany, gdzie długość wczytywanego fragmentu wynosi 2,5 sekundy ze względu na występowanie w tych ramkach emocji. Po tym następuje ekstrakcja cech sygnałów dźwiękowych dla wszystkich oryginalnych danych, za pomocą techniki MFCC, gdzie ilość zwracanych wartości ustawiono na 58. Otrzymane wyniki wprowadzane są do macierzy, po czym następuje augmentacja danych z użyciem każdej z wyżej wymienionych funkcji. Dla każdego z powstałych zbiorów przeprowadzana jest ekstrakcja cech i zapis otrzymanych wyników (wertikalnie) do tej samej macierzy. Dzięki tym działaniom dla każdej z emocji otrzymano 6 razy więcej danych i zwiększono efektywność rozwiązania. Następnie etykiety emocji łączone są z danymi numerycznymi. Użyto po tym *OneHotEncoder()*, który służył do przekształcenia etykiet w wektor binarny. Wszystkie surowe i augmentowane dane podzielono na zbiór treningowy, stanowiący 80% całości (64456 wektorów), oraz walidacyjny jako pozostałe (16114 wektorów). Na zakończenie dane uczące zostały przekształcone do wymiarów kompatybilnych z siecią CNN, gdzie ich ostateczny rozmiar wyniósł (64456, 58, 1), gdzie 64456 oznacza wszystkie pojedyncze wektory treningowe, 58 to liczba wartości MFCC, a 1 określa cechę do której przynależy, w tym przypadku jedną z etykiet emocji.

6.4.2. Opis działania

Model głębokiej sieci konwolucyjnej wykonano z użyciem sekwencyjnego modelu *Sequential*. Na niego składa się wiele warstw, pokazanych na rysunku 6.6. Użyto pięciu warstw konwolucyjnych. Dwa pierwsze posiadające 512 filtrów i rozmiarze jąder równym 5. Kolejne dwie warstwy złożone z 256 jednostek, jedna o rozmiarze 5, druga 3. Natomiast ostatnia złożona była z 128 jednostek i *kernel_size* też równym 3. Wszystkie z warstw konwolucyjnych stosowały funkcję aktywacji ReLU, dodając tym samym nieliniowości do modeli i umożliwiając jego trenowanie na bardziej złożonych wzorcach. Po każdej z tych warstw zastosowano normalizację wsadową oraz warstwy typu *Max Pooling*. *Batch normalization* pomagało w regulacji oraz stabilizacji uczenia się, a warstwa *Max Pooling* zmniejszała ryzyko nadmiernego dopasowywania. Po 5 zestawach warstw jako kolejną zastosowano warstwę *Flatten*, która spłaszczała dane

wielowymiarowe otrzymywane ze swoich poprzedników. Tworzyła ona przy tym jednowymiarowy wektor, co było konieczne przed wprowadzeniem danych do kolejnych (gęstych) warstw sieci. Pierwsza warstwa *Dense* posiadała 512 neuronów, a także funkcję aktywacji ReLU, po czym użyto również normalizacji wsadowej. Ostatni element modelu stanowiła druga warstwa gęsta, przy czym posiadała ona 12 jednostek oraz funkcję softmax, która umożliwiała określić prawdopodobieństwo należności do każdej z 12 klas (6 emocji, w zależności od płci).



Rysunek 6.6: Schemat modelu "opartego o MFCC"

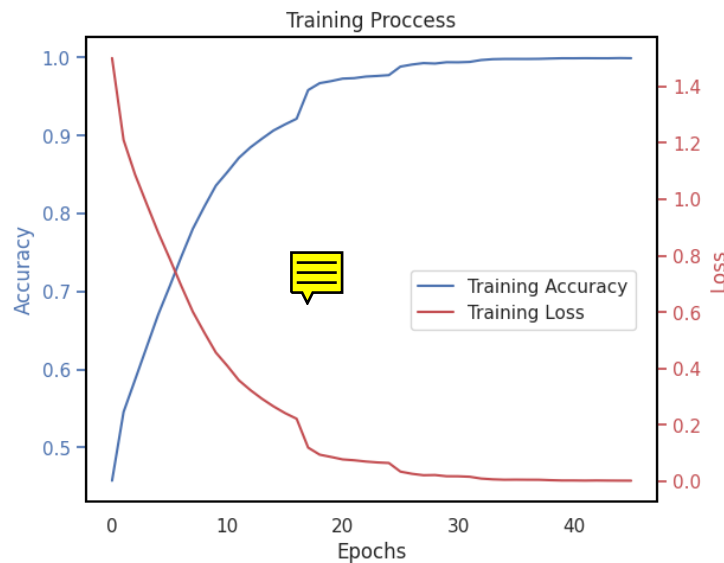
Kompilacja modelu wykonana została z użyciem optymalizatora *RMSProp*, który cieszy się bardzo dużą popularnością przy efektywnym uczeniu sieci. Funkcję straty stanowiło używane także we wcześniejszych modelach *categorical_crossentropy*. Po kompilacji przystąpiono do trenowania modelu, ustawiając ilość epok na wartość 50 oraz *batch_size* na 64, gdzie przebieg wartości funkcji strat i dokładności przedstawiono na rysunku 6.7. Do jak największej optymalizacji uczenia się użyto dwie funkcje zwrotne:

- **ReduceLROnPlateau()** w celu dynamicznej redukcji współczynnika trenowania w trakcie procesu. W tym przypadku ustawiono ją tak, aby oczekiwała na 3 epoki bez poprawy, po czym jej zadaniem było zmniejszenie o 0,5 tego elementu.
- **EarlyStopping()** stosuje się aby zakończyć proces uczenia się przedwcześnie, gdy wyniki się nie poprawiają. Ustawiono tu limit 5 epok nie dających lepszych wyników. Po zakończeniu treningu funkcja ta zwraca wagi modelu, który osiągnął najlepsze wyniki.

Zastosowanie tego modelu działa podobnie do poprzednich, tzn. przed wprowadzeniem do modelu pliku audio, ekstrahowane są z niego cechy i 58 wartości liczbowych zapisywanych jest do wektora. Ten podany zostaje na wejście modelu i następuje predykcja wyrażanej emocji oraz płci. Zwracane są prawdopodobieństwa należności do każdej z nich. Wybrana zostaje etykieta o największym prawdopodobieństwie, po czym następuje jej wysłanie na wyjście.

6.5. Model oparty o transformer (Jakub Bieńkowski)

Zaproponowanym przez nas rozwiązaniem jest model "oparty o transformer". Wykorzystuje on nowoczesny typ uczenia maszynowego, w którym zrezygnowano ze standardowych metod ekstrakcji cech



Rysunek 6.7: Proces uczenia modelu "opartego o MFCC"

takich jak MFCC, ZCR, LPC itd. na rzecz struktury *Wav2Vec2*, która pozwala na działania na nieprzetworzonym sygnale dźwiękowym. Nasz model osiągnął wynik detekcji emocji na poziomie 86% dla zbioru testowego. Dalsze etapy procesu uczenia modelu zostały szczegółowo opisane poniżej.

6.5.1. Dane uczące

Jednym z najważniejszych elementów podczas uczenia maszynowego jest dobranie odpowiednich danych treningowych, w przypadku modelu opartego o transformer użyty został identyczny zestaw zbiorów danych jak w przypadku modelu opartego o CNN. Datasets zostały załadowane przy użyciu biblioteki Kaggle. Następny etap polegał na ujednoliceniu nazw klasyfikatorów emocji dla każdego zbioru danych. Po czym na wzór rozwiązania opartego o CNN został stworzony Dataframe zawierający informacje o emocjach i ścieżkach do danych plików dźwiękowych 6.1. Dane później zostały podzielone na zbiór treningowy składający się z 9730 ścieżek dźwiękowych i zbiór testowy zawierający 2433 takich elementów. Funkcja *librosa.load()* wczytuje pliki, oraz ustawia częstotliwość próbkowania na 16 KHz i przetwarzane audio jest do formatu mono. Tak wczytane dane są rzutowane w obiekty *Dataset*, które są strukturą danych dedykowaną do przechowywania zbiorów. Kolejnym etapem jest zmapowanie ścieżek dźwiękowych do tablic zmiennoprzecinkowych w celu przygotowania danych do ekstrakcji cech stosując do tego funkcję *Wav2Vec2FeautreExtracor*. Jest to komponent modeli *Wav2Vec 2.0* opracowanego przez Facebook AI, który wykonuje zadanie ekstrakcji cech z nagrania audio. Ta funkcja przyjmuje na wejście tablice wartości zmiennoprzecinkowych korespondujących z nieprzetworzonym przebiegiem sygnału dźwiękowego [63]. Jest ona inicjalizowana wstępnie przetrenowanym i skonfigurowanym do naszego problemu wykrywania emocji modelem "superb/hubert-large-superb-er"/, dzięki czemu jesteśmy w stanie ekstrahować cechy w nim używane dla naszych próbek treningowych i testowych. Enkodowane dane przygotowane przy użyciu tej metody są później etykietowane odpowiednimi emocjami. Powodem wykorzystania tej klasy jest to, że cechy wyciągane z funkcji *Wav2Vec2FeautreExtracor* są nieoznaczone, co wynika z

6.5.2. Opis działania

Do zadania wykrywania emocji z próbki dźwiękowej wykorzystujemy *superb/hubert-large-superb-er*, który jest przetrenowanym modelem opartym na architekturze HuBERT (Hidden Unit BERT). Jest on oparty na technice Transformer oraz wykorzystuje samoistne uczenie się (*self-supervised learning*), którego celem jest identyfikacja ukrytych jednostek dźwiękowych [64].

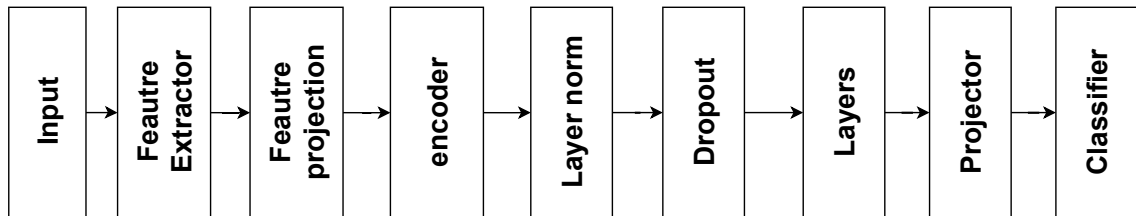
Architektura modelu *superb/hubert-large-superb-er* przedstawiona na rysunku 6.8. Jego działanie polega na tym, że na wejście do modelu wchodzi przetworzony dataset z zakodowanymi cechami i etykietami, które przekazywane są do bloku *Feature extractor*. Zadaniem tego bloku jest redukcja wymiarowości danych i wydobycie z nich istotnych cech. Sam blok zbudowany jest z 7 warstw konwolucyjnych, które posiadają 512 filtrów gdzie dla każdej z tych warstw, stosowane są warstwy normalizacji i funkcja aktywacji GELU (ang. *Gaussian Error Linear Unit*) wprowadzająca nieliniowość modelu w celu uczenia się bardziej złożonych wzorców. Pierwsza warstwa konwolucyjna posiada 512 filtrów, rozmiar jądra równy 10 i przesunięcie równe 5 mające na celu zwiększenie informacji w jednej ramce. Następne 3 warstwy mają zmniejszony rozmiar jądra równy 3 i przesunięcie równe 2. Ostatnie dwie warstwy konwolucyjne w tym bloku posiadają rozmiar jądra równy 2 i takie samo przesunięcie.

Cechy później przechodzą do bloku *Feature projection*. Na ten blok przypadają trzy warstwy: warstwa normalizacji, warstwa liniowa i dropout. Pierwsza warstwa ma na celu, jak sama nazwa wskazuje normalizację wszystkich 512 cech. Warstwa liniowa natomiast służy do transformacji liniowej cech z przestrzeni 512-wymiarowej do 1024-wymiarowej przestrzeni. Ma to na celu lepszą separację klas. Dropout zapobiega nadmiernemu dopasowaniu modelu.

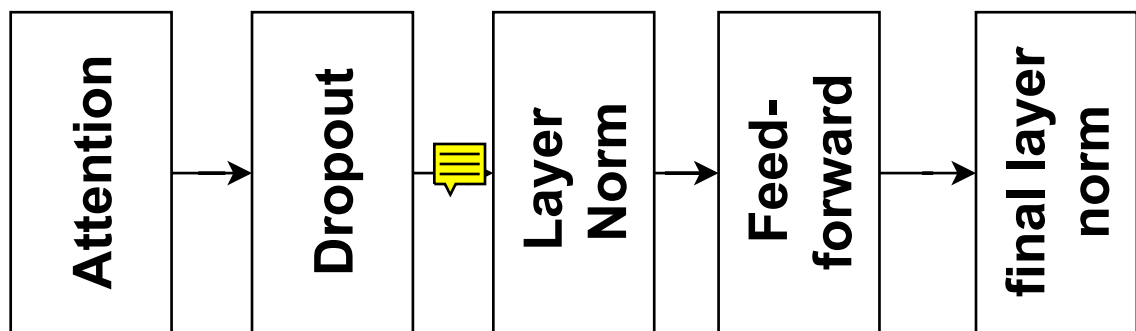
Następnie cechy przesyłane są do bloku enkoder. Zawierają się w nim: warstwa konwolucyjna splotowa, normalizacja wag, warstwa dopełnienia oraz funkcja aktywacji GELU. Pierwsza warstwa przetwarza cechy dodając do nich informacje o pozycji. Rozmiar jądra wynosi 128, przesunięcie 1 oraz dopełnienie równe 64 mające na celu utrzymać długość sekwencji konwolucji. Normalizacja wag ma na celu ustabilizować proces uczenia, a dodatkowa warstwa dopełnienia ma zapewnić, że wymiar sekwencji wyjściowej się nie zmieni. Po tym bloku przechodzimy do warstw normalizacji i warstwy dropout.

Następnym blokiem w procesie uczenia jest blok *Layers*, którego dokładniejsza budowa została pokazana na rysunku 6.9. Blok ten składa się z 24 identycznych warstw enkodera. Blok *Attention* jest mechanizmem samouwagi. Wartości wejściowe zostają podzielone na trzy wektory: wektor zapytania, wektor klucza, wektor wartości poprzez użycie warstwy liniowej. W tej warstwie liniowej przypisywane są różne wagi dla każdego z wektorów, co pozwala na równoczesne skupienie się na wielu cechach w jednej sekwencji. Wektory później są łączone w jeden wektor poprzez kolejną warstwę liniową. Następnie wykonywany jest dropout i normalizacja danych. Znormalizowane wektory przekazywane są do bloku *feed-forward*. Jego celem jest dodanie dodatkowego przetwarzania danych i zdolności do abstrakcji po operacji samouwagi. *feed-forward* najpierw zwiększa wymiarowość danych z 1024 do 4096 w celu generowania bardziej złożonych reprezentacji, po czym nakładana jest funkcja aktywacji GELU. Następnie poprzez użycie kolejnej warstwy liniowej warstwy wymiarowość powraca do wartości 1024. Ostatnimi

czynnościami w tym bloku jest nałożenie warstwy dropout w celu regulacji i końcowej warstwy normalizacji. Po wykonaniu tych czynności dla 24 warstw enkodera dane z bloku *feed-forward* przekazywane są do liniowej warstwy zmieniającej wymiarowość cech z 1024 do 256. Spośród tych cech model klasyfikuje emocje poprzez kolejną redukcję wymiarowości z 256 do 6 w bloku *Classifier*. Całkowita liczba parametrów uczących wynosi 13742198.



Rysunek 6.8: Schemat modelu opartego o transformer



Rysunek 6.9: Schemat bloku Layers

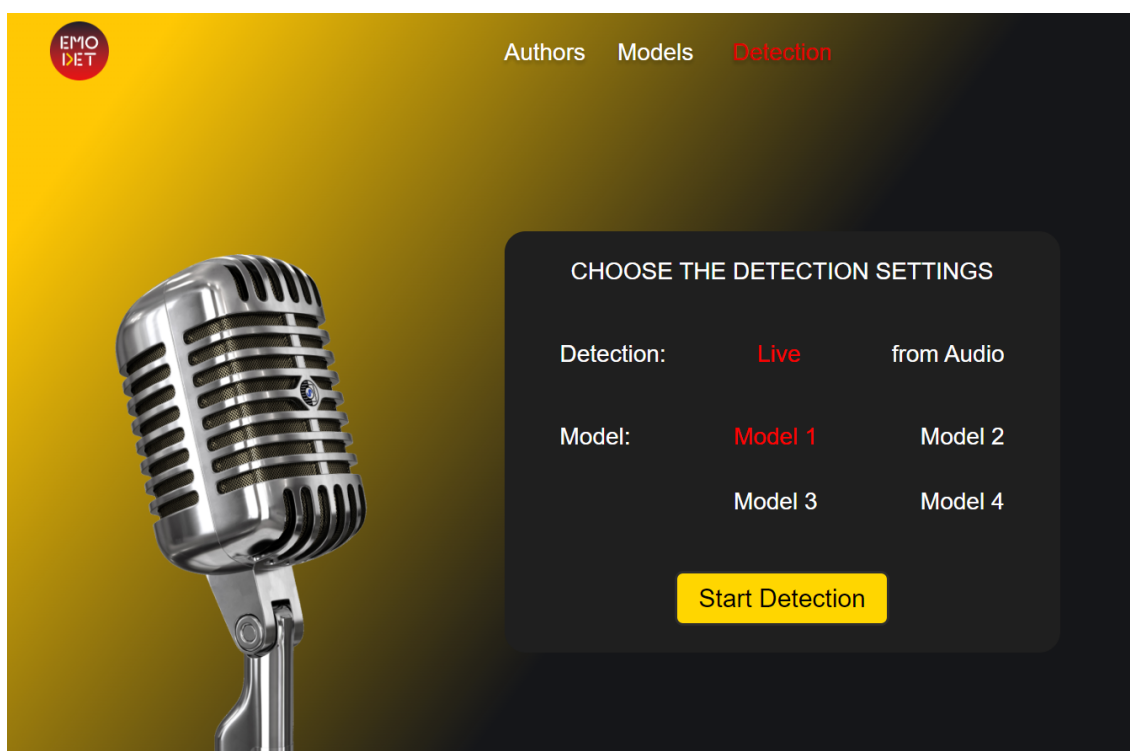
Dla modelu wybrany został optymalizator Adam, którego zadaniem jest minimalizacja funkcji straty jak i aktualizacja wag uczenia modelu. W czasie dotrenowania wykorzystywana jest *prediction()*, której głównym elementem jest użycie *torch.softmax()* dla wyjść modelu w celu uzyskania prawdopodobieństwa z jakim wykrywane są emocje. Proces trenowania modelu składa się z 24 epok, a dane uczące są wczytywane przy użyciu *DataLoader()*. Na wejściu funkcji ustawiany jest zbiór danych, zostaje określona liczba próbek przetwarzanych w partii uczenia poprzez ustawienie *batch_size*, który w tym przypadku wynosi 8, dane treningowe zostają pomieszczone, co ma na celu zapobiegnięcie nadmieremu dopasowywaniu modelu do podawanych mu próbek. Podczas treningu jako dodatkowy optymalizator używamy akumulacji gradientów, co doprowadzić ma do zmniejszenia użycia mocy obliczeniowej dla każdego kroku uczenia maszynowego.

6.6. Aplikacja webowa

"EMODET" to stworzona aplikacja webowa, która służy do wykrywania emocji z głosu. To w niej wykorzystano wszystkie z wyuczonych przez nas modeli, aby użytkownik mógł sprawdzić jakie aktualnie emocje wykazuje lub jakie odczucia wykazywał ktoś w krótkim nagraniu. Została ona wykonana przy użyciu kilku technologii często używanych w dzisiejszych czasach przy software developmencie. Back-end tej strony opiera się na Flasku, który umożliwia proste oraz szybkie nagrywanie audio i przekazywanie wyników na

front, front-end natomiast utworzono przy użyciu Next.js oraz React.js, gdyż technologie te umożliwiają płynną komunikację z serwerem i działanie strony [65][66][67]. Strona, którą widzimy na rysunku 6.10, składa się z komponentów, co stanowi element architektury aplikacji używających React. Dzięki temu łatwo zamieniać różne elementy i ograniczać ilość powtarzającego się kodu. Wielką zaletą jest także brak załadowywania kolejnych stron, gdyż przy użyciu tych technologii wszystko dzieje się na jednej, a zmieniane są tylko komponenty, co przyspiesza działanie witryny.

Najważniejszą część tej aplikacji stanowi server.py, który odbiera zapytania od użytkownika, po czym opracowuje odpowiedź, a ona wysłana jest do odpowiednich komponentów jako wyjście. Tak też działa detekcja emocji. Po kliknięciu przez użytkownika rozpoczęcia procesu, serwer przyjmuje wiadomość, aby zacząć nagrywanie dźwięku. Dwu-sekundowe ramki sygnału dźwiękowego są wtedy zapisywane i przetwarzane zgodnie z przygotowaniem danych do wybranego modelu. Następnie wykonywana jest za jego pomocą detekcja emocji i wynik zostaje przekazany do elementu odpowiedzialnego za wyświetlanie jego na stronie. W przypadku detekcji emocji z pliku, zostaje on podzielony na części, przy czym dla każdej z nich następuje wykrycie emocji i cały ich zbiór zwracany jest użytkownikowi. Oprócz detekcji emocji w aplikacji znajdują się także opisy wszystkich modeli, które użytkownik może przetestować.



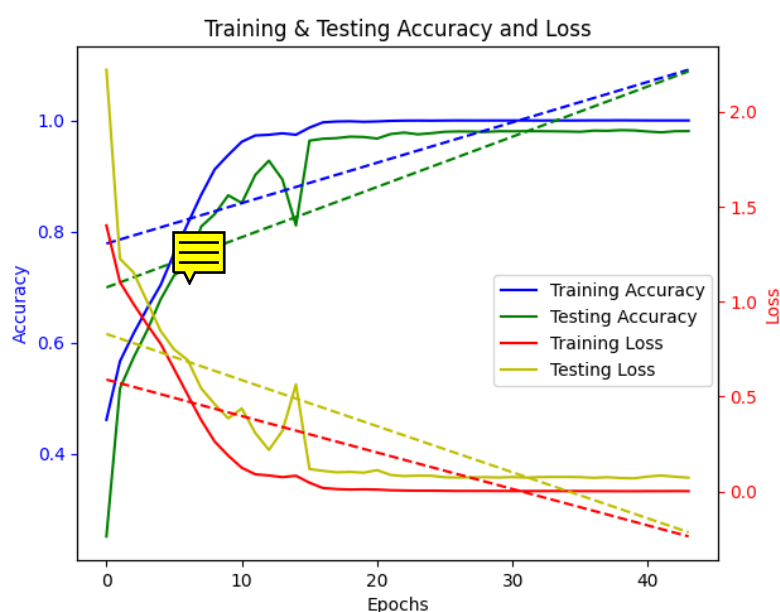
Rysunek 6.10: Aplikacja webowa EMO DET

7. WYNIKI (JAKUB MAKOWSKI)

Problem SER nie jest prosty do rozwiązania, mimo wielu propozycji nie ma jednej, która sprawdzała by się pod każdym względem i zawsze dawała najlepsze wyniki. W tym rozdziale przedstawione zostały otrzymane wartości końcowe kilku z czynników przez nas mierzonych w każdym z modeli. Pokazane zostały też rezultaty testu zrobionego dla porównania wydajności i precyzji na prawdziwym przykładzie. Jako pierwszy opisany został proces uczenia, a później porównanie dokładności modeli względem ich zbiorów testowych oraz rezultaty otrzymane na specjalnie przygotowanym pliku składającym się z wszystkich 6 emocji.

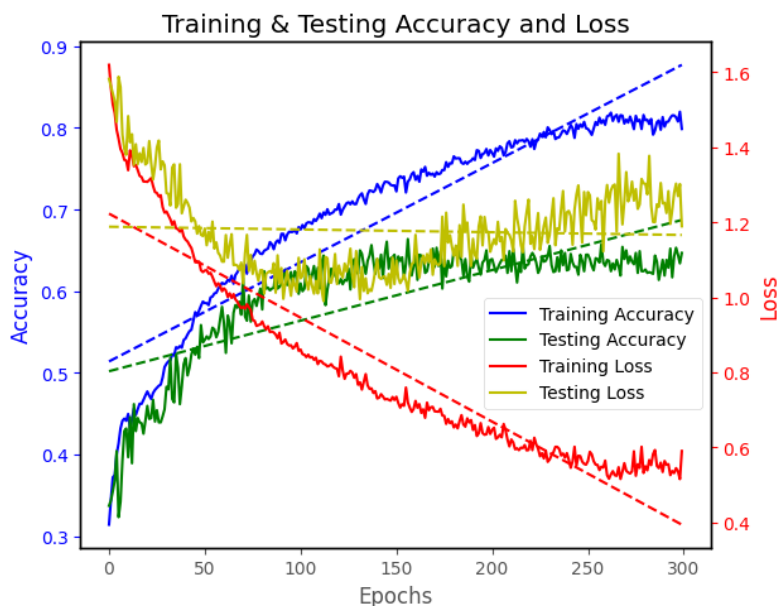
7.1. Trenowanie modeli

Każdy z modeli opisanych w tej pracy posiadał podczas uczenia funkcje nadzorujące ten proces. Ich wyniki zostały zapisane i przedstawione poniżej, przez co określić można jak szybko oraz skutecznie trenowanie to przebiegało. Pierwszym z opisywanych rozwiązań jest model "oparty o CNN". Jego wartości dokładności oraz straty wraz z liniami trendu przedstawione na rysunku 7.1 świadczą o szybkim uczeniu się, gdyż po epoce 15 otrzymywane rezultaty funkcji dokładności oscylują przy wartości 0,99 dla danych treningowych oraz przy 0,97 dla zbioru walidacyjnego. W piętnastej epoce nastąpił skok ze względu na reakcję funkcji *lr_reduce*, która po zauważeniu braku poprawy wyników zredukowała szybkość uczenia się modelu. Koszt podczas uczenia (*Loss*), czyli ilość źle określonych przewidywań modelu w porównaniu do aktualnych wartości cech, świadczy o dokładności, gdzie zauważyć można, iż po 15 epoce wartości znajdują się w okolicach zera. Wynika z tego, że predykcja emocji jest dokładna dla danych treningowych, a także ponadprzeciętna (w okolicach 0,11) dla zbioru walidacyjnego.



Rysunek 7.1: Funkcja dokładności i straty podczas uczenia modelu "opartego o CNN"

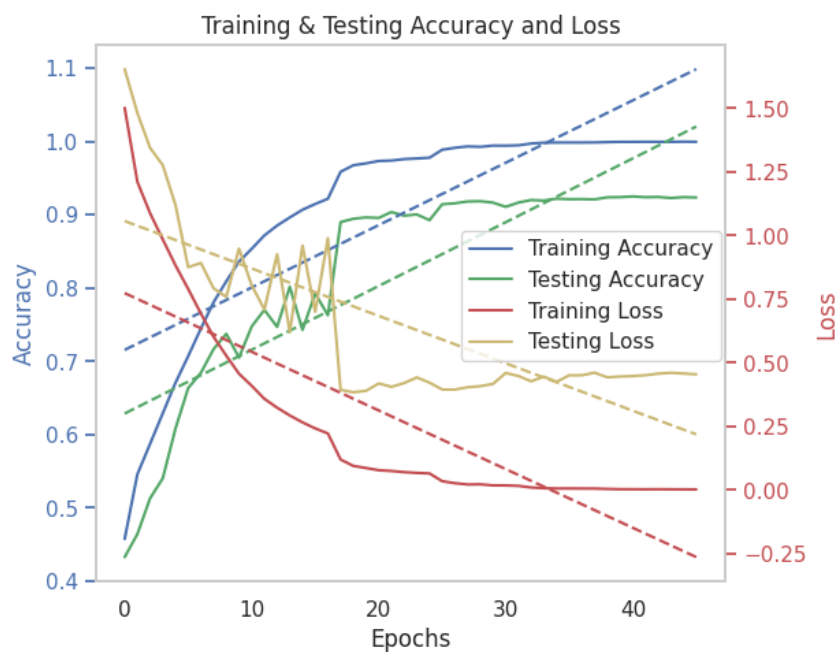
Kolejne wyniki przedstawiają proces uczenia modelu "opartego o LSTM". Z wyników przedstawionych na rysunku 7.2 można zauważyć, że proces uczenia jest bardzo szybki do około 50 epoki. Na danych treningowych funkcja straty zaczęła oscylować po 250 epoce, tak jak i dokładność, natomiast na danych walidacyjnych model nie poprawiał swoich wyników już po 150 epokach, co może świadczyć o zbyt małej ilości funkcji, poprawiających proces uczenia. Wynik uzyskany podczas uczenia dla *accuracy* bliski jest wartości 0,83 dla zbioru testowego oraz 0,64 dla zbioru walidacyjnego. Natomiast wartości straty w przypadku danych walidacyjnych, zamiast zmniejszać się, zaczęły wzrastać od 150 epoki, co może świadczyć o utknięciu w lokalnym minimum i nie działaniu funkcji redukcji szybkości uczenia.



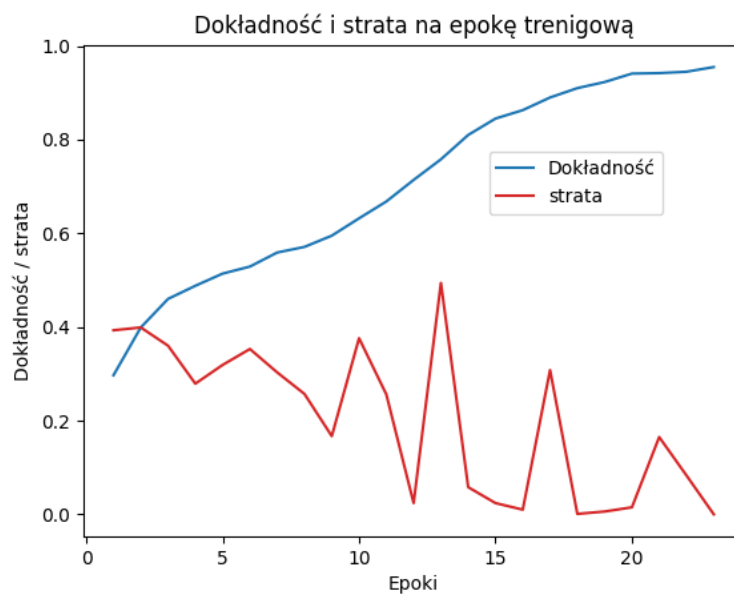
Rysunek 7.2: Funkcja dokładności i straty podczas uczenia modelu "opartego o LSTM"

Następne wyniki, widoczne na rysunku 7.3, przedstawiają etapy uczenia dla modelu "opartego o MFCC". W tym rozwiązaniu trening odbywa się dla 46 epok (zadziałała funkcja *early stop*, z których widać bardzo szybki proces uczenia. Wartości strat dla zbioru treningowego wynosiły ok. zero już około 25 epoki. Dokładność modelu na danych treningowych osiągnęła wartość ok. 1, czyli prawie maksymalną podczas trenowania, również w okolicach 25 epoki, a to może świadczyć o możliwym dążeniu do przetrenowania modelu. Wiadomo jednak, że używana podczas treningu była funkcja *early stop*, która zapobiegła temu zdarzeniu i model w najlepszej swojej wersji nie został utracony. Na danych walidacyjnych wartości *accuracy* doszły do ok. 0,92, a wartości *loss* utrzymały się w okolicach 0,45. Można zauważyć, że kilka razy nastąpiło wywołanie po: 17 epoce, 25, 32 oraz 38; co znacznie poprawiło rezultaty otrzymane na danych testowych.

Dla rozwiązania zaproponowanego w tej pracy, uczenie przebiegało w sposób ukazany na rysunku 7.4. Model ten cechował się stabilnym wzrostem dokładności z każdą kolejną epoką, a wartości straty oscylowały w zakresie od 0 do 0,4, zmniejszając się do wartości w okolicach zera dla 25 epoki. Jak można zauważyć model na tym etapie otrzymał *accuracy* na poziomie około 0,95. Podjęte zostały także próby dalszego uczenia modelu, lecz wyniki dla kolejnych epok nie polepszały się.



Rysunek 7.3: Funkcja dokładności i straty podczas uczenia modelu "opartego o MFCC"



Rysunek 7.4: Funkcja dokładności i straty dla modelu opartego o transformer

7.2. Test dokładności

Modele po trenowaniu zostały także sprawdzone na swoich zbiorach testowych, których wyniki przedstawiono w tabeli 7.1. Zbiory testowe dla modeli opartych o: CNN, LSTM, MFCC, transformer; posiadały kolejno ilości danych: 9208, 967, 16114, 1112. Największą dokładność w tym przypadku otrzymał model "oparty o CNN", co widać także na rysunku 7.5. Popełnił on tylko pojedyncze błędy przy predykcji. Drugi najlepszy wynik otrzymał model "oparty o MFCC", co również ukazuje jego macierz pomyłek przedstawiona na rysunku 7.7. Kolejny w hierarchii dokładności znajduje się model "oparty o transformer", gdzie jak widać na rysunku 7.8, wykazał się bardzo dużą dokładnością na danych treningowych równą 88,40%. Najgorzej ze wszystkich wypadł model "oparty o LSTM", gdzie jego dokładność predykcji, widoczna również na rysunku 7.6, wyniosła tylko 64,68%. Należy pamiętać, że zbiory testowe każdego z modeli miały różną wielkość oraz posiadały inne nagrania, dlatego też nie świadczy to o tym, który z nich jest tak naprawdę najbardziej precyzyjny w wykrywaniu emocji.

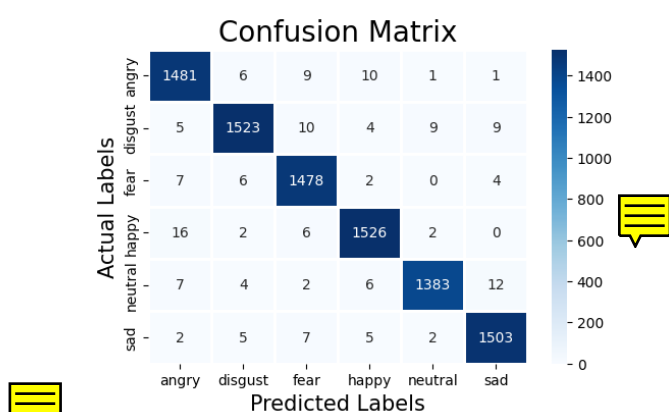
Tabela 7.1: Dokładność modeli na własnych zbiorach testowych

CNN	LSTM	MFCC	Transformer
98,22%	64,68%	92,47%	88,40%

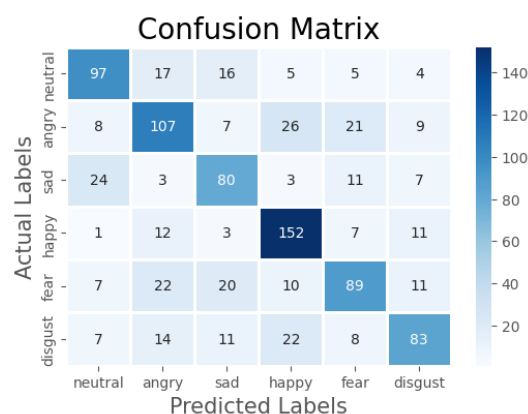
Na potrzeby porównania każdego z modeli w rzeczywistym przypadku, stworzono kilka plików audio zawierający wszystkie z sześciu emocji, wykazane w języku angielskim. Połączono kolejno szczęście, smutek, strach, złość, spokój oraz wstręt. W tabeli 7.2 przedstawiono wyniki testu. Pliki te zostały wprowadzone do detekcji dla każdego z modeli, po czym ich zadaniem było określenie znajdujących się w nich emocji. Najgorzej spośród wszystkich spisał się model "oparty o LSTM", wykazał on ok. 83,33% poprawności, przy czym najgorzej radził sobie z wykrywaniem emocji neutralnej (spokoju) i smutku, a bez pomyłek określił za każdym razem złość oraz wstręt. Modele bazujące na sieciach konwolucyjnych, tzn. model "oparty o CNN" i "oparty o MFCC", wykazały stuprocentową poprawność detekcji emocji. Natomiast model "oparty o transformer" najgorzej radził sobie z wykrywaniem emocji smutku i złości, a tylko dla wstrętu nie pomylił się ani razu i łącznie wykazał dokładność detekcji ok. 86,36%.

Tabela 7.2: Wyniki testu detekcji w języku angielskim przeprowadzonego na każdym z modeli

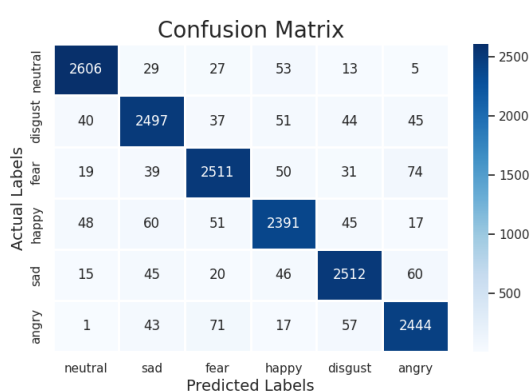
Emocje w audio	CNN	LSTM	MFCC	Transformer
Szczęście	11/11	9/11	11/11	10/11
Smutek	11/11	6/11	11/11	7/11
Strach	11/11	10/11	11/11	10/11
Złość	11/11	11/11	11/11	9/11
Spokój	11/11	8/11	11/11	10/11
Wstręt	11/11	11/11	11/11	11/11
Dokładność	66/66	55/66	66/66	57/66



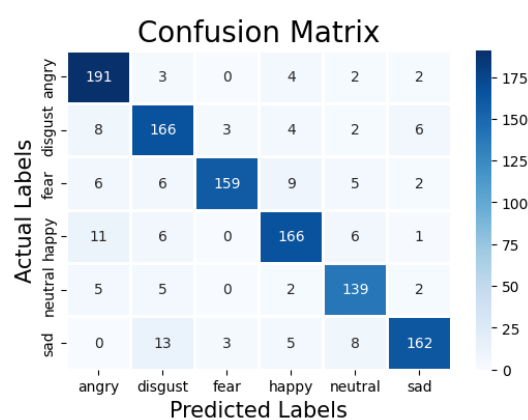
Rysunek 7.5: Macierz pomyłek model "oparty o CNN"



Rysunek 7.6: Macierz pomyłek model "oparty o LSTM"



Rysunek 7.7: Macierz pomyłek model "oparty o MFCC"



Rysunek 7.8: Macierz pomyłek model "oparty o transformer"

Przeprowadzono także testy na ścieżkach dźwiękowych w języku polskim, niestety nie otrzymano zadowalających wyników, przez co warto zwrócić uwagę, że rozwiązania te działają najlepiej w języku angielskim. Poprawa rezultatów dla języka polskiego mogłaby nastąpić, gdyby douczono każdy z modeli na odpowiednio dużej ilości nagrań w danym języku.

7.3. Podsumowanie

Z powyższych wyników wywnioskować możemy, że każde z rozwiązań otrzymuje zadowalające wyniki dla próbek w języku angielskim. Najgorzej jednak radzi sobie z tym model "oparty o LSTM", a to spowodowane jest pewnie za małym zbiorem uczącym oraz liczbą epok trenujących. Rozwiązanie zaproponowane w tej pracy jest porównywalnie dobre do modeli opartych o CNN oraz MFCC. Na podobnych zbiorach testowych otrzymały one bardzo wysokie wyniki, a ich dokładność sprawdzona podczas testu nie zawiodła.

8. ZAKOŃCZENIE (JAKUB BIEŃKOWSKI, JAKUB MAKOWSKI)

Powyższa praca przedstawia przykładowe rozwiązania problemu SER w oparciu o algorytmy uczenia maszynowego. Po przestudiowaniu zagadnień teoretycznych jak i po przeszukaniu ogólnie dostępnych rozwiązań, zdecydowano użyć modeli opisanych w rozdziale 6.1. Przedstawione przez nas rozwiązania składają się z następujących modeli: oparty o MFCC, CNN, LSTM, Transformer. Najlepsze wyniki na zbiorze testowym osiągnął model "oparty o CNN", którego dokładność wyniosła 98,22%, co jest wynikiem ponad przeciętnym. Natomiast wyniki najgorsze osiągnęło rozwiązanie "oparte o LSTM", dla którego rezultaty podczas testu dokładności, którego wyniki ukazano w tabeli 7.2, pokazały, że model ma problemy z rozpoznawaniem w szczególności emocji neutralnej oraz smutku.

8.1. Ograniczenia i problemy

Głównym problemem napotkanym podczas trenowania była moc obliczeniowa. Zbiory danych złożone z plików dźwiękowych są w stanie zająć do kilkudziesięciu gigabajtów, co znacznie obciążało CPU (*Central Processing Unit*) oraz GPU (*Graphics Processing Unit*). Problem ten znacznie wpływał na możliwości tworzenia bardziej skomplikowanych modeli oraz korzystania z większych zbiorów danych w trakcie uczenia. Kolejną dużą barierą funkcjonalności aplikacji jest poprawne działanie wykrywania emocji wyłącznie w języku angielskim, co podyktowane było brakiem użycia datasetów opartych o język polski.

8.2. Zastosowania

Przedstawione w niniejszej pracy rozwiązanie, znaleźć może zastosowanie w wielu aspektach życia. Przykładem tej aplikacji jest użycie jej w celu badań i nauki wśród ludzi ze spektrum autyzmu. Takie osoby często nie są w stanie stwierdzić jaką emocję wyrażają w danym momencie, co wizualizować może proponowane przez pracę rozwiązanie. Dzięki korzystaniu z tej aplikacji umożliwiliby to im naukę wzorców emocjonalnych. Kolejnym możliwym wykorzystaniem opisywanego modułu jest integracja z systemami HCI. Takie połączenie jest w stanie umożliwić stworzenie wielu aplikacji. Przykładem takiego zastosowania mogą być asystenci głosowi w module smart house, którzy na podstawie emocji wydobytych z głosu będą w stanie zaproponować np. włączenie ulubionej piosenki w przypadku wykrycia smutku w głosie. Na podstawie tego zastosowania, wykorzystanie HCI z detekcją emocji użyte może być w pomieszczeniach psychiatrycznych do alarmowania pracowników jednostki w celu wczesnego reagowania przy wykryciu ataków lękowych. Kolejnym użyciem tego modułu mogą być gry wideo, które polegają na komunikacji werbalnej z maszyną. Przykładem takiej aplikacji może być gra, w której wcielamy się w detektywa i przeprowadzamy śledztwo wśród postaci, będących w stanie reagować na zmianę stanu emocjonalnego w naszym głosie, np. irytację.

8.3. Możliwy dalszy rozwój

Przedstawiona powyżej aplikacja posiada wiele aspektów, które są w stanie usprawnić jej działanie. Pierwszym krokiem, który można poczynić jest stworzenie modelu umożliwiającego detekcję emocji dla

krótszych ramek czasowych. Pozwoliłoby to na bardziej dokładne śledzenie zmian stanu emocjonalnego w czasie wypowiedzi. Kolejnym krokiem jest rozwinięcie zbioru danych o próbki innych języków, co umożliwiłoby wielojęzyczną detekcję emocji, gdzie obecne rozwiązanie zwraca poprawne wyniki jedynie dla wypowiedzi w języku angielskim. Obecny dataset powinien być również zaopatrzony w zbiór plików audio, nagranych na lokalnej maszynie w celu dostrojenia wykrywania emocji, gdyż rozwiązania te nie są przygotowane na nagrywanie w słabej jakości.

SPIS RYSUNKÓW

2.1	Kołowy uporządkowany podział ośmiu deskryptorów afektywnych	12
2.2	Rozpięta paraboloida emocji Plutchika	13
3.1	Wykres obrazujący ramkowanie [21]	17
3.2	Architektura LPC [27]	20
3.3	Proces metody PLP [27]	21
3.4	Proces ekstrakcji cech przy użyciu MFCC [27]	23
4.1	Schemat jedno-jednostkowej rekurencyjnej sieci neuronowej	28
4.2	Architektura komórki LSTM zawierająca bramkę zapominania	30
4.3	Architektura enkodera modelu transformer [48]	31
4.4	Architektura dekodera modelu transformer [48]	32
5.1	Schemat rozwiązania [49]	35
5.2	Schemat modelu CNN [52]	37
5.3	Schemat blokowy propozycji rozwiązania problemu SER z użyciem LSTM [54]	38
6.1	Procentowy podział zebranych plików audio w zależności od emocji i płci	41
6.2	Schemat modelu "opartego o CNN"	43
6.3	Proces uczenia modelu "opartego o CNN"	44
6.4	Schemat modelu "opartego o LSTM"	45
6.5	Proces uczenia modelu "opartego o LSTM"	46
6.6	Schemat modelu "opartego o MFCC"	48
6.7	Proces uczenia modelu "opartego o MFCC"	49
6.8	Schemat modelu opartego o transformer	51
6.9	Schemat bloku Layers	51
6.10	Aplikacja webowa EMODET	52
7.1	Funkcja dokładności i straty podczas uczenia modelu "opartego o CNN"	53
7.2	Funkcja dokładności i straty podczas uczenia modelu "opartego o LSTM"	54
7.3	Funkcja dokładności i straty podczas uczenia modelu "opartego o MFCC"	55
7.4	Funkcja dokładności i straty dla modelu opartego o transformer	55
7.5	Macierz pomyłek model "oparty o CNN"	57
7.6	Macierz pomyłek model "oparty o LSTM"	57
7.7	Macierz pomyłek model "oparty o MFCC"	57
7.8	Macierz pomyłek model "oparty o transformer"	57
1	Schemat modelu "opartego o CNN"	70

2	Schemat modelu "opartego o LSTM"	71
3	Schemat modelu "opartego o MFCC"	72
4	Schemat modelu opartego o transformer	73
5	Schemat bloku Layers	73

SPIS TABEL

7.1 Dokładność modeli na własnych zbiorach testowych	56
7.2 Wyniki testu detekcji w języku angielskim przeprowadzonego na każdym z modeli	56

BIBLIOGRAFIA

- 
- [1] Wohr J.: *Voice assistants: What they are and what they mean for marketing and commerce*, 2023, <https://www.insiderintelligence.com/insights/voice-assistants/> (data dostępu 14.11.2023r.).
- 
- [2] Vieira A. D., Leite H., Lachowski Volochchuk A. V.: *The impact of voice assistant home devices on people with disabilities: A longitudinal study*, 2022, <https://www.sciencedirect.com/science/article/abs/pii/S0040162522004826> (data dostępu 16.11.2023r.).
- [3] American Psychological Association: *APA Dictionary of Psychology: emotion*, 2018, <https://dictionary.apa.org/emotion> (data dostępu 21.11.2023r.).
- [4] Burton N.: *What are basic emotions?*, 2016, [https://www.psychologytoday.com/us/blog/hide-and-seek/201601/what-are-basic-emotions#:~:text=Basic%20emotions%20\(or%20%22affect%20programs,with%20a%20high%20survival%20value.](https://www.psychologytoday.com/us/blog/hide-and-seek/201601/what-are-basic-emotions#:~:text=Basic%20emotions%20(or%20%22affect%20programs,with%20a%20high%20survival%20value.) (data dostępu 21.11.2023r.).
- [5] Ekman P.: *Basic emotions.*, Handbook Cognit Emotion, 1999.
- [6] Koolagudi, Shashidhar G., Sreenivasa Rao K.: *Emotion recognition from speech: a review.*, International journal of speech technology 15, 2012, s. 99-117.
- [7] Russell J. A., Pratt G.: *A description of the affective quality attributed to environments.*, Journal of personality and social psychology 38.2, 1980, s. 311.
- [8] Plutchik R.: *Emotion.*, A psychoevolutionary synthesis, 1980.
- [9] Sapinski T., Kamińska D.: *Emotion recognition from natural speech – emotional profiles.*, 2015.
- [10] Wirsing K.: *Time Frequency Analysis of Wavelet and Fourier Transform*, 2020, <https://www.intechopen.com/chapters/74096> (data dostępu 1.01.2024r.).
- [11] Podstawowe informacje o dźwięku, <https://www.mega-acoustic.pl/podstawowe-informacje-o-dzwieku/> (data dostępu 12.12.2023r.).
- [12] *Sygnały*, Politechnika Rzeszowska http://mikrosys.prz.edu.pl/SPWF_pliki/Sygna%C5%82y.pdf
- [13] Sircar P. *Parametric Modeling of Non-Stationary Signals* Indian Institute of Technology Kanpur, <https://arxiv.org/ftp/arxiv/papers/1801/1801.09045.pdf>
- [14] *What is sound?*, <https://www.pasco.com/products/guides/sound-waves> (data dostępu 12.12.2023r.).

- [15] Bos S.: *What Are Decibels, and How Are They Measured?*, 2023, <https://science.howstuffworks.com/question124.htm> (data dostępu 26.12.2023r.).
- [16] *Characteristics of Sound Wave.*, 2022, <https://www.turito.com/learn/physics/characteristics-of-sound-wave-grade-8> (data dostępu 12.12.2023r.).
- [17] Banse R., Scherer K. R.: *Acoustic profiles in vocal emotion expression*, 1996, <https://psycnet.apa.org/doiLanding?doi=10.1037%2F0022-3514.70.3.614> (data dostępu 10.11.2023r.).
- [18] Kwasny D., Hemmerling D.: *Gender and Age Estimation Methods Based on Speech Using Deep Neural Networks*, Sensors (Basel), 2021, <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8309811/> (data dostępu 1.12.2023r.).
- [19] Pedersen M.: *Understanding Emotional Prosody Unveiling Human Emotion in Speech*, 2023, <https://imotions.com/blog/insights/research-insights/understanding-emotional-prosody-unveiling-human-emotion-in-speech/> (data dostępu 18.11.2023r.).
- [20] Rayden M.: *What is RMS in audio world?*, 2023, <https://majormixing.com/what-is-rms-in-audio-world/> (data dostępu 28.11.2023r.).
- [21] Malek A.: *Signal framing*, 2020, https://superkogito.github.io/blog/2020/01/25/signal_framing.html (data dostępu 04.07.2023r.)
- [22] Kramarczyk M.: *Analiza dźwięku - funkcje okien i ich właściwości*, https://mkramarczyk.zut.edu.pl/?cat=M&l=D_07 (data dostępu 12.12.2023r.).
- [23] Szwoch G.: *Analiza dźwięków muzycznych*, Politechnika Gdańska, Katedra Systemów Multimedialnych, <https://sound.eti.pg.gda.pl/student/akmuz/02-Analiza.pdf> (data dostępu 12.12.2023r.).
- [24] Bachu R.G., Kopparthi S., Adapa B., Barkana B.D.: *Voiced/Unvoiced Decision for Speech Signals Based on Zero-Crossing Rate and Energy*, University of Bridgeport, https://www.researchgate.net/publication/259823741_VoicedUnvoiced_Decision_for_Speech_Signals_Based_on_Zero-Crossing_Rate_and_Energy (data dostępu 28.12.2024r.).
- [25] Arý B., Siddique K., Alçin Ö., Aslan M., Şengür A., Mehmood R. M.: *Discrete Wavelet Transform coefficients for emotion recognition from EEG signals*, 2012, <https://ieeexplore-ieee-org-1000007dp0081.han.bg.pg.edu.pl/document/6346410> (data dostępu 19.11.2023r.).
- [26] Abinaya B., Abishai C., Ashika S.: *Speech Emotion Classification Using DWT and Neural Network*, Hindusthan College of Engineering and Technology, Coimbatore, Tamilnadu, India, 2021, <https://nebula.wsimg.com/bec792d1c08ff22048938cf4c13bbbc2?AccessKeyId=DFB1BA3CED7E7997D5B1&disposition=0&alloworigin=1> (data dostępu 15.12.2023r.).
- [27] Mazumder M. A.: *Feature Extraction Techniques for Speech Processing: A Review*, 2019, <https://dx.doi.org/10.30534/IJATCSE/2019/5481.32019> (data dostępu 19.11.2023r.).

- [28] Politechnika Gdańska: *Metoda Durbina-Levinsona - iteracyjne wyznaczanie współczynników filtra traktu głosowego*, <https://eti.pg.edu.pl/documents/176770/34413920/Durbin.pdf> (data dostępu: 16.12.2023r.).
- [29] O'Shaughnessy D.: *Review of methods for coding of speech signals*, 2023, <https://doi.org/10.1186/s13636-023-00274-x> (data dostępu 26.12.2023r.).
- [30] Kain A., Macon M. W.: *Spectral voice conversion for text-to-speech synthesis*, Proceedings of the 1998 IEEE International Conference on Acoustics, 1998, s. 285-288.
- [31] Drobner M., Golachowski S.: *Akustyka muzyczna*. Polskie Wydawnictwo Muzyczne, 1953, s. 62–63.
- [32] *Perceptual Linear Prediction*, Texas Instruments Institute for Signal and Information Processing, 1996, https://isip.piconepress.com/courses/msstate/ece_8463/lectures/2000_spring/lecture_06/lecture_06_0.pdf (data dostępu 14.12.2023r.).
- [33] Kacprzak S.: *Inteligentne metody rozpoznawania dźwięku*, Politechnika Warszawska.
- [34] McCarthy J.: *What Is Artificial Intelligence?*, Computer Science Department, Stanford University, 2007, <https://www-formal.stanford.edu/jmc/whatisai.pdf> (data dostępu 16.11.2023r.).
- [35] Turing A. M.: *Computing Machinery and Intelligence.*, Mind, 1949, s. 433-460.
- [36] Mitchell T. M.: *Machine learning.*, 1997.
- [37] Dreyfus G.: *Neural networks: methodology and applications.*, Springer Science & Business Media, 2005.
- [38] Chen Z., Feng X., Zhang S.: *Emotion detection and face recognition of drivers in autonomous vehicles in IoT platform.*, Image and Vision Computing, 2022.
- [39] Hubel D. H., Wiesel T. N.: *Receptive fields of single neurones in the cat's striate cortex.*, The Journal of physiology, 1959, s. 574.
- [40] LeCun Y.: *A theoretical framework for back-propagation.*, Proceedings of the 1988 connectionist models summer school, 1988.
- [41] LeCun Y., Bengio Y.: *Convolutional networks for images, speech, and time series.*, The handbook of brain theory and neural networks, 1995.
- [42] Mamczur M.: *Jak działają konwolucyjne sieci neuronowe (CNN)?*, 2021, <https://miroslawmamczur.pl/jak-dzialaja-konwolucyjne-sieci-neuronowe-cnn/> (data dostępu 20.11.2023r.).
- [43] Amidi A., Amidi S.: *Recurrent Neural Networks cheatsheet*, 2018, <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks> (data dostępu 09.07.2023r.).

- [44] Hochreiter S., Schmidhuber J.: *Long short-term memory*, Neural computation, 1997, s. 1735-1780.
- [45] Gers F. A., Schmidhuber J., Cummins F.: *Learning to forget: Continual prediction with LSTM.*, Neural computation, 2000, s. 2451-2471.
- [46] Vaswani A. i inni.: *Attention Is All You Need*, Google Brain, 2017, <https://arxiv.org/abs/1706.03762> (data dostępu 28.11.2023r.).
- [47] Thomas E. B.: *A Clear Explanation of Transformer Neural Network*, 2023, <https://www.linkedin.com/pulse/clear-explanation-transformer-neural-networks-ebin-babu-thomas> (data dostępu 1.01.2024r.).
- [48] Jia Y.: *Attention Mechanism in Machine Translation*, 2019, <https://iopscience.iop.org/article/10.1088/1742-6596/1314/1/012186> (data dostępu 30.12.2023r.).
- [49] Aouani H., Ayed B. Y.: *Emotion Recognition Using Speech Analysis and Machine Learning Techniques*, Procedia Computer Science, 2020, <https://www.sciencedirect.com/science/article/pii/S1877050920318512> (data dostępu 28.11.2023r.).
- [50] Zhao J., Mao X., & Chen L.: *Speech emotion recognition using deep 1D & 2D CNN LSTM networks*, Biomedical signal processing and control, 2019.
- [51] Dias I., Demirci M. F., Yazici A.: *Speech emotion recognition with deep convolutional neural networks*, Biomedical Signal Processing and Control, 2020.
- [52] Qayyum A. B. A., Arefeen A., Shahnaz C.: *Convolutional neural network (CNN) based speech-emotion recognition*. 2019 IEEE international conference on signal processing, information, communication & systems (SPICSCON), IEEE, 2019.
- [53] Al-onazi B. B. i inni.: *Transformer-Based Multilingual Speech Emotion Recognition Using Data Augmentation and Feature Fusion*, 2022 <https://www.mdpi.com/2076-3417/12/18/9188> (data dostępu 02.12.2023r.).
- [54] Al-Hussain M. M., Turki-Hadj Alouane M.: *Speech Emotion Recognition based on Long Short Term Memory network*, IEEE, 2022.
- [55] Livingstone S.R., Russo F.A.: *The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS): A dynamic, multimodal set of facial and vocal expressions in North American English*. Department of Psychology, Ryerson University, Toronto, Canada; Department of Computer Science and Information Systems, University of Wisconsin-River Falls, Wisconsin, WI, United States of America, 2018, <https://doi.org/10.1371/journal.pone.0196391> (data dostępu 16.11.2023r.).
- [56] Dupuis K., Pichora-Fuller M. K.: *Toronto emotional speech set (TESS)*, University of Toronto, Psychology Department, 2010, <https://tspace.library.utoronto.ca/handle/1807/24487> (data dostępu 28.11.2023r.).

- [57] DavidCooperWCU: *Crowd Sourced Emotional Multimodal Actors Dataset (CREMA-D)*, 2014, <https://github.com/CheyneyComputerScience/CREMA-D> (data dostępu 28.11.2023r.).
- [58] Lok E. J.: *Surrey Audio-Visual Expressed Emotion (SAVEE)*, 2015, <https://www.kaggle.com/datasets/ejlok1/surrey-audiovisual-expressed-emotion-savee> (data dostępu 28.11.2023r.).
- [59] Hamid M. A.: CNN Model, 2023, <https://www.kaggle.com/code/mostafaabdlhamed/speech-emotion-recognition-97-25-accuracy#CNN-model> (data dostępu 30.11.2023r.).
- [60] Dokumentacja biblioteki Librosa, <https://librosa.org/doc/main/index.html> (data dostępu 29.11.2023r.).
- [61] S-Gemmin: LSTM Model, <https://github.com/S-Gemmin/speech-emotion-recognition/blob/98a578ba29dac1e4d68750230a2179ea3be1b4a2/speech-emotion-recognition-90.ipynb>(data dostępu 30.11.2023r.).
- [62] Hussien A.: Speech emotion recognition model, <https://www.kaggle.com/code/abdullahhussien/speech-emotion-recognition-93-accuracy> (data dostępu 01.12.2023r.).
- [63] Dokumentacja biblioteki Wav2Vec, https://huggingface.co/docs/transformers/model_doc/wav2vec2 (data dostępu 02.12.2023r.).
- [64] Dokumentacja modelu HuBERT, https://huggingface.co/docs/transformers/model_doc/hubert (data dostępu 02.12.2023r.).
- [65] Dokumentacja Flask, <https://flask.palletsprojects.com/en/3.0.x/> (data dostępu 02.12.2023r.).
- [66] Dokumentacja Next.js, <https://nextjs.org/> (data dostępu 02.12.2023r.).
- [67] Dokumentacja React.js, <https://legacy.reactjs.org/docs/getting-started.html>(data dostępu 02.12.2023r.).



A. OPIS DYPLOMU

Celem pracy jest stworzenie aplikacji umożliwiającej detekcję emocji z nagrań live i plików audio oraz porównanie czterech wytrenowanych modeli. Do teraz powstało wiele propozycji rozwiązań problemu SER (ang. *Speech Emotion Recognition*), lecz nie znaleziono jeszcze rozwiązania w tej dziedzinie. W pracy wykorzystano modele oparte o: CNN, LSTM, CNN z użyciem do ekstrakcji emocji tylko techniki MFCC, a także sieci typu transformer. Do ich uczenia wykorzystano ogólnodostępne zbiory danych: RAVDESS, CREMA-D, SAVEE, a także TESS.

Zarys teoretyczny

Emocje w historii dzielono na wiele sposobów. Russel zaproponował dla przykładu model kołowy, gdzie emocje dzieli się za pomocą wymiarów: przyjemność - nieprzyjemność, pobudzenie - ospałość. Kolejną propozycję podziału przedstawił Plutchik, gdzie uznał on siedem podstawowych emocji: radość, zaufanie, strach, zaskoczenie, smutek, wstręt, gniew oraz oczekiwanie i przedstawił je na rozpiętej paraboloidzie. Niestety żadne z wyżej wymienionych podziałów nie nadawały się do opisu emocji dla modeli wykrywających je, w odróżnieniu do propozycji podstawowych emocji Ekmana. Ustanowił on pięć podstawowych emocji: złość, obrzydzenie (wstręt), szczęście, smutek oraz strach. Model przez niego przedstawiony ze względu na swoją prostotę definicji i zastosowania uznawany jest w tego typu pracach jako podstawowy. Do zbioru tych emocji zalicza się także emocję neutralną (spokój), ze względu na potrzebę uznania emocji, w której nie wykazuje się cech emocji podstawowych. W ten sposób otrzymujemy zbiór sześciu emocji używanych często w modelach wykrywania emocji.

Dźwięk jest zaburzeniem mechanicznym w ośrodku sprężystym, gazowym, ciekłym lub stałym (fale sprężyste) wywołującym subiektywne wrażenie słuchowe u człowieka lub zwierząt [11]. Natomiast głos jest zjawiskiem akustycznym wynikającym z drgań fałd głosowych w krtani, wzmacnianych i kształtowanych przez jamę ustną, gardło oraz zatoki nosowe. Do jego cech należą: tempo mowy, natężenie głosu, rytm, a także energia. Aby przeanalizować i wyciągnąć te cechy stosuje się: RMSE, które służy jako miara kwantyfikacji średniej mocy sygnału; ramkowanie, które dzieli sygnały na mniejsze bloki, aby umożliwić ekstrakcję cech; okienkowanie, dzięki któremu zapobiega się zjawisku przecieku widma, powstającemu podczas ramkowania; ZCR, które zlicza przejścia sygnału przez poziom zero i zwraca ich liczbę jako jedną z właściwości sygnału; DWT, używane często do analizy w dziedzinie czasu i skali; LPC, które przybliża próbki dźwięku jako kombinację poprzednich próbek; PLP, które ma na celu odrzucenie nieistotnych informacji (szumy, dźwięki niepodobne do głosu); MFCC, najpopularniejszą metodę, która opisuje ogólny kształt dźwięku wykorzystując skalę melową i cepstrum.

W problemie detekcji emocji najczęściej tworzone są propozycje opierające się na mechanizmach sztucznej inteligencji, które zaliczają się do uczenia maszynowego z nadzorem, dokładnie sieciach neuronowych. Sieci neuronowe są systemami przetwarzania informacji, które inspirowane są działaniem ludzkiego mózgu, gdzie ich podstawowym elementem jest sztuczny neuron. Każda z sieci ma inną architekturę i występujące w niej elementy. Konwolucyjne sieci neuronowe charakteryzują się po-

siadaniem warstw splotowych. LSTM posiada komórkę pamięci, która umożliwia radzenie sobie z długotrwałymi zależnościami. Natomiast transformer posiada warstwy enkodera i dekodera, a także blok *multi-head attention*. Posiadają one wiele zastosowań w szczególności używane są przy analizie obrazów, a także przetwarzaniu języka naturalnego.

Przegląd rozwiązań

Aby lepiej zapoznać się z problemem SER przeprowadzony został także przegląd prac, gdzie w szczególności skupiono się na trzech, które wykorzystywały sieci CNN, LSTM, a także sieć typu transformer. Pierwsza z nich, model wykorzystujący sieć CNN, wytrenowany został z użyciem datasetu SAVEE [52]. Złożony był on z siedmiu jednowymiarowych warstw splotowych, o ilości filtrów kolejno: 32, 64, 128, 512, 1024, 1024; i rozmiarach jąder: 21, 19, 17, 15, 13, 11, 9. Emocje wykrywanych w tym modelu było 7: szczęście, strach, smutek, złość, wstręt, zaskoczenie, emocja neutralna. Ilość epok trenowania ustawiono w nim na 400. Na danych testowych składających się z 80 plików wykazał dokładność równą 83,61%

Model wykorzystujący LSTM w pracy [54], użył podczas procesu trenowania zbiorów danych RML, który posiadał sześć emocji. Na architekturę tego systemu składały się: blok ekstrahujący pliki audio w częstotliwości próbkowania 16000Hz, a także klasyfikacji LSTM. Model trenowano przez 4000 epok i w najlepszym przypadku (*timestamp* równe 10) wykazał dokładność 97%.

Artykuł [53] porównał model bazujący na sieci typu transformer do innych standardowych rozwiązań. Podczas treningu wykorzystano tam datasety BAVED, SAVEE, EMOVO, a także EMO-DB. W propozycji podzielono emocje w zależności od datasetu na którym był uczony model. Do ekstrakcji cech wykorzystano takie mechanizmy jak: mel-spektrogram, chromatogram, MFCC, Δ MFCC, $\Delta\Delta$ MFCC, reprezentacja Tonnetz oraz kontrast spektralny. Łącznie wyekstrahowanych cech z każdego pliku otrzymano 273. Wyniki przeprowadzonych testów dla różnych zbiorów danych wynosiły: dla BAVED 95% dokładności, EMO-DB 93% dokładności, SAVEE 85% dokładności i EMOVO 91% dokładności.

Użyte zbiory danych

W naszej pracy do uczenia każdego z modeli użyto jeden wielki zbiór danych, który składał się z: RAVDESS, TESS, CREMA-D i SAVEE. Pierwszy z nich, *Ryerson Audio-Visual Database of Emotional Speech and Song* zawiera łącznie 1440 plików audio, w których znajdują się emocje zagrane przez 24 aktorów. Pliki podzielone w tym zbiorze na 7 emocji: szczęście, smutek, złość, strach, zaskoczenie, wstręt oraz emocję neutralną [55]. *Toronto Emotional Speech Set* to zbiór posiadający 2800 plików audio, gdzie zagrane emocje zostały przez dwie aktorki. Emocje podzielono w nim tak samo jak w przypadku RAVDESS. Kolejny *Crowd-sourced Emotional Multimodal Actors Dataset* składa się z 7442 plików audio nagranych przez 91 aktorów. Emocje podzielono w nim na sześć emocji, gdzie w porównaniu do poprzednich zbiorów nie występuje emocja zaskoczenia. Ostatni *Surrey Audio-Visual Express Emotion* (SAVEE) to zbiór nagrań czterech mężczyzn. Łącznie znajduje się w nim 480 plików, przy czym emocje grane przez nich podzielone były na siedem etykiet.

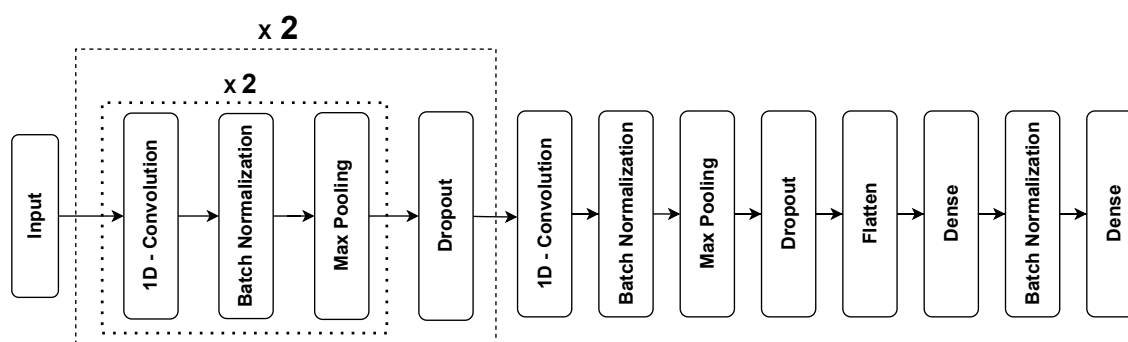
Emocje w naszym przypadku podzielone zostały na sześć etykiet: szczęście, smutek, złość,

wstręt, strach (emocje podstawowe wg Ekmana) oraz spokój (emocję neutralną). Z tego względu usunięte zostały nagrania, gdzie grana była emocja zaskoczenie i w ten sposób nasz zbiór danych zawierał ostatecznie 11510 nagrań.

Model oparty o CNN

Model oparty o CNN jest propozycją rozwiązania problemu SER zaprezentowaną w [59]. Dane dla tego modelu, przed podzieleniem na zbiór uczący i testowy, przeszły augmentację za pomocą czterech funkcji: *noise*, która dodawała szum; *stretch*, gdzie następowała zmiana tempa nagrania; *shift*, która przesuwała dane dźwiękowe w czasie; *pitch*, po której następowała zmiana tonacji. Dzięki temu otrzymaliśmy znacznie większy zbiór danych, na który składało się łącznie 46040 nagrań. Po tym przeprowadzono ekstrakcję cech używając do tego ZRC, RMSE oraz MFCC, gdzie otrzymane wartości liczbowe wraz z etykietą emocji zapisano w wektory. Podzielono następnie te wektory na zbiór uczący stanowiący 80% oraz testowy 20%.

Architektura modelu widoczna na rysunku 1 składa się z pięciu warstw konwolucyjnych z funkcją aktywacji ReLU, których ilość filtrów wynosi kolejno: 512, 512, 256, 256, 128; oraz wielkość okien splotu dla kolejnych warstw wynosi: 5, 5, 5, 3, 3. Po każdej z warstw splotowych występują warstwy *Batch Normalization* i *Max Pooling*. Po drugiej, czwartej oraz piątej warstwie *Max Pooling* ustawiono warstwy *Dropout* o częstotliwości równej 0,2. Następnie użyto warstwy: *Flatten*, *Dense* o parametrach: 512 filtrów i funkcji aktywacji ReLU, *Batch Normalization* oraz *Dense* z 6 jednostkami (6 etykiet emocji).



Rysunek 1: Schemat modelu "opartego o CNN"

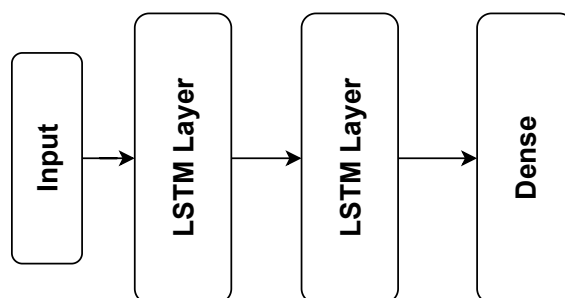
Jako dane wejściowe do modelu wprowadzono zbiór wektorów o rozmiarze (36832, 2376). Do kompilacji modelu użyto optymalizatora *adam* i funkcji straty *categorical_crossentropy*. Proces trenowania ustawiono na 50 epok i ilość próbek przetwarzanych naraz na 64. W trakcie trenowania działały również funkcje przedwczesnego zatrzymania oraz redukcji szybkości uczenia w razie braku poprawy wyników w kolejnych epokach.

Model ten dla na zbiorze testowym (9208 nagrań) wykazał skuteczność równą 98,22%. W teście dokładności w języku angielskim, gdzie na wejście modelu podano 11 nagrań łączących każdą z sześciu emocji, nie popełnił ani jednego błędu.

Model oparty o LSTM

Model oparty o LSTM jest propozycją rozwiązania ukazaną w [61]. W naszej pracy jednak nie został on wyuczony w taki sam sposób, lecz proces ten wykonany został na przygotowanym przez nas zbiorze danych, czyli dokładnie 11510 nagraniach. Dane po połączeniu w jeden zbiór przeszły preprocessing i przeprowadzono ekstrakcję cech za pomocą ZCR, RMSE i MFCC. Otrzymane wektory cech podzielono następnie na dane treningowe stanowiące 88% i dane testowe 12%.

Na wejście modelu wprowadzono zatem zbiór wektorów o rozmiarze (10128, 15). Architektura modelu jest bardzo prosta, jak widać na rysunku 2, tzn. występują tylko dwie warstwy LSTM, obydwie zawierające 64 jednostki. Po nich znajduje się warstwa *Dense*, która składa się z 6 jednostek, ze względu na taką ilość wykrywanych emocji.



Rysunek 2: Schemat modelu "opartego o LSTM"

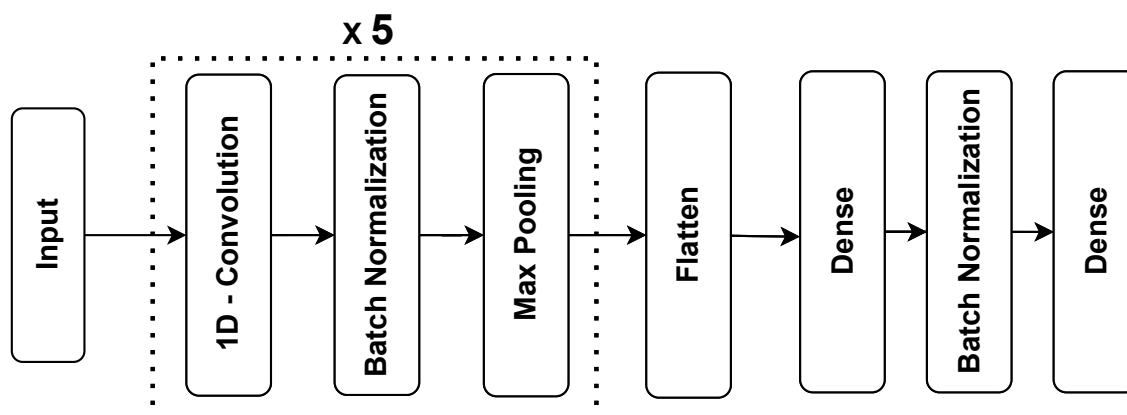
Przy kompilacji użyto entropię krzyżową oraz optymalizator RMSProp. Trenowanie ustawiono na 300 epok, a ilość naraz przetwarzanych próbek na 6. Dla danych testowych (967 nagrań) model wykazał skuteczność 64,68%. Dla testu na nagraniach zawierających każdą z emocji, wynik ten okazał się być lepszy w okolicach 83,33%.

Model oparty o MFCC

Model oparty o MFCC, przedstawiony w [62], stanowi ostatni z zapożyczonych propozycji rozwiązań. W tym przypadku oprócz funkcji augmentacji wykorzystanych w modelu opartym o CNN użyto dodatkowo: *higher_speed*, które zwiększało tempo nagrania; oraz *lower_speed*, które analogicznie zmniejszało tempo. Do ekstrakcji cech użyto tylko MFCC o ilości zwracanych współczynników 58. Dzięki zastosowaniu tych funkcji otrzymano 80570 wektorów nagrań, które następnie podzielono na dane treningowe stanowiące 80% oraz testowe 20%.

Na wejście do trenowania modelu wprowadzono zbiór wektorów o rozmiarze (64456, 58, 1). Jego architektura, widoczna na rysunku 3, składa się z pięciu warstw konwolucyjnych. Posiadają one ilość filtrów kolejno: 512, 512, 256, 256, 128; oraz rozmiar jąder: 5, 5, 5, 3, 3. Każda z nich używała jako funkcję aktywacji funkcję ReLU. Po każdej z nich użyto warstwy *Batch Normalization* oraz *Max Pooling*. Następnie zastosowano warstwy: *Fatten*, *Dense* o ilości neuronów 512 i funkcji aktywacji ReLU, *Batch Normalization* oraz *Dense* o ilości jednostek równej 12, ze względu na podział na 6 emocji oraz płeć.

Do kompilacji użyto optymalizator *RMSProp*, a jako funkcję straty użyto jak we wcześniejszym modelach entropię krzyżową. Dla procesu trenowania ustawiono ilość epok równą 50 i ilość naraz prze-



Rysunek 3: Schemat modelu "opartego o MFCC"

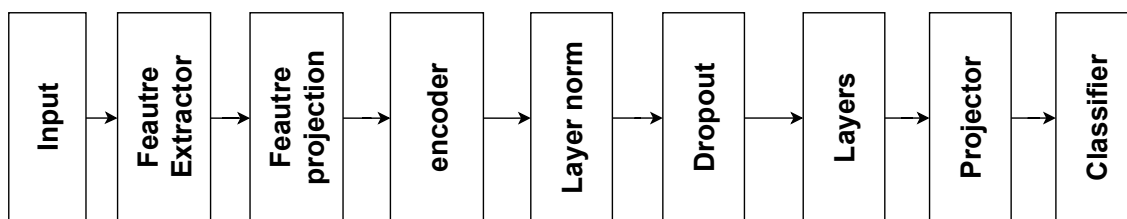
tworzonych próbek jako 64. Dla danych testowych (16114 nagrań) model ten wykazał się skutecznością na poziomie 92,47%. Przy teście na nagraniach zawierających każdą z emocji nie wykonał ani jednej błędnej predykcji.

Model oparty o transformer (Jakub Bieńkowski)

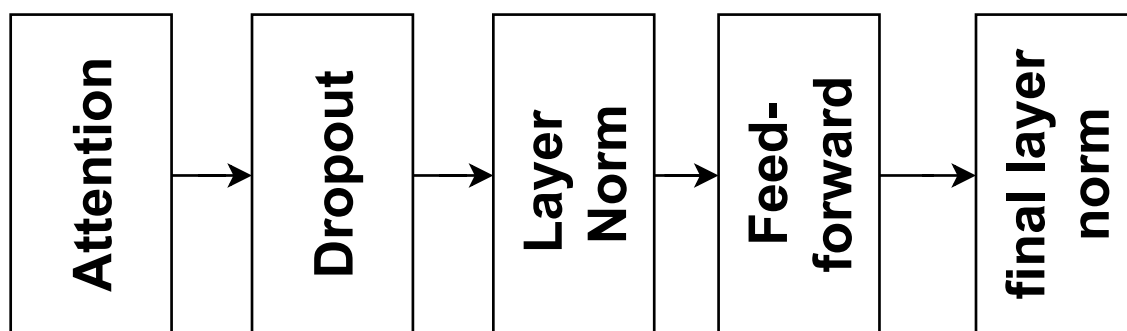
Model oparty o transformer stanowi własną propozycję rozwiązania problemu SER. Zbiór danych przygotowany dla tego modelu jest złożony z tej samej liczby nagrań co model LSTM. Na dane treningowe przypadło 80% wszystkich próbek, a na zbiór testowy przypadła reszta. Do ekstrakcji cech z nagrań użyto *Wav2Vec2FeautreExtracor*, który tworzy nieokreślone cechy głosu z nieprzetworzonego sygnału wejściowego. Cechy te później są etykietowane przy pomocy przetrenowanego modelu "superb/hubert-large-superb-er".

Na wejście modelu przedstawionego na rysunku 4 zostają prowadzone przetworzone odpowiednio dane. Blok *Feature Extractor* składa się z 7 warstw konwolucyjnych, gdzie dla każdej z nich dokładane są warstwy normalizacji o rozmiarze wejściowym równym 512 oraz funkcja aktywacji GELU. Pierwsza warstwa konwolucyjna ma wymiary (1, 512), rozmiar jądra równy 10 i przesunięciem równym 5. Kolejne 4 warstwy są identyczne i wymiary są równe (512, 512), rozmiar jądra wynosi 3, a przesunięcie jest równe 2. Ostatnie dwie warstwy mają identyczny wymiar co 4 wymienione wyżej, ale ich rozmiar jądra i przesunięcie są równe 2. Warstwa *Feature projection* posiada warstwę normalizacji o wymiarze wejściowym równym 512, warstwę liniową rozszerzającą o wymiarach (512,1024) i warstwę dropout. Blok *enkoder* składa się z konwolucyjnej warstwy splotowej o wymiarach (1024,1024) rozmiarze jądra równym 128, przesunięciu o wartości 1, dopełnieniu równym 64. Blok ten również zawiera normalizację wag i dodatkowe dopełnienie. Na koniec bloku stosowana jest GELU. Blok *Layers* jest bardziej złożony i jego schemat został przedstawiony na rysunku 5. Składa się on z 24 warstw enkodera. Na każdą z tych warstw składają się 4 warstwy liniowe o wymiarach (1024,1024). Po nich następuje dropout i warstwa normalizacji o wymiarze wejściowym równym 1024. Blok *Feed-forward* składa się z liniowej warstwy o wymiarze (1024, 4096), po której następuję aktywacja GELU. W tym bloku jest również kolejna warstwa liniowa o wymiarach (4096, 1024), po której następuję dropout. Na koniec tego bloku następuję warstwa

normalizacji o wymiarze wejściowym 1024. Po bloku *Layers* przechodzimy do bloków *Projector* i *Classifier*, które zawierają warstwy liniowe odpowiednio o wymiarach (1024, 256) i (256, 7). Liczba wszystkich parametrów treningowych dla tego modelu wynosi 13742198.



Rysunek 4: Schemat modelu opartego o transformer



Rysunek 5: Schemat bloku Layers

W przypadku tego modelu został wybrany optymalizator Adam. *torch.softmax* do uzyskiwania prawdopodobieństwa wykrywania emocji. Proces uczenia składał się z 24 epok, *batch-size* został ustalony jako 8, a dane treningowe zostały pomieszczone. Dla danych testowych model uzyskał dokładność równą 88,40%, a w przypadku nagraniach zawierających każdą z emocji wykrył poprawnie emocje dla 57 próbek z 66.

Podsumowanie

Nasza praca opisuje problem detekcji emocji z głosu i porównuje cztery modele oparte kolejno o: CNN, LSTM, MFCC oraz sieć typu transformer. Powstałe modele wykorzystywane są w stworzonej przez nas aplikacji, która powstała przy użyciu technologii Flask, Next.js oraz React.js.

Wyniki przeprowadzonych testów ukazały, że największą skuteczność w detekcji emocji z głosu wykazują modele wykorzystujące sieć CNN, model "oparty o CNN" wykazał dokładność 98,22%, a drugi z kolei "oparty o MFCC" miał dokładność równą 92,47%. Autorska propozycja wykazała skuteczność równą 88,40%, co również jest wynikiem ponadprzeciętnym. Najgorzej spisał się model "oparty o LSTM", gdzie wynik dokładności na zbiorze testowym wyniósł tylko 64,68%.

Wynioskowano, że modele te działają najlepiej dla nagrań w języku angielskim. Aby otrzymać lepsze rezultaty w innym języku należałoby przetrenować modele na zbiorach posiadających wystarczającą ilość nagrań w danym języku.

B. INSTRUKCJA DLA UŻYTKOWNIKA (JAKUB BIEŃKOWSKI)

W celu uruchomienia programu, należy zainstalować obsługę języka python dla wersji wyższej niż wersja 3.6. Dodatkowo wymagane jest pobranie środowiska node.js. W celu uruchomienia programu wymagane jest wpisanie w terminal następujących komend:

- Sklonowanie repozytorium `git clone https://github.com/xXMaKuSXx/EMODET.git`
- Nawigacja do folderu o nazwie backend poprzez wpisanie komendy `cd backend`
- Stworzenie środowiska wykonawczego poprzez wpisanie komendy `python -m venv venv` i aktywowanie go komendą `venv\Scripts\activate.bat`
- Instalacja potrzebnych bibliotek do uruchomienia programu odbywa się poprzez komendę `pip install -r requirements.txt`
- Po wpisaniu komendy `python server.py` serwer naszej aplikacji powinien być włączony.
- w celu wizualizacji interfejsu programu trzeba przejść do folderu frontend. Będąc w głównym katalogu projektu wpisujemy w terminal `cd frontend`
- W celu pobrania potrzebnych bibliotek wpisujemy komendę `npm i`
- Ostatnim krokiem jest uruchomienie aplikacji poprzez wpisanie `npm run start`

Po wpisaniu wszystkich komend powinna wyświetlić się strona pod adresem `http://localhost:3000/`

C. INSTRUKCJA DLA PROGRAMISTY (JAKUB MAKOWSKI)

Główną część aplikacji stanowi moduł "server.py", który znajduje się w folderze "backend", gdzie wczytane zostają wszystkie z modeli, kolejno: Transformer, CNN, LSTM, MFCC.

```
1 # Loading and preparing Transformer model
2 model_path = 'models/TRANSFORMERS_model/TRANSFORMERS_MODEL.pth'
3 feature_extractor = Wav2Vec2FeatureExtractor.from_pretrained("superb/hubert-large-superb-er")
4 device = torch.device('cpu' if torch.cuda.is_available() else 'cpu')
5 model = HubertForSequenceClassification.from_pretrained("superb/hubert-large-superb-er")
6 model.load_state_dict(torch.load(model_path, map_location=device))
7 model.to(device)
8 model.eval()
9
10 # Loading and preparing CNN model
11 with open('models/CNN_model/CNN_model.json', 'r') as json_file:
12     cnn_model_json = json_file.read()
13     cnn_model = model_from_json(cnn_model_json)
14 cnn_model.load_weights('models/CNN_model/CNN_model_weights.h5')
15 with open('models/CNN_model/encoder2.pickle', 'rb') as f:
16     encoder2 = pickle.load(f)
17 with open('models/CNN_model/scaler2.pickle', 'rb') as f:
18     scaler2 = pickle.load(f)
19
20 # Loading and preparing LSTM model
21 with open('models/LSTM_MODEL/LSTM_model.json', 'r') as json_file:
22     lstm_model_json = json_file.read()
23     lstm_model = model_from_json(lstm_model_json)
24 lstm_model.load_weights("models/LSTM_MODEL/LSTM_model_weights.h5")
25
26 # Loading and preparing MFCC model
27 with open('models/MFCC_MODEL/MFCC_MODEL.json', 'r') as json_file:
28     loaded_model_json = json_file.read()
29     mfcc_model = model_from_json(loaded_model_json)
30 mfcc_model.load_weights("models/MFCC_MODEL/MFCC_MODEL.h5")
31 with open('models/MFCC_model/encoder2.pickle', 'rb') as f:
32     encoder_mfcc = pickle.load(f)
```

Listing 1: Opis kodu wczytania modeli i ich przygotowania

Gdy użytkownik wybierze jako model do detekcji CNN, aktywuje się funkcja *detect_emotion_live_CNN*, która zwraca następnie odpowiedź i wyświetla ją użytkownikowi.

```
1 @app.route('/detect-emotion-live/model-2')
2 def detect_emotion_live_CNN(duration=3.2, samplerate=22050, channels=1, filename="
    temp_recording.wav"):
3     detected_emotion = "Unknown"
4     try:
```

```

5     myrecording = sd.rec(int(duration * samplerate), samplerate=samplerate, channels
    =channels)
6     sd.wait()
7     sf.write(filename, myrecording, samplerate)
8     detection = prediction_CNN(filename)
9     detected_emotion = detection[0][0]
10    os.remove(filename)
11
12    except Exception as e:
13        print(f"An error occurred while processing audio: {str(e)}")
14
15    return jsonify({'detected_emotion': detected_emotion})

```

Listing 2: Opis kodu predykcji i wysyłania emocji na żywo - model oparty o CNN

Podczas jej działania zapisywane jest audio do pliku *temp_recording.wav* i przechodzi on przez wszystkie niezbędne do wprowadzenia do modelu funkcje. Ekstrahowane są cechy z nagranych audio, następnie następuje detekcja emocji i zwracana zostaje etykieta, która następnie przesyłana jest na front-end, plik jest usuwany i funkcja działa od nowa do momentu aż na serwer nie przyjdzie zapytanie zatrzymujące nagrywanie.

Dla modelu "opartego o LSTM" oraz "opartego o MFCC" działanie jest takie samo. Inaczej trzeba postąpić jednak z modelem "opartym o transformer". W jego przypadku nagrane audio musi być zapisane w bitach, dzięki czemu model może wykonać detekcję i zwrócić prawidłowy wynik na wyjście.

```

1 @app.route('/detect-emotion-live/model-1')
2 def detect_emotion_live_Transformer():
3     detected_emotion = "Unknown"
4     audio_data = np.frombuffer(stream.read(32000), dtype=np.float32)
5
6     try:
7         unique_name = f"temp_{uuid.uuid4()}.txt"
8         audio_path = os.path.join(BASE_DIR, "audio", unique_name)
9         with wave.open(audio_path, 'wb') as wf:
10             wf.setnchannels(1)
11             wf.setsampwidth(pyaudio.PyAudio().get_sample_size(pyaudio.paInt16))
12             wf.setframerate(16000)
13             wf.writeframes(audio_data.tobytes())
14
15         detected_emotion = detect_emotion_from_audio_transformer(audio_path)
16
17         os.remove(audio_path)
18
19     except Exception as e:
20         print(f"An error occurred while processing audio: {str(e)}")
21
22     return jsonify({'detected_emotion': detected_emotion})

```

Listing 3: Opis kodu predykcji i wysyłania emocji na żywo - model oparty o transformer

Aby wyuczyć kolejne modele i wprowadzić je do aplikacji należy przejść przez każdy z notatników dodanych w repozytorium w folderze "backend", wygenerować klucz aplikacji Kaggle, aby móc pobrać datasey. Następnie wykonać wszystkie komendy znajdujące się w kodzie. Zapisać na urządzenie pliki modeli oraz skalara i enkodera. Po tym utworzyć folder "models" w lokacji "backend", utworzyć kolejno foldery dla każdego z modeli i uzupełnić odpowiednimi plikami. Następnie zaktualizować ścieżki do plików.